Claude Code



CLAUDE FOR SOFTWARE ENGINEERS

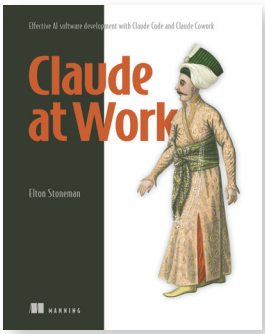
BUILD, TEST, AUTOMATE

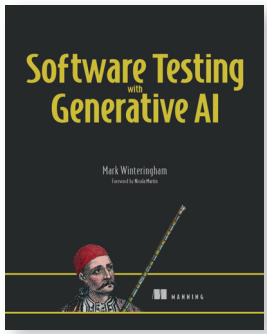
Learn to use Claude Code, translate requirements into staged implementations with behavior-driven development, verify results through AI-assisted testing, and integrate AI into a repeatable software delivery workflow

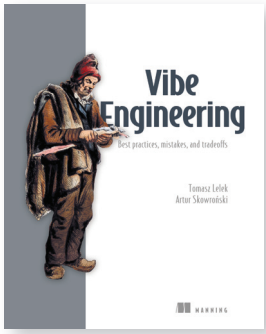
Elton Stoneman
Mark Winteringham
Tomasz Lelek • Artur Skowroński

MANNING

Save 45% on any one of the books listed below by using the code CLAUDE_FSE at checkout.



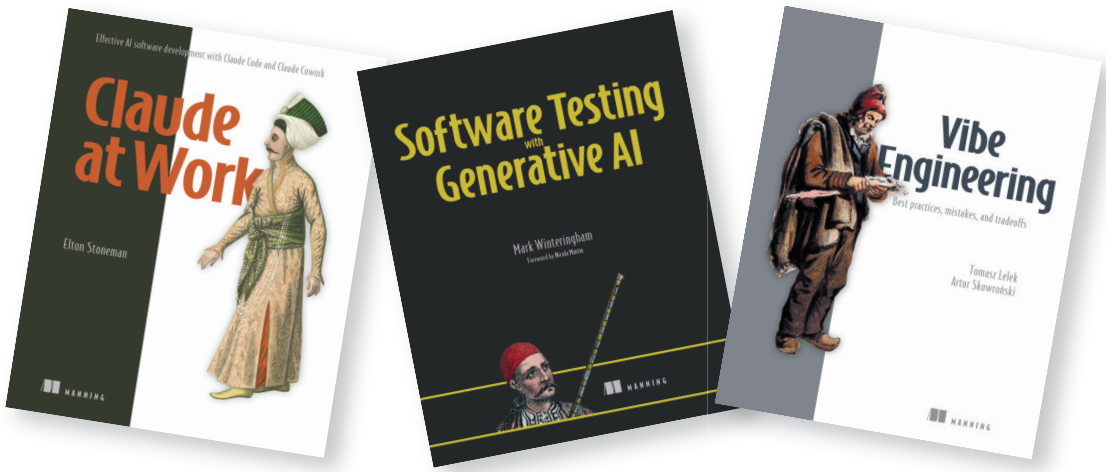




Save 50% on 2 or more of these books, here.



Get all 3 ebooks for $99.99, here.





Claude for Software Engineers

BUILD, TEST, AUTOMATE

Copyright 2027 Manning Publications

For online information and ordering of this and other Manning books, please visit www.manning.com. The publisher offers discounts on this book when ordered in quantity.

For more information, please contact

Special Sales Department
Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964
Email: orders@manning.com

© 2027 Manning Publications Co. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by means electronic, mechanical, photocopying, or otherwise, without prior written permission of the publisher.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in the book, and Manning Publications was awware of a trademark claim, the designations have been printed in initial caps or all caps.

Recognizing the importance of preserving what has been written, it is Manning's policy to have the books we publish printed on acid-free paper, and we exert our best efforts to that end. Recognizing also our responsibility to conserve the resources of our planet, Manning books are printed on paper that is at least 15 percent recycled and processed without the use of elemental chlorine.

The author and publisher have made every effort to ensure that the information in this book was correct at press time. The author and publisher do not assume and hereby disclaim any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from negligence, accident, or any other cause, or from any usage of the information herein.



Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964

ISBN 9781638359296
Printed in the United States of America

contents

introduction *iv*

Getting started with Claude Code 2

Chapter 5 from *Claude at Work* by Elton Stoneman

Planning and building in stages with BDD 20

Chapter 6 from *Claude at Work* by Elton Stoneman

AI-assisted testing for developers 37

Chapter 4 from *Software Testing with Generative AI* by Mark Winteringham

From autocomplete to agent fleets: the road to Continuous AI 72

Chapter 5 from *Vibe Engineering* by Tomasz Lelek and Artur Skowroński

introduction

Working code is important, but that's only one part of the project. You also need to understand the requirements, make sound design decisions, and verify that what you've built behaves as intended. AI can help with all of that. But first, you need to know how to direct its work and evaluate the results.

This collection of chapters shows you how to incorporate AI into everyday software development without leaving good engineering practices behind. Drawing on three Manning books, it takes you from your first session with Claude Code to a repeatable workflow for planning, implementing, testing, and delivering software.

Start with a project and a task worth handing over. In "Getting started with Claude Code," from *Claude at Work* by **Elton Stoneman**, you'll learn to give Claude the context it needs, set appropriate permissions, and review the changes it makes. These first practical steps establish habits you'll rely on as you delegate more substantial work: knowing what your assistant can do, what it needs from you, and what you should check before accepting its code.

Of course, even a capable coding assistant can produce the wrong code. The trouble often starts with requirements that seem clear until someone tries to implement them. That's where "Planning and building in stages with BDD," also from *Claude at Work*, comes in. Describing expected behavior through Gherkin scenarios lets you resolve ambiguities before they become code. Breaking implementation into smaller stages enables you to inspect, correct, and approve the work as it progresses.

Testing deserves the same care. A collection of generated tests may look reassuring, but what do they actually tell you? "AI-assisted testing for developers," *from Software Testing with Generative AI* by **Mark Winteringham**, explores that question through hands-on testing examples. See how AI can help you examine code and develop unit tests, then bring it into the red/green/refactor cycle. You'll learn more ways to investigate risks and refine your design—and a better basis for judging whether your tests are useful.

The final selection widens the view from your editor to the team's delivery process. Shared instructions, CI pipelines, and handoffs between agents all shape whether AI-assisted work can be repeated reliably. *Vibe Engineering's* "Continuous AI development with AI-Native SDLC" by **Tomasz Lelek** and **Artur Skowroński** examines how these pieces fit together, helping you decide where automation belongs and where human review is essential. It's the next step toward making the practices you've tried on one task useful across many.

The aim of this curated ebook is a development process you can trust and repeat. Whether you're experimenting with a coding assistant or introducing AI into your team's workflow, these chapters are a great place to begin. If you're interested in delving further into these skills and concepts, we highly recommend the complete books featured here for more examples, deeper explanations, and guidance for your own projects.

Thank you,
The Manning Team

Put Claude Code to work in your editor, with a project it can understand and permissions you're comfortable granting. As Claude builds, you'll practice reviewing its changes and deciding what to keep.

Chapter 5 from *Claude at Work* by Elton Stoneman

Getting started with Claude Code

This chapter covers

- Agentic coding and your role as the reviewer
- Installing and running Claude Code in your editor
- Initializing the repository so Claude can navigate around
- Setting permissions so Claude can work autonomously
- Building and testing the first complete service

You're used to driving the conversation with Claude. You take turns with the model in a chat session, and if you have nothing to add when Claude replies, the conversation ends. You've also seen that in Cowork, Claude's turns can get much longer — your prompt might trigger multiple tool calls and generate output files before you see the response. Claude Code is where you'll see just how much work Claude can do on its own. This is Anthropic's agentic coding tool — it's Claude running inside your repository, with access to all your code and all the tools on your machine. One

prompt from you might start Claude on a turn that lasts hours and produces thousands of lines of code and tests.

Is that really what you want? It means putting a lot of trust in the model to write good code and proper tests, and it leaves you with a huge review task. It's a valid approach for work which is clearly specified and easy to acceptance-test, but it doesn't suit every task. Working effectively with Claude Code is partly about phrasing your prompts correctly, using the techniques we're already building up. It's also learning when to direct Claude at a lower level, working closely and sharing the decisions, and when to give Claude free rein and just let it build.

A big production-grade project which will never actually run in production is a great testing ground. It lets you experiment with Claude Code, see what it can do and how it does it, without worrying about customer requirements, support processes, or scale. We've designed and specified all the components for our application, and now we'll pick a core service and see how we can direct Claude to build it with minimal prompting.

5.1 What is agentic coding?

Writing code is a perfect job for AI. Programming languages follow clear sets of rules, algorithmic patterns are well documented, and there are billions of lines of open-source code which models are trained on. So the generative part is straightforward, but where agentic coding really excels is that it uses the same tools you do. It doesn't just write code — it also runs `dotnet build` to check for compilation errors, `npm start` to run the application, and `pytest` to run tests.

> **The lightbulb moment**
>
> This is often the lightbulb moment. When you see Claude running the same tools you would use yourself, you realize the model can do much more than generate static lines of code. I had this experience early on when I was trying to find a problem in a Kubernetes deployment. I was looking at the YAML file in the editor and asked Claude a question about it, and in the response Claude said it wanted to run a `kubectl` command. First I thought — wow, I didn't even know Claude could do that. Then I thought — wow, it's clever that it wants to look at the live deployment to compare it to the YAML. And then I thought — wow, that's the exact command I was planning to run next.

Agentic coding just means using AI to write your code, but it can be an almost entirely hands-off approach, where you describe the problem and the model operates autonomously to build the solution.

Claude Code is a terminal application which you run in a shell session, or in the terminal window of your editor. It has the same choice of models as the chat and Cowork

interfaces, but you run it inside your repository folder so it has access to all of your source files, tests, build scripts, and documentation. Claude can navigate through the whole codebase, so if your prompt includes an error message it can search to see where the message gets written, and reason about the call stack and the data that led to the error.

The fundamental workflow is the same: your prompt gets sent to the model running in Anthropic's cloud, and it generates a response. But in Claude Code the response can span multiple tool calls — search for this text, edit these files, run the build script, execute the tests — those are all folded into a single turn, and all the tools run on your machine, sending the response back to the model. The big difference between Claude Code and the other interfaces is that there is no sandbox physically restricting what tools or files the model can access. Figure 5.1 shows the setup — Claude Code can access anything you give it permission to use.

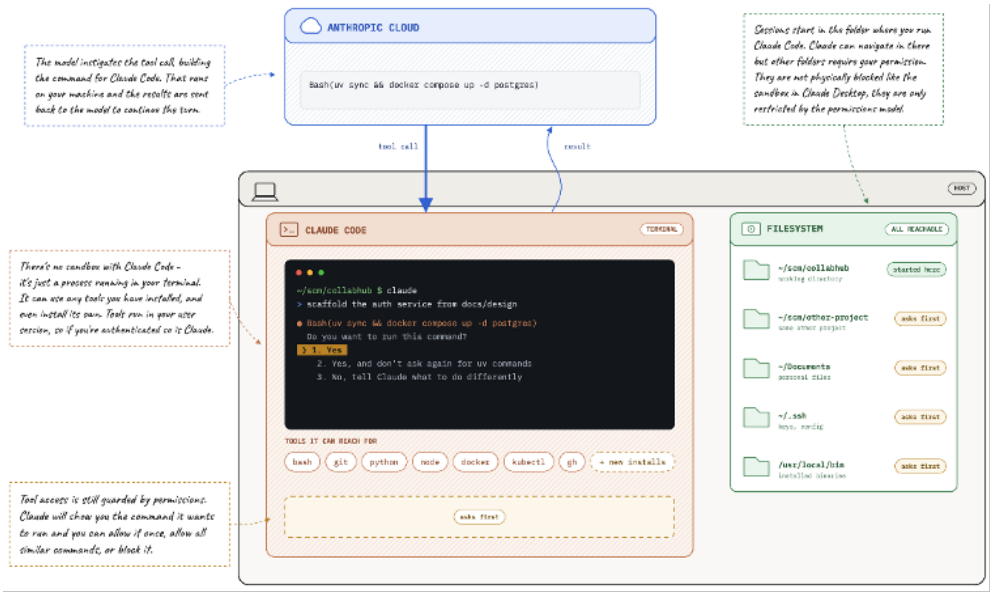


Figure 5.1 Claude Code runs inside your repository with no sandbox. The permissions model asks your consent to access files and run tools, but there is no physical backstop to prevent unwanted behavior.

This makes Claude Code tremendously powerful and productive. In a session you might write a prompt in a few lines which starts Claude working on a response which takes an hour. In that time you're working on other tasks in other sessions. It's time to install Claude Code and see how it works.

5.2 Installing and running Claude Code

There are native installers for Claude Code to run on macOS, Windows, and Linux. Browse to https://code.claude.com/ to download and run the installer, or install from the terminal:

```
# macOS or Linux:
curl -fsSL https://claude.ai/install.sh | bash

# OR - PowerShell on Windows:
irm https://claude.ai/install.ps1 | iex
```

Now you have a `claude` command. Before you can do anything useful, you need to log in:

```
claude auth login
```

This will open a browser for you to sign in with your Anthropic account. Follow the login flow and then back in the terminal you will see a `Login successful` message. Claude Code uses the same token consumption model as the rest of the platform, and your account has a single set of quotas which apply across all the tools. So if you max out your usage in a Claude Code session, you can't do anything in the mobile or Desktop apps until your quota resets (and vice versa).

When Claude starts writing code, a big part of your role will be reviewing the output. It's much easier to do that if you run Claude Code inside your usual editor, because it will list all the files Claude has created or edited, and show you the differences in files which Claude has changed. Any editor will do, preferably with Git integration and the option to run a terminal window alongside the file editor. VS Code is a good option if you don't have a preference.

> **TIP** There are Claude Code plugins for VS Code, Cursor, and the JetBrains IDEs. They integrate with the editor to show inline file differences and conversation history, but they all use the same Claude Code engine. I occasionally use the VS Code plugin to check out the new features, but I usually just run `claude` from inside a terminal window in VS Code. I prefer to work with the native app, so I get the same experience when I'm working on a machine without the plugin, or without any GUI at all.

5.3 Try it now — run Claude Code in your editor

Launch your editor and open the folder where you have the Git repository for your application. You should see the `docs` folder with all the design documents Claude built for you. Open a terminal window (in VS Code you can use the [Ctrl-`] shortcut) and run Claude Code in manual permission mode, so Claude asks your approval before editing files or running tools:

```
claude --permission-mode manual
```

Figure 5.2 shows my editor with Claude Code running in manual mode.

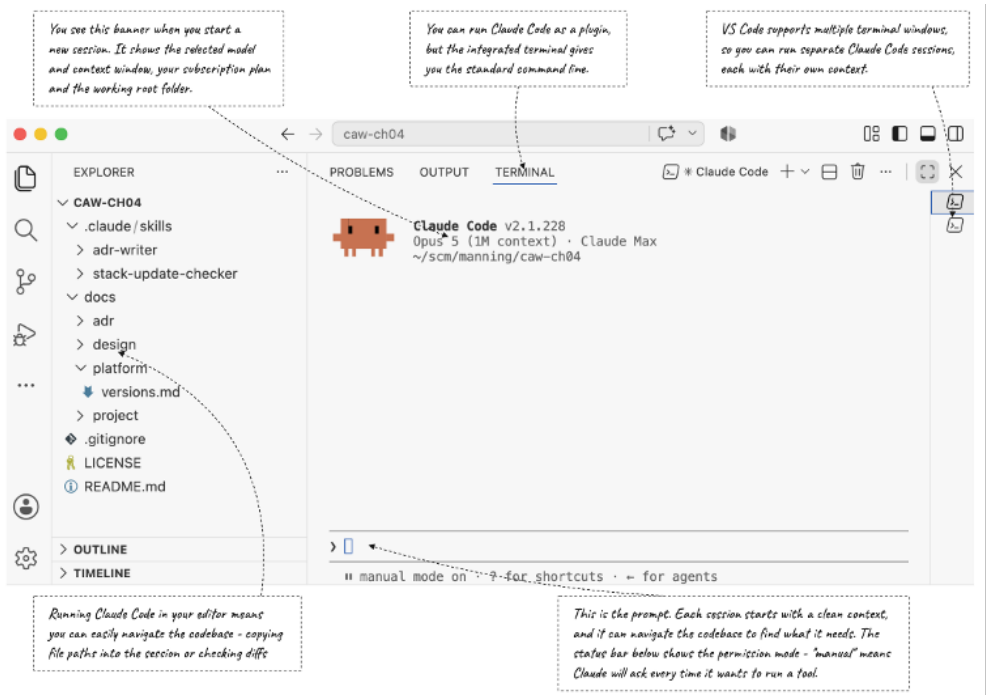


Figure 5.2 Claude Code running in a VS Code terminal window. You and Claude are looking at the same repository, so it's easy to work collaboratively.

This is the session we'll use to start building the application.

5.4 Your first Claude Code session

Claude Code sessions start with an empty prompt and no context: just like conversations in chat and Cowork, you start fresh each time. It's not much fun if Claude has to scour your codebase in every new session to work out which language and build tools it needs to use, so you can capture a brief description of the repo in a markdown file called `CLAUDE.md`. When you start a Claude Code session, if that file exists in the repo then it is automatically added to the context and gives Claude a head start with all the key information it needs.

You could write the `CLAUDE.md` file yourself, but Claude can do it for you with the `/init` command. If you run this in an established codebase then Claude will write the equivalent of a project README file with all the technology and tool details. It has the same purpose as the README — getting new joiners up to speed on the repo — but it's specifically focused on new joiners being future instances of Claude.

We don't have any code yet, but there is a lot of design documentation which Claude can use to bootstrap the instructions.

5.5 Try it now — use Claude to build the agent documentation

A single `/init` prompt would probably be enough, but I've added some more detail so Claude can document how developers will run the app locally, and how we will deploy to other environments.

ES /init - this repo will build out the whole app specced in the docs/design folder. we will use docker to run the stack locally, and package with helm for deployment to kubernetes

In my session Claude asked if I wanted a project-level `CLAUDE.md` file to live inside the repo, or a personal one to live in my documents folder. We want anyone who works on the project to benefit from the instructions file, and to be able to extend it as the application evolves, so I chose project level.

Claude also found my skills folder and asked if I wanted to bootstrap any other skills (I said no), then moved on to specific details about the Python toolset I wanted to use. Then it reviewed all the documentation and came back with some discrepancies it found between the original design documents and the component version list — it knew the design was written earlier and asked for confirmation that the version file was authoritative.

You'll get other questions as you work through the initialization, and this is where you need to fix on any decisions you left out, or had only vaguely answered. Telling Claude we're about to start building will trigger all those questions, because the `CLAUDE.md` file needs to be good enough for the next agent to start work. Let the session guide you — Claude will flag any gaps and provide suggestions on how to fill them.

My `CLAUDE.md` is at https://github.com/sixeyed/claude-at-work-project/blob/main/CLAUDE.md — you can check the file history to see how it has evolved over time. Listing 5.1 shows a snippet from the initial version which Claude generated.

Listing 5.1 The CLAUDE.md file — a guide for AI agents joining the project

```
# CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working
with code in this repository.

## What this is

CollabHub — team chat + collaborative canvas. Five Python backend
services and a React SPA, specified in `docs/design/` and being built
out from that spec. Run locally with Docker Compose; deploy to
Kubernetes with Helm.

**Read `docs/design/00-platform-conventions.md` before writing or
changing any service code.** It is the authoritative cross-service
contract (errors, pagination, auth, IDs, jobs, config, observability).
Where a per-service doc disagrees with it, it wins. Per-service specs
are `docs/design/01`-`06`.
```

This is human-readable markdown, just like a `SKILL.md` file. It uses conventional markdown syntax — broken into sections with headings, and highlighting in **bold** to tell agents when instructions are important and shouldn't be skipped. My document included sections on the repo layout, open decisions, and "conventions that are easy to get wrong". That last section comes from some introspection on Claude's part and shows why the `CLAUDE.md` file is best left to Claude to generate and maintain. It lists areas where it suspects agents might take a wrong path and steers them in advance, with guidance that wouldn't have occurred to me.

It took a few more turns before Claude was happy we had all the major decisions nailed down. During the process it saw my Architecture Decision Record skill in the project folder and offered to use that to write up the key points we'd agreed. This was a no-code session, just prep work for future agents so they could quickly get oriented.

When you go through this process yourself, you will probably be surprised at how thorough Claude is — and frustrated by how often it stops to ask your permission to do something. If you want to leave Claude running unattended to work on a long task, you need to change the permission mode.

> **FOLLOWING MY PROGRESS** I'm building my application in a public GitHub repo, using tags to capture the progress. This is separate from the sixeyed/claude-at-work repo — that contains all the transcripts and reference material for the chapters. The project repo is a clean build at sixeyed/claude-at-work-project. If you want to see how the work develops through the chapters you can clone it and switch between tags: `git clone` https://github.com/sixeyed/claude-at-work-project.git and then `git checkout ch04-setup` to get to this position.

5.6 Changing the permission mode to let Claude run free

We explicitly set the permission mode to `manual` when we ran Claude Code — which means pretty much everything gets checked. You send in your prompt and Claude's response is constantly interrupted because it needs to ask your permission to run any tools — `cat` to read file contents, `for` loops to iterate over folders, `git` to check history. You'll see approval requests like this pop up all the time in manual mode:

```
This command requires approval

 Do you want to proceed?
 > 1. Yes
   2. Yes, and allow access to caw-project/ and git ls-tree commands
   3. No
```

Option 2 approves the tool use for the rest of the session so it looks like it will save you a lot of time, but in practice every tool call is slightly different so you'll still get plenty of permission checks. Claude Code supports alternative permission modes to reduce the approval burden. These are the main options - the default mode changes over time so it's good to experiment and be explicit:

- `manual` — file reads and read-only commands are pre-approved, everything else gets permission checked
- `acceptEdits` — file reads, edits and commands judged to be safe within the working directory are pre-approved
- `plan` — file reads and navigation allowed, but no edits — Claude builds a plan you need to approve before it starts work
- `auto` — uses the same mechanism as `manual` but replaces you as the human approver with an AI classifier that judges the permissions
- `bypassPermissions` — skips the permissions check for everything, only blocks commands which wipe your hard drive

`bypassPermissions` is just as bad an idea as it sounds. It means Claude can do whatever it wants, without ever asking for confirmation. You're relying on Claude behaving correctly and nothing malicious injecting itself between your machine and the model running in the cloud. If you cloned a repo with a malicious `CLAUDE.md` file that tricked the model into issuing dangerous commands, Claude Code would run them.

I'm just going to delete all this stuff

Having said it's a bad idea, I do use `bypassPermissions` myself, in strictly risk-accepted circumstances. When I'm working on a prototype project which is all my own code, deploying to a dedicated environment which doesn't impact anything else, then I will switch to `bypassPermissions` if I want a session to do a lot without stopping. I've used it to run soak tests over the weekend on my own dev cluster, with Claude monitoring the run, fixing issues, redeploying, running the soak test again, and getting everything working for Monday morning.

But I've also come close to being burned. I was building out a project which deployed to Azure using Terraform. The setup assumed only one instance of the app would be deployed, so it didn't use unique object names. I had Claude deploy a second instance, which failed because of name collisions. Instead of fixing the config to use unique names, Claude just deleted the whole Azure Resource Group — which took out my database, Kubernetes cluster, and Storage Accounts. Luckily it was just a throwaway dev environment, but it was an important lesson: the risk you think you accepted is probably a subset of the real risk.

A better option is `auto` mode. It makes separate model calls to evaluate tool use suggested by the main model, using a lower-spec family (it would use Sonnet if you're using Opus for the main work). It replaces you as the security guard, keeping Claude running unless something dangerous crops up. There's an additional token cost for the classifier, but it's much smaller than the main conversation because the assessment only sees the tool call, not the whole context of the session.

5.7 Try it now — build out the application scaffolding in auto mode

You can leave your current session cleanly with the `/exit` slash command. Then run Claude Code again but in `auto` mode:

```
/exit

claude --permission-mode auto
```

This new session will automatically read the `CLAUDE.md` from the last session, so it's ready to start work. What work? This is a greenfield project running across distributed components, so my preference is to start by getting the basics in place for every component. The goal is to get a deployable app which doesn't do anything, but where all the components start up and are correctly wired together.

My sense of the first task is to generate the boilerplate code for all the components.

ES we're ready to make a start on the code. scaffold everything.

I'll start by getting oriented in the design docs before scaffolding anything. …

In my session Claude asked how deep the scaffolding should be — a *walking skeleton* with just the boilerplate, a skeleton with a full vertical slice or "phase 0 done". The last one confused me for a minute — was *Phase 0* an idea that Claude came up with? I left Claude waiting while I checked through the repo and there I found my Excel workbook with the project phases mapped out. That was just meant to be an example to show to a stakeholder, but Claude discovered and read it, and offered to build in line with the plan. I said to skip that and go for the walking skeleton.

Then Claude went on to do its stuff. I exported a transcript at the end of this session (https://github.com/sixeyed/claude-at-work/blob/main/chapters/ch05/conversations/01-project-scaffold.md) — if you browse through it you'll see 7,000 lines of Claude reasoning and working by itself, including all the tool calls it made. There are only five occurrences of `AskUserQuestion` when Claude stopped for clarification (one of those was about producing the transcript); the rest of the time Claude was the engineering team.

When you let Claude evaluate permission requests you're relying on it to keep your environment safe, but you're also deferring workflow decisions to the model. In my session Claude created a feature branch for the work, committed changes under its own user ID and created a PR in GitHub for me to review. Fine if that's your workflow, but it puts your agency right at the end of the process. I'd rather review changes before the commit, and it's easy to change Claude's opinion — I asked it to update the `CLAUDE.md` file with a rule saying it should never commit changes (and also ignore the project Excel workbook). Claude added the section in listing 5.2.

Listing 5.2 My preferred working practices, added to CLAUDE.md by Claude

```
## Working in this repo

- **Never commit.** Stage nothing, run no `git commit`, open no PR.
  Leave the working tree dirty and tell the user what changed —
  committing is theirs to do, always. If work belongs on a feature
  branch, create the branch and say so, but stop short of the commit.
- **Ignore `docs/project/`.** Those files are book-production material,
  not project input. Do not read them, cite them, or act on anything in
  them.
```

At the end of the scaffolding phase, Claude had generated the boilerplate code for all the backend services and the front-end web UI. It wrote Dockerfiles to build each component, a Docker Compose file to run the whole stack, and an initial Helm chart to run the application in Kubernetes. About 1,000 lines of application code and 1,800 lines of infrastructure code.

Your project will be different, especially if you focused on a different infrastructure stack, but if Claude did well you'll have a similarly large chunk of work to review. How do you do that efficiently?

5.8 Your role as reviewer

Don't read every line of code Claude writes. Use your experience and knowledge to take shortcuts and focus on the important things. In my case the services are all Python components, mostly FastAPI HTTP servers. I scanned through the setup code and looked at the routes — Claude had written a health check endpoint in every service, which verified that the component could reach its dependencies (the Postgres database or the Redis cache). The front-end HTML and React code was minimal so just needed a quick glance. The Dockerfiles all followed best practices. The Compose file included all the custom components and the dependencies — pinned to specific release versions. The Helm chart had templates for each component and the networking layer, with all configurable options split out to a values file.

You don't get a high level of confidence from a quick source code scan, so you need to enhance it with external proofs. Tests are the obvious one — and Claude should have generated unit tests for you as part of the scaffolding work. I have a suite of `pytest` fixtures and I can run `pytest` from the project root as the next confidence step. Then there's running the app for real. In the project initialization I specified we should use Docker for every component, and in the scaffolding session Claude wrote a Docker Compose file to build, package, and run the app locally. So I can run `docker compose up --build` to test the whole thing, dependencies and all.

The front end Claude built is a working web app — with just the static text you can see in figure 5.3.

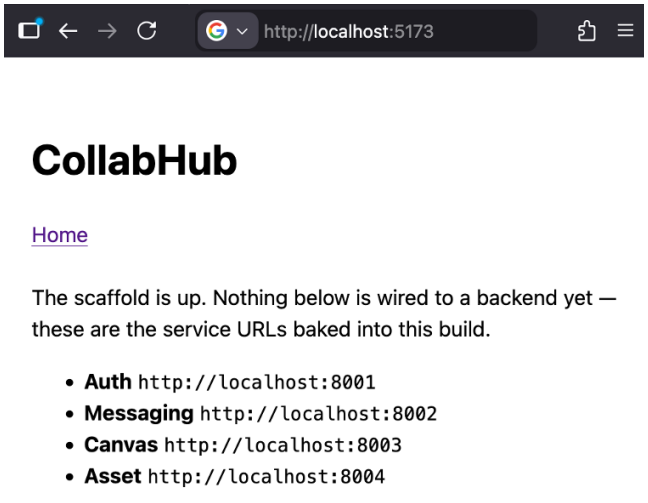


Figure 5.3 My web front end, with the code, build and deployment resources written by Claude.

Your own stack may be different but the key thing is: you know what to do to increase your own confidence. Open up a new terminal session outside of Claude and run the commands you would expect to use to build, test, and run your app. If they don't work as expected, check the README — Claude should have written one with all the new starter details. You don't have to micro-review every commit if you can take a step back and define what "good" looks like for this particular piece of work.

> **TIP** This is the new definition of done, but it doesn't need a team meeting to agree it. You need to think about the quality gates you would use at the end of development — what final checks would you run before you create the PR? Apply those to Claude's work.

In my case Claude had added `healthcheck` sections in the Docker Compose file, so the application containers get tested while they run. As long as the container images build and the containers keep running, then the healthchecks are passing and the skeleton app is working.

Trust like Uncle Bob

Robert Martin is known to the coding world as Uncle Bob. He's the author of *Clean Code* and *Clean Architecture*, and he coined the term SOLID. Not everyone is a fan of his approach, but he has pretty serious credentials and he's a big advocate of stepping away from the code and building trust from outside. He posted: "My current strategy is to not read any of the code written by my agents. That's the only way I can take advantage of their productivity. What I do instead is to surround the agents with extreme constraints. Unit tests, gherkin tests, QA procedures, quality metrics..." [do we need a ref for this? https://x.com/unclebobmartin/status/2080257779395154409]. As you would expect, that thread is full of challenges. Someone asked — if AI agents do all the work, are you really the engineer? Bob's response: "I am the engineer because I am accountable."

The further you step away from the code, the more you have to think about how to get to a level of confidence you're happy with. I have had Claude build and test large features entirely autonomously, including deployment to the dev environment, testing through a browser harness, and validating functionality by querying the database and checking logs. I positioned myself in the role of user-acceptance tester, because it was a clearly defined set of functionality and it would be very obvious from the UI if it worked correctly. Not every piece of work fits into that approach, but where it does you really see the productivity leap.

5.9 Writing code with the Superpowers skills

How much time has Claude saved on the build phase so far? If I had written this all from scratch then I think it's probably about one day's effort. There's not much code, but the time goes in things like finding the latest stable dependency versions, repeatedly running tests and building Docker images, and testing the deployment artifacts. Claude can do all of that so much more quickly (and without having to stop to fix typos).

But we need to build proper code to see what Claude can really do. We'll continue in `auto` mode and let Claude build the first component. In my design that's the auth service which handles authentication and authorization, and is a core dependency for the rest of the stack. There's quite a bit of complexity there as I want to support multiple identity providers (IdPs), so although I want Claude to do the work for me I don't want a huge final draft to review.

A good way to approach a big greenfield build - where you begin with an empty repo - is to start with a minimum viable product (MVP), so Claude can build out the key workflows and you can test an early version of the component. That gets you a genuinely useful step forward in the implementation, and it keeps the review work manageable.

5.10 Try it now — build the MVP for your first service

Your existing Claude Code session has a whole pile of context built up from the scaffolding work. That doesn't really help with the next stage, but you can stay in the current session and just reset the context with the `/clear` command.

```
/clear
```

We're going to start some real work now, and Anthropic has adopted the community Superpowers plugin (https://github.com/obra/superpowers.git) which is pretty much essential for coding. You can install plugins from a marketplace in Claude Code — the approach is the same as in Cowork, but you do it through the terminal, managing plugins with the `/plugins` command.

```
/plugin install superpowers@claude-plugins-official
```

There are a set of skills in the Superpowers plugin which Claude Code will use all the time:

- *brainstorming* explores an idea and possible implementation approaches, then researches and presents you with a design for implementation
- *systematic debugging* investigation into issues, split into phases aimed at confirming the root cause before suggesting or implementing a fix
- *test-driven development* insists Claude should write a test and see it fail before implementing new code to make the test pass
- *code review* how to start a code review for new work, and how to respond to another agent's review.

You don't need to specify any of these, Claude Code will just pick them up as it needs them. When you ask it to build out the MVP for your service, it sees that as a design task and will (probably) invoke the *brainstorming* skill.

ES ready to implement src/services/auth - start with an MVP to get it running

I'll start with the brainstorming skill, since this is creative work — designing what the auth MVP actually covers. …

In my session (https://github.com/sixeyed/claude-at-work/blob/main/chapters/ch05/conversations/02-auth-mvp.md), Claude went off and read the design docs and examined the current state of the codebase. It came back with specific questions about what functionality we should build for the MVP, whether we should write a stub identity provider, and clarification on some outstanding design decisions. Then Claude showed me what it planned to do so I could approve or tweak it before we got started.

ES go for it /goal get the mvp working and tested with unit and integration tests

The `/goal` slash command is the Nike of agentic coding with Claude ("just do it"). It tells Claude to keep going until it reaches the condition you specify. In my case it's a fairly vague "working" condition, with some testing gates set. But this is part way into a session and Claude has all the context to understand what the goal is. This is when you can set Claude on a long piece of work and move onto something else — if there are questions then Claude will make a best-guess decision, and when it completes it will tell you if there's anything you need to check.

> **WARNING** Setting a goal means Claude Code will take turns for you. The model works on the initial prompt and sends a response when it thinks it's done. Then an evaluator in Claude Code checks what's been done against the stated goal — and if it doesn't agree that the work is done, it sends another turn into the model saying what's needed. Context and token usage can build up, especially with an ambiguous task, so you can add a guard to your prompt like "don't go beyond 10 turns". That becomes a second exit point for the conversation.

When Claude was happy it had met the goal, I had over 4,500 lines of Python code and 109 tests, including unit tests and integration tests that operated on the stack running

in containers. What's the best way to get confidence for this? I used Docker Compose to build and run the app, then ran the test suite. The containers stayed healthy, the tests all passed, and there were no errors in the logs. But to get a good feel of what's there, I really wanted to do some exploratory testing on the API itself.

There's a VS Code plugin called *REST Client* which lets you document HTTP calls in a text file, which turns into a clickable resource to execute services and view the response. Claude knows the spec to use for the plugin and it knows what endpoints the new API has, so I asked it to build the file.

ES update the readme and write src/services/auth/api.http as a REST Client extension doc, for manual testing of the API - assume running in docker

I got a `.http` text file which covered the full login workflow with token refresh, together with error paths to test missing data or incorrect request formatting. The full file is just under 300 lines and you can read it as documentation for the API. Listing 5.3 shows a snippet of the login flow.

Listing 5.3 A REST Client file for exploring the API, generated by Claude

```
# Signing in

### Local-only sign-in: any email, no password, no IdP.
# @name login
POST {{host}}/api/v1/auth/dev-login
Content-Type: {{json}}

{
  "email": "ada@example.com",
  "displayName": "Ada Lovelace"
}

# Using the access token

### The signed-in user's profile.
GET {{host}}/api/v1/users/me
Authorization: Bearer {{login.response.body.$.accessToken}}

### The claims on the current token — no database read.
GET {{host}}/api/v1/auth/userinfo
Authorization: Bearer {{login.response.body.$.accessToken}}
```

This is a nice, clear way to understand how the API works from the client's perspective. I can see the routes Claude built and how they chain together. The REST Client format is a simple representation of each HTTP call — the URL, whether it's a GET or a POST, then the request headers, and the request body. The output of one request can be used later — if you run the login call you'll see the response in the output window (see figure 5.4), and the access token is stored for other calls to use.

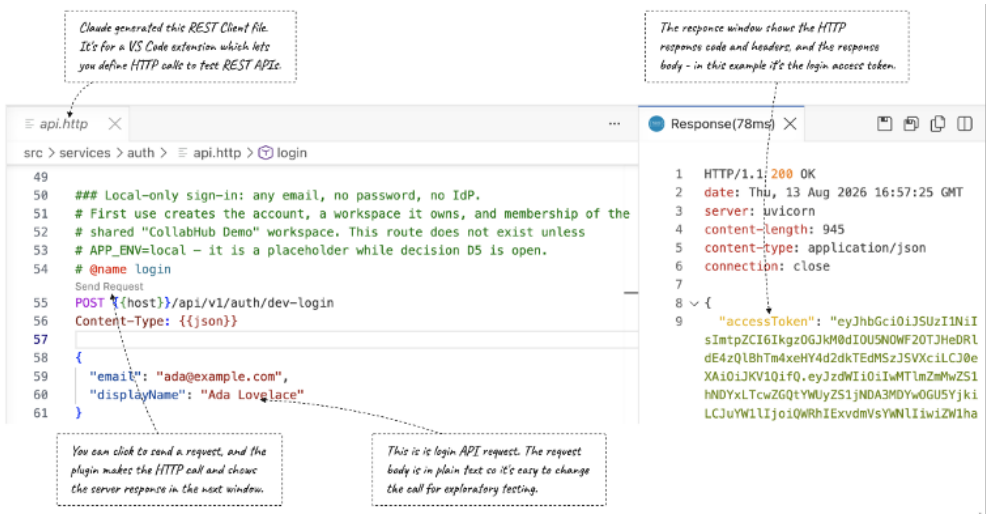


Figure 5.4 Running the login call from the REST Client file — the response shows in the output window and the access token is stored for later calls.

REST Client is how I like to work with APIs, so it's the obvious choice for me to explore this service and gain confidence that Claude has built the right thing, as well as building it correctly. If you prefer curl or Postman you can ask Claude to generate client calls using those tools. You may not even need to do that — my Python stack publishes an OpenAPI docs page by default, and Claude had documented the request and response schemas so that becomes a usable test client.

Effective agentic coding is finding a balance between the productivity of the agent, and the amount of reviewing you need to do to trust the agent's work. You need to consider both parts when you're planning the task: is this one session with a good prompt, where I can be happy with the result if I scan the diffs and run the tests? Or is this a larger piece which I need to break into separate sessions, setting the goal for each so it's clear to Claude what it needs to do, and it's clear to me how I will accept it.

5.11 Your turn: Complete the first service

If you've been following along with me you'll have the MVP for one of your services built, tested, and running on your laptop. You might have taken some time to explore the API with your favorite tool, or read through the integration tests to gain confidence that Claude has got all the workflows covered. Hopefully you'll be pleasantly surprised at *just how good Claude is*. The code follows best practices, Claude writes and runs tests as part of the build, and doesn't return to you until it's happy with what it's delivered.

But an MVP is not the full service. It's time for you to direct Claude to finish the first part of the build, and get it done to your satisfaction.

Objective

Build the complete implementation of one of your application services. Claude's work should include everything you would expect from a senior engineer on your team — SOLID principles, automated testing, scripted deployments, and up-to-date documentation. Add any quality gates you need for your own confidence. When you and Claude have finished, you should be able to run the application on your chosen platform (locally or in the cloud) and see all the features of the service working.

In my case I already have the foundations from my MVP build. I'm planning to leave the cloud deployment until later; I'm going to run in Kubernetes, so as long as I can get everything packaged and working in a local environment, it should be simple to move to the cloud.

The key point in moving from my auth MVP is to build out the remaining services in the design spec, and to replace the stub identity provider with a real one which I can run locally. I could let Claude decide on that, but I have used Dex and OAuth Proxy for similar work before, so I wanted to stick with that. Here are the first couple of prompts I used to give you an idea how the session went:

ES we have implemented the auth service as just an mvp - now we need to build out the full functionality; use dex/oauth-proxy for the idp and add that to the docker compose setup

…

go for it - /goal complete the full auth stack so it can be run and tested with dex

You can see my session transcript here: https://github.com/sixeyed/claude-at-work/blob/main/chapters/ch05/conversations/03-full-auth-implementation.md. By the end I had a fully OpenID Connect compliant auth service with a pluggable identity provider — so I can use Dex locally and switch to Azure Entra, Amazon Cognito, or Okta. When I run the app with Docker Compose I can browse to the web front end and I get an authentication page, where I can redirect to Dex to log in (using dev credentials which are already set up).

Getting to the stage where Claude has done all the work and you're happy with it shouldn't take more than an hour or two, and you'll spend most of that time doing something else and just checking in on Claude's progress. Remember that your Claude Code session is interactive, it's not just about generating thousands of lines of code. If you want something explained in more detail, or you want a test coverage check, or even if you want ideas about how to gain confidence in the results — just ask Claude.

How much work does Claude do in those few hours? I would estimate my service is about a week's worth of development time.

Summary

- Agentic coding means Claude works inside the repository — navigating and extending the codebase.
- Claude Code will run the same tools you do in the development workflow, building and deploying the app, and running tests.
- The `CLAUDE.md` file describes the application and includes instructions for working on it; it gets loaded into context at the start of every session.
- There is no sandbox to protect your machine. Claude can access any files, run any tools, and install new ones.
- The permissions model is what protects you from rogue activity, but if you run in a strict mode it will slow Claude down.
- Your role shifts to task design and review. Let Claude do the low-level work and focus on how you gain confidence in the output.

A missing requirement is easier to address before it becomes working code. In this chapter, you'll use Gherkin scenarios to spell out how a feature should behave and check Claude's understanding before implementation begins. Then you'll build in stages, with opportunities to review and correct Claude's work as you go.

Chapter 6 from *Claude at Work* by Elton Stoneman

Planning and building in stages with BDD

This chapter covers

- Using plan mode so Claude writes plans rather than code
- Breaking down a large feature into multiple plans
- Building with behavior-driven development
- Focusing your acceptance reviews on test scenarios
- Working with subagents to run multiple tasks in parallel
- Asking Claude to write the prompt for a complex task

How do you feel about letting Claude Code run free and build an entire component with very little oversight? Your confidence comes from running tests, using the app, and layering on more quality gates until you're satisfied. But there's too much work to review it fully. Are you happy to be accountable for thousands of lines of code you haven't read? For certain types of work, you should be. I let Claude build the auth service which was closely specced and aligned to published standards. I saved at least a week of project time and any issues would have been obvious.

Other types of work need a different approach, because you want to be closer to the decisions and have more frequent feedback opportunities. You don't gain productivity having Claude write a week's worth of code, if you throw it away because it solves the wrong problem. Claude Code supports a finer-grained approach with *plan mode*. It's a normal session and you take turns in the conversation as usual, but instead of doing the work Claude plans it thoroughly and you iterate on the plan together before you tell Claude to go ahead and execute.

You can plan with Claude at different levels. If you're looking at a huge piece of work, you might plan the overall strategy with Claude before you start doing anything. Then you can iterate on a delivery plan, to break the work up into manageable — and easily reviewable — stages. And at the lowest level, you want to understand how each feature gets built, so you work with Claude on a set of implementation plans. We'll use a structured planning approach to build out the next phase of the application.

6.1 Working with a hierarchy of plans

Long before Claude, software projects were delivered with the *waterfall* approach, where everything was planned in advance before anyone wrote any code. Planning took months until everyone said they were confident in the design, then the build started. A year later the app was delivered to the client, but it had taken so long their requirements had completely changed. So we went agile and started delivering in much smaller timescales, committing to loosely-defined features within a short sprint. We cut the feedback cycle from years to weeks and kept the client close in the delivery phase. They gained confidence from seeing the actual software, rather than the plan.

Getting Claude to write a hierarchy of plans from strategy to delivery to implementation seems like a step back into the waterfall, but it's not. These plans are cheap to generate, fast to iterate on, and they have a different purpose: the implementation plan describes how we want an agent to approach one task from the delivery plan, in line with the overall strategy. The goal is still confidence: giving us (the tech lead) early feedback for the proposed delivery from our team (the Claudes), and we get it by reviewing the plans. Figure 6.1 shows how we can layer strategy, delivery, and implementation plans.

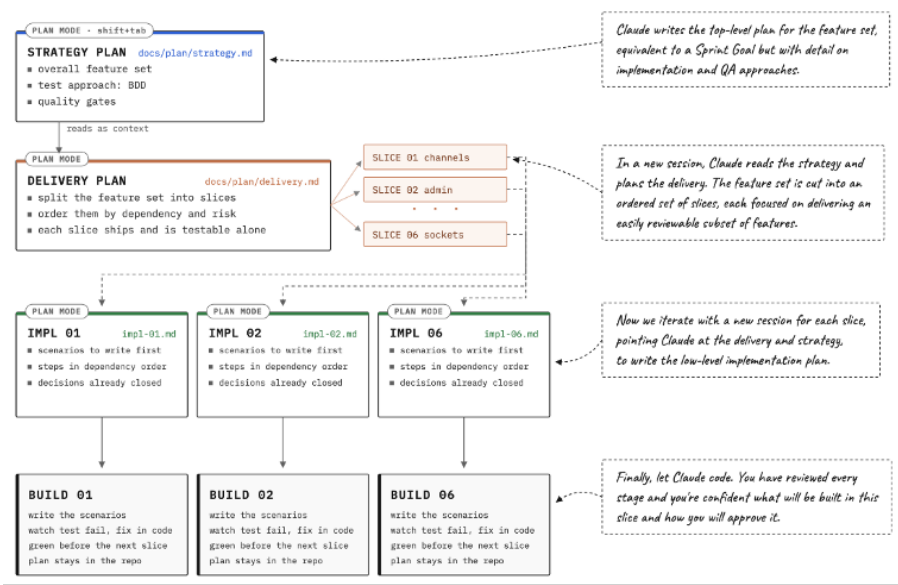


Figure 6.1 A hierarchy of plans, from the overall strategy to the delivery stages and the individual feature implementations.

We'll use this approach to build out one of the major feature sets in our application.

6.2 Writing the behavior-driven strategy plan

What to build next? A core part of my app is a messaging platform. I have the auth service in place now but that doesn't give us much to see — if I had a review meeting coming up with stakeholders, I'd want a decent set of features to show them. So I'm thinking in terms of the UI and how I could demonstrate messaging, but also how I can structure tasks for Claude so that I can review them quickly and accept them with confidence.

Behavior-driven development (BDD) is the right approach here. I can have Claude document the expected behavior in user-focused terms. Listing 6.1 shows how a message delivery feature could be described in *Gherkin* syntax, with one simple scenario.

Listing 6.1 The BDD approach — sending a message described as a Gherkin scenario

```
Feature: Real-time message delivery

  Scenario: A message reaches another member live
    Given Ada and Grace are both viewing the "general" channel
    When Ada sends a message "deployment is green"
    Then Grace sees "deployment is green" without reloading
```

Gherkin is a technology-agnostic way of describing behavior. The clear "given… when… then" formula describes the initial state, the action, and the expected outcome. The full syntax is much richer to make scenarios extensible, but the key thing is this is a shared vocabulary between testers and developers. Testers can write scenarios and developers turn them into executable tests (all the major languages have libraries or products with Gherkin support).

BDD is a good way to get confidence in Claude's work as an external proof. You can have Claude write the Gherkin specifications before it writes any code, and that's where you spend your time — reviewing and iterating over them. You're adopting a role which is a mix of product owner, user, and tester, being sure that Claude has the correct understanding of the feature, and thinking about edge cases and additional scenarios you'd want to see.

There's no mention of BDD in my application repo so far, so I'll work with Claude on a strategy plan to build out the messaging features using BDD.

6.3 Try it now — enter plan mode and think through the BDD strategy

Planning is actually a permission mode in Claude Code. It constrains Claude to read-only actions and navigation, and the ability to write the plan as a markdown file in a temporary folder, outside of the project repo.

Fire up your editor, open a terminal window, run Claude, and switch to plan mode:

```
claude --permission-mode auto

/plan
```

You'll see `plan mode on` in the status bar under the prompt. You can cycle between `plan`, `auto`, and `manual` modes with [Shift-Tab], and Claude can automatically exit plan mode when you approve the plan.

ES next phase - we want to build out the whole messaging part of the app, complete with UI that we can demonstrate. we will use BDD for this, driving from gherkin specs.

This is effectively asking Claude to build a vertical slice across the app, with end-to-end features which use multiple services. This flips the design work we've already done, which was horizontal by component. In my session Claude wanted to clarify a few things:

Where should the Gherkin specs sit in the test pyramid — i.e. what do the `.feature` scenarios actually drive?

- How much of messaging is in scope for this phase?
- What runs the Gherkin scenarios against the browser?
- The SPA needs a data layer to render a live channel. What do we commit to?
- Core chat includes edit and delete — what semantics do we build?
- The SPA has 27 lines of unclassed CSS and no design system. A chat UI needs one.

You can see my full transcript at https://github.com/sixeyed/claude-at-work/blob/main/chapters/ch06/conversations/01-messaging-bdd-plan.md. For each of those questions Claude included the background detail, and options for me to choose between, with its own recommendation flagged. I've included the full set here because it shows how Claude has identified cross-cutting concerns at a high level. This is a strategy plan which combines the goal, the approach, and the QA mechanism.

Claude writes the plan in markdown but it only shows you a summary — you need to ask for the file path to read it. Mine started with the *Context* section in listing 6.2, which defined the overall goal and approach.

Listing 6.2 Part of Claude's strategy plan, which sets the overall context for the approach

```
This phase builds **core chat, end to end, demonstrable in a browser**:
channels and membership, send/edit/delete, history pagination, and
real-time delivery — plus the React UI to show it working. The specs are
written first as Gherkin and drive the implementation outside-in; the
scenarios run full-stack through Playwright against the Docker Compose
stack, so "passing" and "demonstrable" are the same fact.
```

Then there are sections on reference points, to ensure new code matches the existing codebase, and phases to cover the full feature set. Claude didn't know I would be breaking the work down in a subsequent delivery plan, and these rough phases aren't much use to me. They're too broad — "Phase 4 — the chat UI" is not the level I want to be reviewing at. And they're also too detailed — "`App.tsx` gains a `/c/:channelId` route" is not necessarily how a later agent will plan the implementation.

I could iterate over the design here and tighten it up, but I'm not going to. *This plan is not really for me.* It's to set the overall strategy another plan will drill down on. It doesn't matter if there's too much detail, the next Claude will just pick out what it needs. To keep the plan available, I asked Claude to copy the file into my project repo (it's at https://github.com/sixeyed/claude-at-work-project/blob/main/docs/plans/ch06/01-messaging-core-strategy-plan.md), so it's there for the next session.

> **TIP** You might not want to pollute your repo with plans which are point-in-time snapshots, but it is useful to have a persistent copy somewhere. You could attach them to a GitHub issue or pin them to a chat channel, a place the rest of the team can find them (and future Claudes, provided they can download them using a command line tool).

Something interesting happened in my session which you'll also see if you watch the progress updates in Claude Code. During the initial phase Claude decided that two streams of research could run in parallel, and it launched separate agents to do that work: those are *subagents*.

6.4 How Claude delegates work to subagents

Claude will manage its own team of agents when it decides parts of a task can be broken down and run in parallel. It hands them over to subagents — separate conversations which are invoked by Claude Code in the main session, not by you. Spinning up an agent is just like a normal tool call. A turn in the main session suggests using an agent, optionally with a specific role and model to use, and Claude Code evaluates that request to see if it's permitted. If so, it drives a separate conversation with a new prompt.

Subagents are powerful because they keep up the productivity of the main session without diluting the context; subagents each run with their own context. A subagent is a separate session which starts with the standard system prompt and your `CLAUDE.md` (if Claude Code decides to include it). None of the context from the main session gets passed through, so Claude generates a prompt with enough background to make sense of the task. The subagent takes turns with Claude Code directing the conversation, and when it's finished the output is sent to the main agent as a tool response. Figure 6.2 shows the workflow.

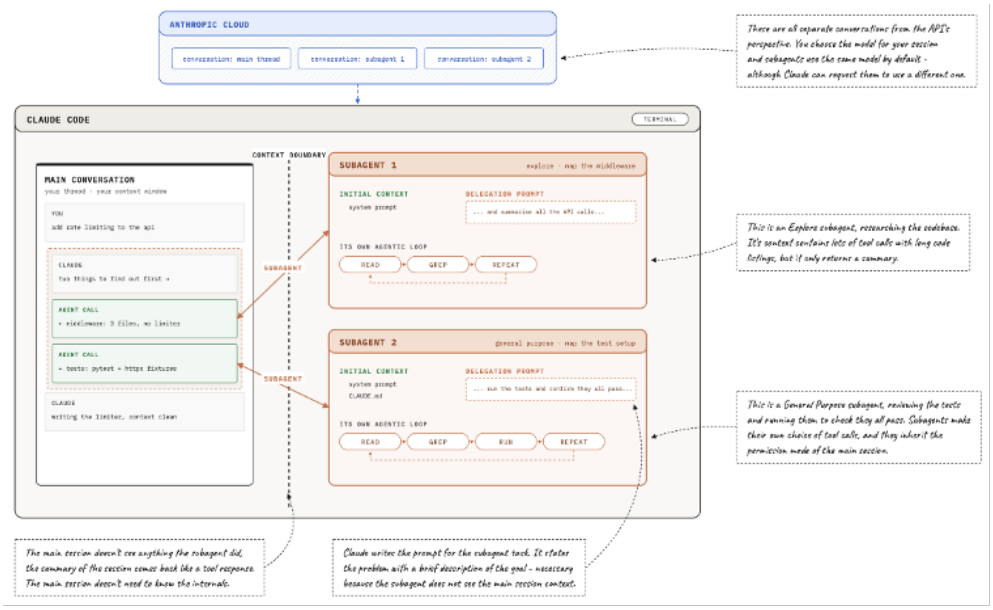


Figure 6.2 Subagents run as separate sessions with their own context. They're still subject to the permission mode and when they finish they send a summary back to the main agent.

There are different types of agent and Claude specifies which one it wants when it makes the request. *Explore* agents are read-only research assistants. *Plan* agents are tasked with building a plan for the main session to implement. *General purpose* agents can edit and run commands, limited by the same permission model in the main session.

> **TIP** You can invoke subagents explicitly in your prompt, asking something like: "run the tests using a subagent for each component and report back with the summary". If you want to keep your main session context clean you can even nest subagents so one layer can summarize the raw responses from a deeper layer for the main session: "launch a subagent and tell it to launch subagents to do this; the parent subagent should summarize the other responses back here".

The more work you can do in parallel, the more productivity you gain. Ultimately you'll want to run multiple Claude Code sessions in parallel, but before you can do that you need to plan the delivery to work out which tasks have dependencies on others.

6.5 Generating the delivery plan

Dependency planning is one of those intricate tasks which are expensive to get wrong. If you only realize task Y had to be done before task X when you are halfway through building X, that's a problem. You need to stop, do a huge context switch, move Y into the sprint, build it — and then when you come back to X you realize you misunderstood Y and you need to throw away the work you'd done and start again. Cheap planning with Claude should put an end to that.

I have my strategy plan which describes the overall approach: we're going to build out the messaging features and use BDD to do it. The next level is the delivery plan which defines how we're going to break the big piece into manageable tasks. The strategy plan is the input, and the output will feed into the next round of planning. If this seems like a lot of overhead, remember the detail of the plans is for Claude — the structure is for you. Claude writes the plans in minutes, and you can see in figure 6.3 that the process is geared to minimizing your reviewing time.

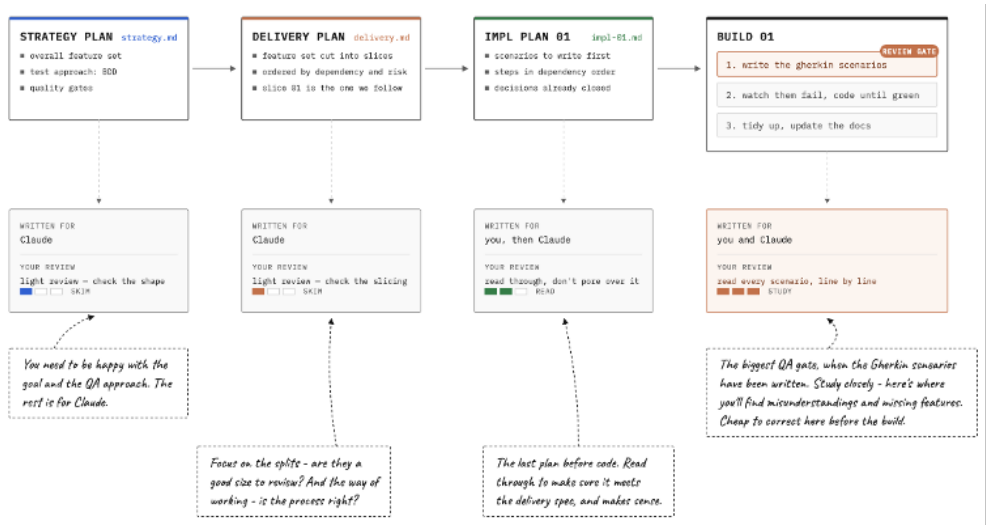


Figure 6.3 The plans are really for Claude. You can skim-review the strategy and delivery plans, spend a little time on the implementation plans, but really focus on the BDD specs.

You can scan the strategy and delivery plans, and when you get to the implementation plan that can be a quick read through. The real depth of the review comes in the first part of the build phase. You will tell Claude to stop when the BDD specs are written, and those you will scrutinize because that's how you get confidence that the feature is correctly described. Once you've iterated the Gherkin scenarios and are happy with them, it's back to Claude for the build.

6.6 Try it now — build the delivery plan from the strategy

First we need the delivery plan. If you have the same Claude Code session running, clear the context and make sure you're in plan mode.

```
/clear

/plan
```

This is effectively a new session with no context except the `CLAUDE.md` (which we have not updated, so it might be out of date). Point Claude at your strategy plan and ask for the next level, breaking the feature set into manageable pieces of work.

ES check docs/plans/ch06/01-messaging-core-strategy-plan.md - this is a plan for the approach but i want to break the delivery into chunks which are easy to review, i.e. a set of features which will start with the gherkin tests, then a build plan to implement. write the delivery plan

Claude had some good questions. The first was on how to chunk the work — it offered me options for seven thin vertical slices, four "fatter" slices, or a frontend/backend split. My goal is for pieces of work where I can gain confidence from minimal review effort, so I chose the first option. Claude also noted the strategy plan was stored in the repo and it knows that's unusual, so it asked if this plan should be saved in the same place.

Then there was a technical question about real-time publishing for messages. The recommended option was to do the HTTP work in this phase, and do the real-time socket broadcast later. When Claude offers you options they come with short descriptions, and I wasn't clear if it was suggesting to do the simple part now and extend it later with the complex part, or whether the HTTP path was a throwaway option that would be replaced. You can always choose the *chat about this* option and ask for clarification — which I did — and when you make your choice Claude carries on.

In my session there was enough information in the strategy plan and the design documents to make this conversation very short. Claude generated the plan, and table 6.1 shows the vertical slices it planned out.

Table 6.1 The delivery plan derived from the strategy — focused on small easy-to-review deliverables

#	Slice	Demo at the end
1	Channels: create and list (carries the BDD harness)	Ada creates `#general`, Grace sees it
2	Channel admin and membership	rename, archive, add/remove members, private channels
3	Messages: send and read history	send + infinite scroll; Grace sees on reload
4	Messages: edit and delete (settles D8d)	edited marker, tombstones, admin delete
5	Real-time delivery — broadcast only	two windows, live receive
6	Socket write path, optimistic send, typing	instant render, rollback, typing indicator
7	Record the decisions	ADRs, register, design docs, READMEs

The conversation transcript (https://github.com/sixeyed/claude-at-work/blob/main/chapters/ch06/conversations/02-messaging-delivery-plan.md) runs to 120 lines with no more input from me (until I asked Claude to save the session). The plan itself is just over 300 lines (https://github.com/sixeyed/claude-at-work-project/blob/main/docs/plans/ch06/02-messaging-core-delivery-plan.md). It elaborates on each of the slices, and starts with a delivery protocol which all slices should follow. Listing 6.3 shows a snip of that protocol. "Write the test, see it fail, make it pass" is the BDD approach and not committing code is my own rule from `CLAUDE.md` — I didn't have to specify either of those; Claude worked it out.

Listing 6.3 The delivery protocol every slice follows

```
**Gherkin first.** Write the `.feature` scenarios. Run them. Watch them
fail for the right reason (missing step, missing endpoint — not a broken
harness).
**Build outside-in** until they pass. Backend integration tests land with
the code they cover.
**Never edit a scenario to fit the implementation.** Fix the
implementation.
**Exit criteria** for every slice: its scenarios green
**Branch per slice** Per CLAUDE.md: create the branch, leave the tree
dirty, **never commit** — that's the user's.
```

There's one thing missing here: I want Claude to stop when it has generated the Gherkin scenarios and wait for me to review them before proceeding. I could add that in the next planning session, but this document is the driver for that so it's cleaner to get the process right here.

ES in "How every slice runs" - make it clear that Claude should **stop** after generating the gherkins so the user can review them. there may be iteration over the scenarios, and Claude should wait for approval before continuining

Then Claude added the guard clause in listing 6.4, which is a more terse tone than I would use with another team member.

Listing 6.4 The guard clause Claude added, telling future agents to stop and wait for approval

```
**Stop and wait for approval.** Present the finished `.feature` files
and **do not start building.** The scenarios are the contract for the
slice, and they are the cheapest thing in it to change — so expect
several rounds of "that's not the behaviour I want", "add the case
where…", "drop that one". Rewrite and present again. Only an explicit
go-ahead moves the slice to step 3; silence, a question, or a comment
on one scenario is not a go-ahead for the rest.
```

Your plan may have a different approach to slicing the delivery and specifying the way of work. Those are the things you need to look at carefully when you review. The lower level detail for each slice you can pretty much ignore, because those are just summaries Claude will use to write the implementation plans.

6.7 Building the first feature with BDD

Iterating through a set of plans at different levels keeps you in the loop and treats you like the tech lead. The "let Claude go" alternative is faster, but it can leave you slightly disoriented because so much gets done without your input. This way you have a clear idea of what's being built and in what order. It's slower — but it's not slow. The strategy and design are high-level plans that Claude generates in a few minutes, and they shouldn't take much more time for you to review.

We built one overall strategy plan and described how to break up the work in one delivery plan. Now we'll build an implementation plan for each slice. The implementation plans are a bigger piece of work because you want to encourage Claude to review the approach thoroughly, and flag any issues before the build stage starts. The inputs are the existing plans and the design documents, the output is the detail of how Claude will build it.

It's the decomposition into slices which keeps you more involved. You can see the subset of features Claude is going to implement and the approach it will take, and you have the opportunity to steer the direction before those thousands of lines of code start coming out.

6.8 Try it now — write the implementation plan for slice 1

If you still have your session open from the delivery plan, clear it so you start with fresh context. Otherwise, start up Claude and switch to plan mode. Then start the planning to implement the first slice.

ES docs/plans/ch06/02-messaging-core-delivery-plan.md - plan the first slice. we will implement from this so we need to confirm the delivery plan with the design and close any gaps.

My prompt is brief because we've already set the constraints in the documentation. I don't need to mention the messaging feature, insist on a BDD approach or tell Claude to stop when it has the Gherkin files — that's all in the plan. What I do want to do is make sure that the vertical defined in the delivery plan and the approach in the strategy plan covers everything in the design. Claude did find discrepancies across the documentation and some gaps that needed clarifying, and we worked through those (the transcript is at https://github.com/sixeyed/claude-at-work/blob/main/chapters/ch06/conversations/03-slice-01-build.md).

This was the longest planning session so far, about 10 minutes of work, with three subagents sent off to review documents and the existing code. The total was 15 minutes, with — Claude says — 3.9 minutes waiting for me to answer questions and 2.2 minutes for me to read the result. I did read the plan (https://github.com/sixeyed/claude-at-work-project/blob/main/docs/plans/ch06/03-slice-01-implementation-plan.md), but I was only interested in the *gaps closed* and *how the slice runs* sections; the *work* section details the low-level implementation, which is not where I need to be.

After my 2.2 minutes of reviewing, I was happy for Claude to proceed.

ES ok let's execute.

This is the same session as planning, which means all the context loaded for building the plan is still there. I could have cleared the session and had Claude load the plan fresh, but it would probably have repeated some of the research it had already done, so I chose to carry on here. I haven't said what to do next, but my plan has a hard stop specified for me to review the Gherkin feature and Claude soon got there.

> The plan's own step 1 is a hard gate: write the Gherkin scenarios and stop. Let me create the branch and check the current state first. … Stopping here — this is the plan's first gate. … Expect this to need a couple of rounds — that's what the gate is for. Nothing moves to step 3 until you give an explicit go-ahead.

Claude opened the feature file in VS Code for me, and this is where I spent some real review time. The feature file is snipped in listing 6.5 — the full file only ran to 45 lines of code, so the delivery approach gave me a nice small feature set to work with.

Listing 6.5 Part of the Gherkin file for the channels feature

```
@bdd
Feature: Channels

  Conversation in CollabHub happens in channels. A channel belongs to
  exactly one workspace and a public channel is visible to everyone in
  that workspace whether or not they have joined it. Joining is what
  will gate reading messages, from a later slice; it does not gate
  seeing that a channel exists.

  ...

  Background:
    Given Ada is signed in

  Scenario: Ada creates a public channel and lands in it
    When Ada creates a public channel named "general"
    Then Ada is looking at the "general" channel
    And "general" is in Ada's channel list
```

There were five scenarios in total and they were all correct. The feature description itself was off — it made several references to "slices" and work that was yet to be implemented. These are delivery process details, they're nothing to do with the feature so I had Claude correct that while I read the scenarios again more closely.

Just the facts, Claude

Claude has a strange habit of doing this. You see it often when you're working in an existing codebase and Claude is making changes for you. In your review you'll see multiline comments explaining how this logic used to work, or why it's being changed now. Claude doesn't appreciate that there is a world outside of the source code, where the history is all in Git and the justification is in the work management system. I usually have something like this in my `CLAUDE.md`: "write comments sparingly, only to add explanation to complex logic, never more than two lines and don't describe what has changed or why".

Although the scenarios were all good, the last one made me stop: "a channel name cannot be blank". If a user creates a channel for messages, then we surely need more rules for the name. I told Claude we needed additional validation.

ES we need more channel naming rules - minimum 3 characters, can only contain alpha-numeric and hyphens, must start with alpha

Good — that's a real rule set, and it changes the validation gap in the plan. Adding it to the scenarios.

Claude asked for a couple of clarifications (it seemed disappointed that I didn't want to support Unicode in the channel names), and added two more scenarios to test the validation rules, with a set of example names which effectively work as documentation. It also updated the plan, specifying where the rules were validated and the database schema changes. This was the bulk of my review work done.

ES this is good, let's go. /goal the whole slice is implemented, tests all pass, ready for me to verify with docker compose up and pytest

Go. Starting with the backend. Let me read the exact reference files I'll be copying idiom from.

I'll just give you a summary of the rest of the session, because there's more work we need to get onto. It took Claude about 20 minutes to build this out, which included all the code, the BDD test harness, the Gherkin test implementation, and integration tests. Figure 6.4 shows the completed UI for creating channels, with my own exploratory test data — I had Claude change the CSS to use light mode because it's a better fit for screenshots.

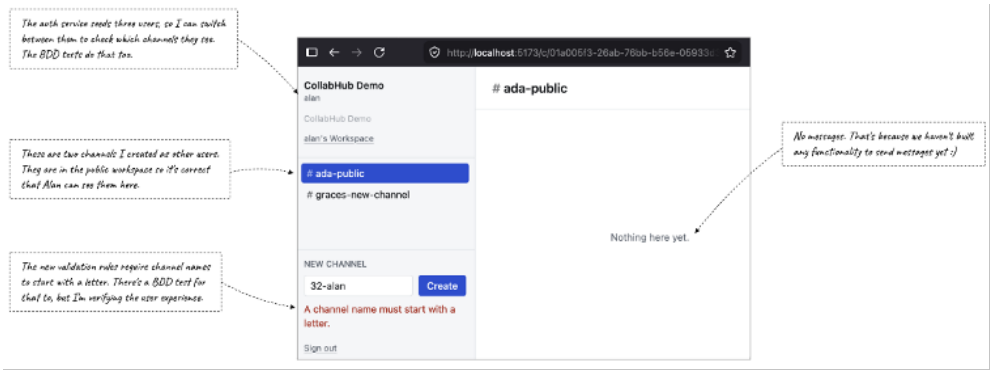


Figure 6.4 Slice 1 completed — we can create and switch channels in the UI.

After my tweak to the color theme Claude rebuilt the app and ran the tests again, and I lost my own test data. The BDD tests are full end-to-end tests which drive a browser to simulate the user and query the database to verify writes — so they need a known state and they start by wiping the database. I got Claude to split the local deployment, so I could run my own stack in Docker and the BDD tests ran on a separate set of containers with their own database, keeping the data separate.

I asked Claude for some stats at the end. It recorded 17.5 minutes of uninterrupted build time, and the full planning-plus-build session used about 40,000 input tokens, 270,000 output tokens, and 700,000 cache writes (these are effectively context reads). If I had been on a pay-per-use plan it would have cost about $60 to build this feature, but Claude estimated it would have taken a human engineer 10 days.

Let's see how we can build the next features even more quickly.

6.9 Using subagents to build plans in parallel

There are different degrees of parallelism. The simplest is when you work on a single feature in one Claude Code session, and it uses subagents when it thinks the work can be split. The most plate-spinning is running completely separate instances of Claude Code in different VS Code windows, each working on their own feature independently. That's the biggest productivity boost but in the early stages of a project you might not be able to get there — there are too many dependencies between features which haven't been written yet.

That's the position I'm in now. I got Claude to check the delivery plan to see if any slices could be built in parallel and the answer was a solid No. The work had been split into discrete sections to make it easier for me to review and accept with confidence, but it accidentally imposed a strict ordering: the slices had to be built in sequence because each one depended on features from the previous slice.

I could rewrite the plan to get a compromise between delivery speed and acceptance time, but it's still early in the project and I want to work in small increments and keep a good handle on the features as they come. But we can still improve productivity by asking Claude to generate all the plans in parallel, and then I can be reviewing the plan for slice N+1 while Claude is building slice N.

6.10 Try it now — use subagents to build all the implementation plans

It's not simple to write a prompt to make effective use of subagents. It really needs a review of all the slices and an understanding of dependencies and open decisions, before you can tell Claude what needs to be done and in what order. And then the prompt needs to carefully describe the role of each subagent and how the main agent coordinates work with them.

I don't need to spend time researching the approach and then crafting a complex prompt — Claude can do that for me.

ES we have implemented slice 1 (docs/plans/ch06). i want to build the remaining implementation plans in parallel. write a prompt in docs/plans/ch06/prompts/parallel-slices.md that will use subagents effectively to build the plans, coordinating through a central agent

Claude had to think quite a bit for this, and it ended up generating a 350-line prompt (https://github.com/sixeyed/claude-at-work-project/blob/main/docs/plans/ch06/prompts/parallel-slices.md). It's still terse and it covers the full process. Pasting the

prompt into a new session will turn that agent into a coordinator with instructions to start six subagents, one to plan each vertical slice. Listing 6.6 shows how the prompt ensures the subagents don't diverge on common questions.

Listing 6.6 Part of the prompt Claude wrote for me to send to the next Claude

```
Do this yourself, before dispatching anything. Write
`docs/plans/ch06/04-slice-contracts.md`.

Six planners working blind will close the same gap two different ways
wherever slices touch. This document is the answer, and it is the reason
the fan-out is safe. It is not a summary of the delivery plan — it is a
set of **rulings**, each naming the one slice that owns the thing.
```

In the prompt-building session (https://github.com/sixeyed/claude-at-work/blob/main/chapters/ch06/conversations/04-generate-parallel-plan.md), Claude had asked how I wanted to deal with open decisions. The lead agent could be told to stop and confirm with me, but I chose to let the agent make an informed decision and report back instead. I pasted the prompt into a new session and Claude duly got to work, with no complaint about being given such a massive prompt.

The initial piece of work from the lead agent completed quickly, and then it spawned the subagents and waited for them to report back. Figure 6.5 shows the point when the slice 3 agent has just returned, and the lead is waiting on two more to complete.

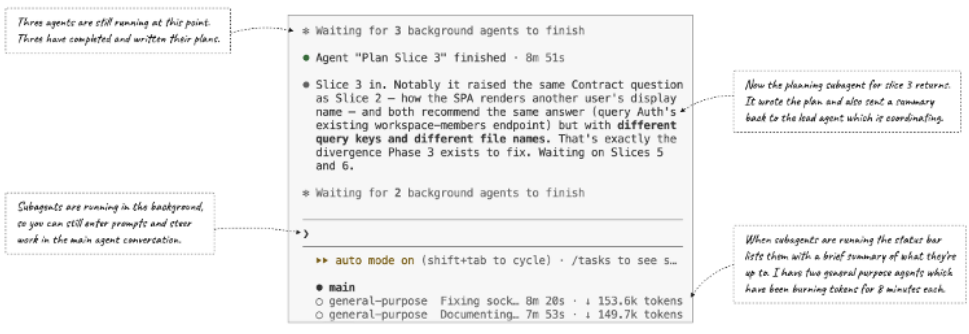


Figure 6.5 Claude Code shows subagent progress in the status bar. The lead agent started six subagents to write the implementation plans. Four have finished and it's waiting on the final two.

With all the plans in, the lead Claude followed the prompt to validate them for inconsistencies and finalize any open decisions itself. Then it dispatched another three subagents to act as reviewers, each with a different focus: gaps between the slice plans, the fidelity of each plan against the documentation, and the feasibility of actually executing the plan. When we built the prompt Claude gave me the choice of where this review happened and it recommended dispatching new subagents, because they would run with no context to bias them.

There was quite a bit of disagreement between the agents, which is what makes this a useful approach. AI is not deterministic and subagents may come up with different conclusions to the lead agent and each other, and this gives Claude the chance to evaluate all the alternatives and choose between them. It took just over 30 minutes to get all six slices planned out and reviewed to Claude's satisfaction.

All that's left is to build the rest of the features.

6.11 Your turn: Implement the remaining features

So now we have a big pile of planning documents, and it seems like we've just shifted the reviewing burden to the left. But that's not really the case. If your sessions have been going in a similar way to mine, the bulk of these plans are lists of gaps in the documentation and how to resolve them. This app doesn't have a product team behind it and I rushed through the initial design reviews to get to the fun part.

In a real project you wouldn't (or shouldn't) be in this position. I deliberately deferred a lot of the unresolved decisions at design time, and the cost of that comes now. But even then, Claude has made sensible decisions for me and these plans are really for other Claudes. My big review will come when the Gherkin files have been created, so that's the next task. You could continue to build the features in sequence, or see what else you could parallelize.

Objective

Finish the implementation of the rest of the feature set. BDD is the key here — focus your reviews on the Gherkin scenarios and make sure that you're confident they are correct and complete. Iterating over the test plan is a huge timesaver compared to iterating over the built feature. When Claude completes each slice, be sure you can deploy the app and run the tests — all the new unit tests as well as the BDD suite — and that everything passes. You should be able to do some exploratory testing for each slice because you know the scenarios from the Gherkin files. Acceptance should be straightforward with the level of confidence you get from the process.

I wasn't ready to drop into sequential builds, so I followed the same pattern we've already used and got Claude to write me a prompt which would spin up subagents to write the Gherkin feature files for each slice. I ran that prompt and ended up with four Gherkin files with scenarios split across the slices. I asked Claude to mark each test with its slice number so I could review them in the correct order. They were all good, only one scenario was missing and Claude added that for me.

Then I started a new session in auto mode and sent in a single prompt:

ES docs/plans/ch06 - we have built slice 1 and the gherkin files are all built and reviewed for the remaining slices. go ahead and finish the build, running implementation plans 02 to 07 in sequence. /goal complete the feature set in line with the ch06 delivery and strategy plans.

The full build took 81 minutes and left me with a usable messaging app, where I can create channels, post messages — and see them appear in real-time on another browser — and give users access to private channels. 7,400 lines of new code and tests, with the suite now totaling 43 BDD tests, 287 integration tests, and 102 unit tests — all passing and easy for me to accept. I would estimate 20 days of human effort for this piece. Add in the 11 from earlier work, and we've saved 30 days of engineering time, by spending one day directing Claude.

Summary

- In plan mode Claude writes documentation rather than code. You decide what level to plan at, and you can iterate over the plan.
- You can use a hierarchy of plans to break down a large feature set into multiple deliveries. It keeps you involved in the decisions but away from the low level details.
- Behavior-driven development is a great way to gain confidence in Claude's work. A close review of test scenarios is much faster than reviewing thousands of lines of code.
- You can include stop signals in your plan. Telling Claude to stop after generating Gherkin files lets you review them, then continue the build session with all its context intact.
- To accelerate delivery Claude Code uses subagents — delegating work to separate Claude sessions which run in parallel.
- You can explicitly tell Claude to use subagents if you want to control parallelism. A good subagent prompt can be complex, but you can ask Claude to generate it.
- The degree of parallelism you can get depends on the dependencies of the features you're working on. In a sequence of dependent tasks you can still do the planning and review in parallel.

AI can produce unit tests quickly, but deciding whether those tests are useful takes a closer look. Through hands-on examples, you'll use AI to analyze code and work through the red/green/refactor cycle, learning to assess generated checks and improve your design.

Chapter 4 from *Software Testing with Generative AI* by Mark Winteringham

AI-assisted testing for developers

This chapter covers

- Developing unit tests and production code with GitHub Copilot
- Developing unit tests and production code with ChatGPT

According to a poll on dev ecosystems conducted by JetBrains in 2022, 81% of respondents have a developer-to-QA ratio greater than one to one. Forty percent reported that they had "less than 1 QA per 10 developers," and only 1% reported that they had "more QAs than developers" (see www.jetbrains.com/lp/devecosystem-2022/testing.)

Understanding and building in quality is essential to delivering value to our users, yet the ratio between development and testing is nearly always imbalanced, and this is so for many reasons. Some organizational leaders choose to educate developers to build in quality with the support of quality coaches, and others simply don't want to invest in roles that advocate for testing and quality. Either way, this situation puts pressure on everyone on a team to deliver high-quality applications. So, how can artificial intelligence (AI) tools help relieve this pressure?

This chapter will specifically focus on how large language model (LLM) AIs, such as GitHub Copilot and ChatGPT, can help developers build quality into our work. Rather than thinking of these AI tools as replacements for developers and testers, we'll learn how they can guide us as we carry out activities that build in quality as we develop, as well as demonstrate how we can use AI tools to improve quality and identify risks when testing is a limited resource to take advantage of.

Setting up Copilot

In this chapter, we'll be looking at examples of GitHub Copilot in use. To familiarize yourself with Copilot before we try out these examples, you can find installation and operation instructions in appendix B.

4.1 Examining the rise of the automated developer

In April 2023, Similarweb, a market competition analysis company, reported that Stack Overflow's traffic in the preceding month had dropped by 14%. According to Similarweb, the cause was a rise in developers switching from Stack Overflow to tools such as GitHub's Copilot and ChatGPT (see https://mng.bz/5Oy4). Whether this is a sign of an ongoing trend of developers jumping ship or an anomaly that will eventually balance out, reports such as this one demonstrate the sweeping changes that LLM-based AI tools (LLMs) are introducing to the role of a developer.

As a tester, it's an interesting change to witness. For many years, as test automation tools became more advanced, I had discussions with developers about whether testing could be replaced by automated testing tools. Now, with LLMs that are trained against billions of lines of code from public and private repositories, conversations have turned toward whether developer roles can be automated. For example, a tool such as ChatGPT can be sent a prompt such as

MW Write me some Java code that sends an HTTP request to a booking API

and it will then use its billions of trained weights and balances to return a working code example like this one:

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;

public class BookingAPIClient {
    public static void main(String[] args) {
        String apiUrl = "https://api.booking.com/endpoint"; // Replace
with the actual API endpoint URL

        try {
            URL url = new URL(apiUrl);
```

```
                HttpURLConnection connection = (HttpURLConnection)
                ➥url.openConnection();

                // Set the request method (GET, POST, etc.)
                connection.setRequestMethod("GET");

                // Set request headers if required
                connection.setRequestProperty("Content-Type",
                "application/json");
                connection.setRequestProperty("Authorization",
                "Bearer YOUR_ACCESS_TOKEN");

                // Get the response code
                int responseCode = connection.getResponseCode();
                System.out.println("Response Code: " + responseCode);

                // Read the response
                BufferedReader reader = new BufferedReader(new
                InputStreamReader(connection.getInputStream()));
                String line;
                StringBuilder response = new StringBuilder();
                while ((line = reader.readLine()) != null) {
                    response.append(line);
                }
                reader.close();

                // Print the response
                System.out.println("Response: " + response.toString());

                // Close the connection
                connection.disconnect();

            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
```

From just one basic prompt, we can use LLMs to create workable code (updating the `apiUrl` to a real API returned a positive result). So, it's not surprising that there are tools that combine prompts attempting to automate development work. Tools such as AutoGPT (https://github.com/Significant-Gravitas/AutoGPT) and MetaGPT (https://github.com/geekan/MetaGPT) have appeared to work as autonomous agents, generating their own prompts based on initial questions to solve complex problems. Although these tools are in their infancy, it's clear why the more hyperbolic claims of developers being automated out of their role are bandied about.

As someone who has spent much of his career explaining why test automation is not a suitable replacement, it's tempting to enjoy the schadenfreude of seeing developers defending their roles in the same way, but instead, it's more valuable to learn from the experiences of testers and the automation topic. Just as a tester's role can't be fully automated, neither can a developer's role. A development role is more than just the code

that is produced. The solutions that developers create are a product of analytical skills, problem-solving, and design thinking. LLM tools give the impression of having these skills, which is not true.

Instead, developers find success with LLM tools by using them to enhance their own abilities: they may use tools such as Copilot to quickly and effectively create code that they want to build or seek advice from ChatGPT to solve issues or learn new APIs. These principles can also be applied to improving a developer's ability to build quality into an application. By combining techniques such as test-driven design (TDD) or the power of LLMs, developers can increase their productivity while ensuring that their analytical and design skills lead the charge. To help demonstrate this symbiosis, let's explore the following two examples:

- Using Copilot to rapidly generate unit checks and production code for TDD loops
- Emulating pairing with a simulated developer, courtesy of ChatGPT

Through the examples, you'll learn to set up and use these LLM tools, as well as appreciate the balance that is possible to strike between the power of AI and the abilities of developers.

Experiences may vary

Given that Copilot relies on predictive algorithms that are frequently trained on newly added code and updated APIs/libraries, it's worth highlighting that the output and experience you have when following the upcoming examples may be different from what has been recorded. Keep in mind that the goal of this chapter is not to replicate the examples 100%, but rather for you to become comfortable with using LLMs to assist our work in a way that helps us build in quality.

4.2 Pairing with LLMs

We've already seen that LLMs are probabilistic in nature, and as a result, it can be useful to think of them as outputting simulations of roles rather than inhabiting specific roles. An LLM has no more awareness of being a software tester than it does of being a restauranteur. But with prompt engineering, we can create prompts that frame an LLM's probabilistic output to simulate a role, helping us to create rubber ducks to interact with. This can be useful in a development capacity when testing resources are limited in either availability or capability. So, let's take a look at a couple of sample prompts we can use to elicit feedback that can help us improve the quality of our work and our products.

Wait, what—rubber duck?

When facing a problem that we have no solution for, it can help to verbalize the problem to others to find an answer. By articulating the problem to another person, we sometimes

find that the solution presents itself to us. However, there isn't always an opportunity to speak with colleagues; therefore, some developers will verbalize their issues to a rubber duck (or another item). Although we're sharing our challenges with an inanimate object, the experience is the same: verbalizing our challenges tricks our brains into finding solutions.

4.2.1 Analyzing ideas

Our first prompt is inspired by the shift-left testing mindset that some teams apply to help build quality into applications as early as possible. *Shift-left* testing means bringing testing analysis earlier into the development process, ideally around the point at which ideas are being discussed and details clarified. This process results in many questions being asked that help us identify issues earlier, deliver valuable features, and improve the quality of our products. Shift-left activities are an invaluable approach to improving quality and should be encouraged regardless of the use of AI in the development process. However, by introducing LLMs into the process, we can use prompts to generate questions that might reveal assumptions or misunderstandings or questions that we can simply disregard.

Let's take a look at a sample prompt that can be used to rapidly generate questions that might be useful. We use delimiters to inform the LLM where the data to analyze is found:

MW You are a software testing expert. Analyze the text delimited by triple hashes and generate questions that will clarify ambiguities in the text.

We then set a series of rules around how questions are asked and output:

MW *Questions will be generated by using the What, Where, Why, When, Who and How heuristic

* Multiple questions can be identified for each quality characteristic

* The quality characteristics we care about are: Accuracy, Security and Accessibility

* Output questions in the format of Question - Explanation

Few-shot prompting allows us to create an example of how we expect the output to look:

MW Example:

What does relevant mean?

The acceptance criteria says relevant search results, but how do we determine what is relevant and what isn't?

We then enter the data for analysis into the delimited section of the prompt:

MW ###

As a B&B owner

I want to be able to manage user access

So that I can allow different levels of access to multiple users

Features:

– DB is configured with an Admin user

– Guests are asked to create an account to make a booking

– Admin can see the accounts created on the platform and can delete them

###

The prompt outlines instructions for an LLM to analyze a user story and acceptance criteria and return a list of questions for us to consider. Notice how we also provide context in the quality characteristics instruction. If we want to focus on different characteristics, we can update these as well.

Entering this prompt into ChatGPT returned the following result:



Accuracy:

1. What criteria are used to determine the accuracy of user access management?

   Explanation: The text mentions managing user access, but it doesn't specify how accuracy is ensured in granting different levels of access to multiple users.

2. How is the system ensuring that the right access levels are assigned to users?

   Explanation: The text refers to allowing different levels of access to multiple users, but it doesn't explain the mechanism for guaranteeing the accuracy of access permissions.

Security:

1. Who has access to the database configured with an Admin user?

   Explanation: The text mentions configuring the database with an Admin user, but it doesn't specify who has access to this Admin account and how its security is maintained.

2. How is the platform protecting user account information and data?

   Explanation: The text talks about guests creating accounts for bookings, but it lacks details about the security measures implemented to safeguard user data.

3. What security measures are in place to prevent unauthorized access to user accounts?

   Explanation: The text mentions that Admin can see and delete accounts, but it doesn't explain how unauthorized access to such account management functionalities is prevented.

Accessibility:

1. What levels of accessibility are provided to different types of users?

Explanation: The text mentions managing user access with different levels, but it doesn't clarify what those levels are and how they vary based on user types.

2. How are guests guided to create accounts for booking, and is the process accessible to all users?

   Explanation: The text states that guests are asked to create accounts, but it doesn't specify how this process is carried out and if it meets accessibility standards.

3. What measures are taken to ensure the platform's accessibility for users with disabilities?

   Explanation: The text doesn't provide information on how accessibility is addressed for users with disabilities, and it's essential to understand the measures taken to accommodate their needs.

4. Where can we find information about the platform's accessibility features?

   Explanation: The text mentions nothing about the platform's accessibility features, so it would be helpful to know where this information is available.

Reading through the response, we can see that there is an interesting collection of questions ranging in quality. For example, consider this question:

MW How are guests guided to create accounts for booking, and is the process accessible to all users?

Explanation: The text states that guests are asked to create accounts, but it doesn't specify how this process is carried out and if it meets accessibility standards.

This is a helpful question that encourages us to consider in more detail the statement "Guests are asked to create an account to make a booking, "found in our user story. We could consider this question and think about how we would design the booking process to make it accessible to all types of users.

However, other questions are perhaps not quite as good. For example,

MW What criteria are used to determine the accuracy of user access management?

Explanation: The text mentions managing user access, but it doesn't specify how accuracy is ensured in granting different levels of access to multiple users.

What makes this question problematic is its reference to the quality characteristic *accuracy*. Typically, accuracy would be more focused on the accuracy of data processed or stored (think bank interest rate calculations). Instead, the phrase "accuracy of user access" feels like an odd way to describe the rules set around users and what they can access. Ultimately, it's up to us to evaluate each question for suitability and use. Some questions can encourage us to build products that are more closely aligned with what a user wants and help us avoid errors, whereas other generated questions will either make little sense or cover topics already considered.

We'll come back to this prompt and how we might use it during the development of a feature, but first, let's take a look at how we can repurpose this prompt to review our code.

4.2.2 Analyzing code

Just as we can prompt an LLM to analyze written-down ideas, we can also have it review code and help us identify risks. Using this prompt is akin to simulating the role of a developer or tester that you are pairing with by having them analyze your work as you develop to offer suggestions for consideration. Let's check out a prompt that could be of use for this type of activity. We use delimiters to signal where the code will be, as well as check for assumptions if no code is given:

MW You are a software testing expert. Analyze the code delimited by triple hashes and identify risks that might impact the code. If no code is provided, respond with "No risks could be found."

We then provide instructions around quality characteristics and output structure:

MW * Risks must be related to the quality characteristics: Performance, security and interoperability

* Multiple risks can be identified for each quality characteristic

* Output identified risks in the format of Risk - Quality Characteristic - Explanation

Few-shot prompting allows us to give an example of output structure:

MW Example:

Overflow - Security - Adding in large integers as parameters might cause the method to trigger a buffer overflow.

Code is provided within the instructed delimiters:

```
###
public BookingResult createBooking(Booking bookingToCreate) throws
➥SQLException {
    if(dateCheckValidator.isValid(bookingToCreate.getBookingDates())) {
        if (bookingDB.checkForBookingConflict(bookingToCreate)) {
            return new BookingResult(HttpStatus.CONFLICT);
        } else {
            CreatedBooking createdBooking =
            ➥bookingDB.create(bookingToCreate);

            if(bookingToCreate.getEmail() != null &&
            ➥bookingToCreate.getPhone() != null){
                MessageBuilder messageBuilder = new MessageBuilder();
                Message message = messageBuilder.build(bookingToCreate);

                messageRequests.postMessage(message);
            }

        return new BookingResult(createdBooking,
```

```
            ➥HttpStatus.CREATED);
        }
    } else {
        return new BookingResult(HttpStatus.CONFLICT);
    }
}
###
```

After I sent the prompt with the sample code and quality characteristics to ChatGPT, the following output was returned:

Based on the provided code snippet, let's analyze the potential risks that might affect the quality characteristics of Performance, Security, and Interoperability:

- Performance - Nested Validation:
  - Risk: The nested validation checks (`dateCheckValidator.isValid` and `bookingDB.checkForBookingConflict`) may lead to performance issues, especially when dealing with a large number of bookings.
  - Explanation: If the system has to process a substantial amount of bookings, the nested validation can result in increased execution time and resource consumption.
- Performance - Message Posting:
  - Risk: The message posting (`messageRequests.postMessage`) might introduce performance bottlenecks, affecting the overall system responsiveness.
  - Explanation: If the message posting process involves network communication or I/O operations, it could slow down the execution of the `createBooking` method.
- Security - SQL Injection:
  - Risk: The code is susceptible to SQL injection attacks as it directly uses the `bookingToCreate` object in the database queries.
  - Explanation: Malicious input in the `bookingToCreate` object could manipulate the SQL queries and compromise the integrity and security of the database.
- Security - Inadequate Authorization:
  - Risk: The code does not perform explicit authorization checks before creating bookings, potentially allowing unauthorized users to create bookings.
  - Explanation: Lack of proper authorization checks can lead to unauthorized access and misuse of the system's booking creation functionality.
- Interoperability - Platform Dependencies:
  - Risk: The code might have dependencies on specific platforms or libraries, limiting its interoperability with other systems or environments.
  - Explanation: If the `MessageBuilder` or `messageRequests` rely on platform-specific features or external libraries, it could cause issues when running the code in different environments.

- Interoperability - Error Codes:
  - Risk: The code returns HTTP status codes directly, which may not be compatible with other systems that expect different error code formats.
  - Explanation: If external systems expect error codes in a different format (e.g., JSON or XML), using HTTP status codes may hinder interoperability.

Similar to the previous prompt, which generated questions around written requirements, the quality of these different risks varies. For example, the risk "Performance - Message Posting" feels quite abstract, as we can see in its explanation:



If the message posting process involves network communication or I/O operations, it could slow down the execution of the `createBooking` method.

Suggesting risks around network communication or I/O operations feels vague because it might refer to a range of implementations, some of which might relate to our work and some that might not.

However, other risks are more concrete and potentially useful—for example, "Security - Inadequate Authorization," which highlights the following:



The code does not perform explicit authorization checks before creating bookings, potentially allowing unauthorized users to create bookings.

This risk feels more concrete because it refers to actions that are carried out in our method and the absence of potentially important checks within it. Of course, we may carry out authorization checks elsewhere, but the information it has given highlighted an explicit activity that we might need to discuss further to improve the security of our booking feature.

Generating more ideas

So far, we've looked at singular prompts to send to LLMs, which gives us useful responses to review. But what if we want to solicit more questions and risks? We simply ask by submitting an additional prompt, such as "Generate more questions" or "Identify further risks." Be wary, though, as this has diminishing returns. An LLM will try to fulfill our requests at the risk of increasing hallucinations. So, as options start to dry out, we may see more suggestions that are less connected to the ideas and code we wanted feedback on in the first place.

4.2.3 Recognizing that a simulation is better than nothing at all

When testing is discussed, the focus is typically on the production and execution of test cases. But a highly trained and experienced tester delivers value by using their critical and lateral thinking skills and asks questions that help view solutions in new ways and reveal potential problems. The prompts we've looked at can offer a simulation of that process. However, it's important to remember that LLMs don't have these critical

and lateral thinking skills and that the questions and risks generated come from the instruction of our prompts. Instead, these types of prompts can offer a lightweight way to simulate the experience of pairing with testers or other developers when the opportunity to pair is unavailable. The key is to develop an eye for generated questions to determine which are of use.

4.3 Building in quality with AI assistance

So far, we've looked at prompts as singular activities, but now, let's turn our attention to how the prompts we've recently learned and other LLM assistant tools can be used in conjunction with TDD to help us build in quality.

Although TDD is not strictly a testing activity when compared to other testing activities, TDD that is carried out correctly helps guide developers to build quality into products. To recap, the process of TDD is to use unit-checking tools to first create failing checks and then just enough production code to make the check `pass` (and fix any other checks that may have failed). Once all our checks are passing, we can refactor our production code while ensuring all our checks are green. Once that's complete, we start the loop again until our work is complete, as demonstrated in figure 4.1.

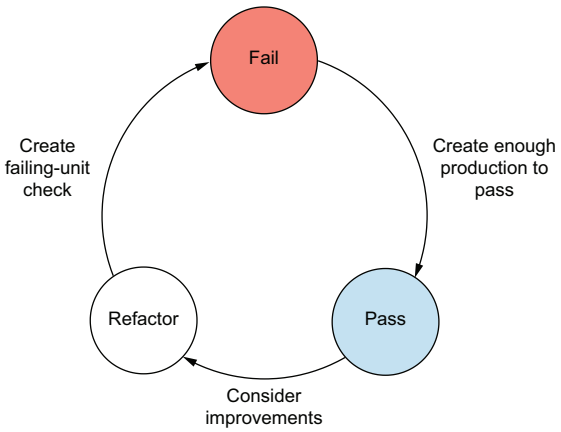


Figure 4.1 The red/green/refactor TDD cycle

What's all this about checks?

In our Automation in Testing training, Richard Bradshaw and I make a distinction between human-led and tool-led testing. We call the latter type *automated checking* because the tools can only assert explicit actions or data that we codify into our automation. This distinction helps us better appreciate that automated tools such as unit-checking frameworks are excellent at rapidly checking small, specific changes in a product but are unable to tell us more about a system beyond its assertions. Humans, however, are slower and less deterministic in testing, although we are much more efficient at identifying many events happening simultaneously. Hence, this is why tools check and humans test. One is not better than the other, and hopefully, this book will demonstrate that we can have the best success when combining both.

This approach enables us to design products that are highly testable while ensuring we deliver what the business or end user desires.

Although its benefits are manifold (improving code design, making it more testable, and most importantly, improving its quality), some developers find it hard to adopt the TDD approach, believing that it slows down development as we create unit checks for each specific section of production code added to our system. However, with tools such as Copilot, we can learn how to establish a balance in which the tools enable us to rapidly create unit checks and build in quality.

> **Setting up Copilot**
>
> This chapter assumes that you have installed and configured the Copilot plugin within your IDE. If you haven't already completed the setup process, you can find installation instructions in appendix B.

4.4 Creating our first TDD loop with LLMs

To demonstrate, let's undergo the process of creating part of a feature for a sample timesheet manager:

- As a timesheet user, I want to be able to track time against specific projects so that I can keep a history of how much time I have spent working on individual projects.
- Features:
  - Time is tracked in half hour units.
  - To complete a timesheet, a project and amount of time must be entered.
  - Only I should be able to see my tracked time.

You can review the test and production code created as part of this example in the following GitHub repository: https://mng.bz/6YMG.

4.4.1 Preparing the work

Before starting to create my timesheet manager feature, I wanted to think about how I would go about building it. To get help with this process and encourage me to think about the solution from various angles, I took the first prompt we explored earlier in this chapter and entered my context into it. Specifically, I changed the quality characteristics to accuracy and consistency and added the user story into the delimited section at the bottom of the prompt:

MW You are a software testing expert. Analyze the text delimited by triple hashes and generate questions that will clarify ambiguities in the text.

* Questions will be generated by using the What, Where, Why, When, Who and How heuristic
* Multiple questions can be identified for each quality characteristic
* The quality characteristics we care about are: Accuracy and Consistency
* Output questions in the format of Question - Explanation

Example:

What does relevant mean?

The acceptance criteria say relevant search results, but how do we determine what is relevant and what isn't?

###

As a timesheet user

I want to be able to track time against specific projects

So that I can keep a history of how much time I have spent working on individual projects

Features:

- Time is tracked in half hour units

- To complete a timesheet a project and amount of time must be entered

- Only I should be able to see my tracked time

###

Sending this prompt to ChatGPT returned the following response:



Accuracy:

- What is the required precision for tracking time in half-hour units? - The text mentions that time is tracked in half-hour units. It's important to know the level of accuracy required for this tracking.
- How will the system handle invalid time entries? - Since time must be entered to complete a timesheet, what measures are in place to ensure that only valid time entries are accepted?
- What validation mechanisms are in place to prevent users from entering incorrect time data? - To maintain accuracy, it's essential to know how the system will validate and handle potential data entry errors.
- How will the system handle time zone differences for users in different locations? - This is important to ensure that the tracked time is consistent and accurate for users in various time zones.

Consistency:

- Where will the history of tracked time be stored and accessed? - To ensure consistency, it's important to know where the data will be stored and how users can access it.
- What are the rules for displaying and formatting tracked time information? - Consistency in displaying the tracked time across the system is crucial for a seamless user experience.

- Who is responsible for verifying and maintaining the consistency of tracked time data? – It's essential to assign responsibility for data integrity and consistency to avoid discrepancies.
- When and how often will the timesheet data be synchronized or updated? - Ensuring consistency in real-time updates or scheduled syncs is vital to avoid discrepancies between users' data.
- How will the system handle potential conflicts if multiple users try to update the same project's timesheet simultaneously? - Maintaining consistency in data updates when multiple users are involved is critical to prevent data loss or inaccuracies.
- What security measures are in place to protect the data and ensure its integrity? - Consistency also involves safeguarding the data from unauthorized access or tampering.

What are quality characteristics?

Quality means very different things to different people and is a multifaceted concept. Therefore, when attempting to understand what quality means to a stakeholder or end user, we break out ways of thinking about it into different characteristics. For example, quality characteristics might include look and feel, usability, compliance, and much more. Different projects will have different quality characteristics that we prioritize, and it's up to us as teams to identify what quality characteristics are of priority to us and our users.

From here, I read each of the questions sent back to me and noted the ones that stood out as raising points I hadn't considered. Let's go through some of them and explore my thinking behind each one:

MW How will the system handle invalid time entries? / What validation mechanisms are in place to prevent users from entering incorrect time data?

These two questions felt very similar, but they highlighted that I will, at some point, need to add date–time handling into the code to prevent problems around invalid entries.

MW How will the system handle time zone differences for users in different locations?

It's a good question, but not relevant to this work as I plan to just track the amount of time carried out and not when it was carried out (at least for now).

MW What are the rules for displaying and formatting tracked time information?

Connected to the previous question, I opted for simply tracking the amount of time that had elapsed and not specifically when it had occurred, meaning that the displaying of information is relatively straightforward.

MW How will the system handle potential conflicts if multiple users try to update the same project's timesheet simultaneously?

At this time, there is no multiple-user feature, although it might be introduced later, so I make a note to consider this for future iterations.

Many more questions could be asked and addressed, but what this process demonstrates is that, by generating different questions, I am required to think about various aspects of this feature's design. Some I can choose to rule out, or shelve for later conversation, and some help me to improve the quality of my feature by raising my awareness to validation and date–time formatting.

4.4.2 Loop 1: Save a timesheet entry

With the questions from the previous prompt in my mind, I start my first loop by creating an empty maven project within my IDE before heading to the `pom.xml` to add my required dependencies. I require JUnit 5 to build my unit checks, so I entered the comment

MW
```
<!--    Junit jupiter engine-->
```

Copilot returns the following code block:


```
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-engine</artifactId>
        <version>5.8.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

At the time of writing, the proposed version of `junit-jupiter-engine` is an older version, so I update to `5.9.2` as shown here:

MW
```
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-engine</artifactId>
        <version>5.9.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

This required update is a good example of how data provenance can affect what is being suggested. While constantly being trained on code that is being pushed to GitHub, Copilot runs the risk of lagging behind the latest versions of libraries and APIs. So, we need to be vigilant when adding in dependencies via AI code assistants to ensure they are suggesting the version we want.

Furthermore, what is demonstrated here is how the Copilot IDE plugin will take code (or in this case a code comment) I have written, add it to a prompt, and then send it to the Copilot LLM to process. The LLM then sends back to the IDE plugin a suggestion of what to enter next, and the plugin then presents me with the suggestion. I now have the option to either accept the suggestion by hitting Tab or continue to write my own code.

Much like other LLMs, Copilot is triggered by prompts. But unlike a chat-based LLM (such as ChatGPT), Copilot is tuned to process code-based rather than text-based prompts. This distinction is important because it highlights both the benefits of different types of fine-tuned LLMs for different activities, as well as the need to write prompts that work for different prompt types. This is why TDD and AI assistants are an interesting combination to work with. The unit checks we create not only frame the design of our work but also serve as prompts to inform how we want to implement features.

For example, for our first check, I prompt Copilot to create my first unit check for me by adding the following comment into a new class titled `TimesheetTest` in `src/test/java`:




```
// Test that when a timesheet is submitted with a project name
➥and hours it returns true
```

This triggers Copilot to suggest the following unit check:



```
@Test
public void testSubmitTimesheetWithProjectNameAndHours() {
    Timesheet timesheet = new Timesheet();
    boolean result = timesheet.submitTimesheet("Project 1", 8);
    assertEquals(true, result);
}
```

Looking at this suggested check, we see that it has the necessary details we require. It's given the class I intend to build a sensible name of `Timesheet` and suggested the method `submitTimesheet` with the correct parameters of project name and hours. This check is enough of a prompt that when I create a new class `Timesheet` in `src/main/java`, Copilot suggests



```
public boolean submitTimesheet(String projectName, int duration) {
    return true;
}
```

The unit check served as a prompt to Copilot to create my production code, and although what has been produced is not exactly exciting, the check has been fulfilled. We can also see that Copilot works by building prompts on prompts to get to what we want. First, I created a prompt in the form of a code comment to help Copilot suggest the code for the unit check, and then that unit check served as a prompt for the suggested production code. As the unit checks grow, so will the prompts and directions to Copilot on what to build. Now, let's look at how I used this process to build out the `submitTimesheet` method.

4.4.3 Loop 2: Retrieve a timesheet entry

With the first loop, I created the skeleton of my project, but at the moment, the actual production code bears little fruit. So, with the second loop, I looked to add more features to the `Timesheet` class to help flesh out the `submitTimesheet` method. Again, I start by adding a new comment prompt to `TimesheetTest`:

MW
```
// Test that when timesheets are added they can be retrieved as a list
```

And upon creating a new line, Copilot returned the following check as a suggestion:

```
@Test
public void testAddTimesheetsToList() {
    Timesheet timesheet = new Timesheet();
    timesheet.submitTimesheet("Project 1", 8);
    timesheet.submitTimesheet("Project 2", 8);
    timesheet.submitTimesheet("Project 3", 8);
    assertEquals(3, timesheet.getTimesheets().size());
}
```

This time, the generated check is using the `submitTimesheet` method to create some data that I can then verify using the newly created `getTimesheets` method. So, heading over to the `Timesheet` class, I use Copilot to generate the necessary `getTimesheets` method, which returns

```
public Collection<Timesheet> getTimesheets() {
    return null;
}
```

Although it's helpful that it created the method signature for me, the internals of the method are not so useful. However, I add the following to the top of the class:

MW
```
private HashMap<String, Integer> timesheets = new HashMap<>();
```

Then I return to the `submitTimesheet` method and remove `return true` from inside the method. When I enter the keyword `if` into the statement, Copilot returns

```
public boolean submitTimesheet(String projectName, int duration) {
    if (projectName != null && duration > 0) {
        timesheets.put(projectName, duration);
        return true;
    } else {
        return false;
    }
}
```

This is a useful demonstration of how a project requires a degree of warmup before the LLM can return more accurate suggestions when working with Copilot. By returning to the `submitTimesheet` method, removing its contents, and then waiting for Copilot to respond, I get a refreshed response from Copilot based on the new details I've

provided. Just as a prompt for a chat LLM requires clear and explicit details, so does Copilot. The initial stock methods that were returned from Copilot for `Timesheet` are likely because there is little context to work with. Essentially, the prompts sent to Copilot lack information. But as I create new unit checks and additional code to show our intent in our code, the suggestions become more useful.

This is also demonstrated when I delete the `getTimesheets` method entirely. With the new `HashMap` added to the class, my updated code prompts Copilot to return an updated version of the method:

```
public Collection<Integer> getTimesheets() {
    return timesheets.values();
}
```

Running both of my unit checks after this loop, I can see both checks are passing and now have working code that is a bit more feature rich than when I started the loop.

4.4.4 Loop 3: Calculating times for a project

For the final loop, I went back to the questions returned from ChatGPT and thought about the question:

MW How will the system handle potential conflicts if multiple users try to update the same project's timesheet simultaneously?

Although there is no functionality for multiple users at this point in the project, the question from ChatGPT did make me think about how to handle multiple timesheet submissions to the same project. So, I created my third unit check to handle this scenario by once again creating a code comment to prompt Copilot for another unit check:

```
MW // Test that the total hours worked can be calculated from a list
➥of timesheets from one project
```

It returned the following unit check:

```
@Test
public void testCalculateTotalHoursWorked() {
    Timesheet timesheet = new Timesheet();
    timesheet.submitTimesheet("Project 1", 8);
    timesheet.submitTimesheet("Project 1", 8);
    timesheet.submitTimesheet("Project 1", 8);
    assertEquals(24, timesheet.getTimesheets().stream()
.mapToInt(Integer::intValue)
.sum());
}
```

This time, the suggested check is close, but I want the `Timesheet` class to return a total, not have a total processed within the unit check itself. So, I opted to update the check to explicitly state there is a method for getting total timesheet hours:

MW
```
@Test
public void testCalculateTotalHoursWorked() {
    Timesheet timesheet = new Timesheet();
    timesheet.submitTimesheet("Project 1", 8);
    timesheet.submitTimesheet("Project 1", 8);
    timesheet.submitTimesheet("Project 1", 8);

    int total = timesheet.getTotalTimesheetHours("Project 1");

    assertEquals(24, total);
}
```

Heading back over to `Timesheet` and waiting for Copilot, the LLM returns the following code:


```
public int getTotalTimesheetHours(String projectName) {
    return timesheets.get(projectName);
}
```

This allows the code to compile, but my new unit check fails because the project count isn't being updated. Instead, because I am using a `HashMap`, the count is being overwritten. I need to update `submitTimesheet` to handle duplicate projects, which I do by dropping the following comment into `submitTimesheet`, just after the initial `if` statement:

MW
```
// Check to see if project already exists
```

Dropping in the code comment returns the following suggested code:


```
if (timesheets.containsKey(projectName)) {
    // If it does, add the new duration to the existing total
    duration += timesheets.get(projectName);
}
```

This expands the `submitTimesheet` function to allow it to increase the duration of existing projects in a timesheet:


```
public boolean submitTimesheet(String projectName, int duration) {
    if (projectName != null && duration > 0) {
        // Check to see if project already exists
        if (timesheets.containsKey(projectName)) {
            // If it does, add the new duration to the existing total
            duration += timesheets.get(projectName);
        }
        timesheets.put(projectName, duration);
        return true;
    } else {
        return false;
    }
}
```

A final run of my three-unit checks confirms that they are all passing and that I have all the features I want in my `Timesheet` class. However, the code is starting to become more complex, so I turn my attention to refactoring my code with the use of ChatGPT.

4.4.5 Refactoring code

One benefit of pairing with others is the ability for a pair partner to see potential problems and risks that we might not see. However, in the absence of a pair partner, I chose to use the second of the prompts we explored earlier in this chapter—specifically, the prompt that analyzes code and returns suggestions. Taking the prompt and adding my code and the quality characteristics' accuracy and consistency, I sent the following prompt to ChatGPT. I slightly updated the instructions to make it clear I am providing Java code:

MW You are a software testing expert. Analyze the Java code delimited by triple hashes and identify risks that might impact the code. If no code is provided, respond with "No risks could be found"

I update the additional instructions with my quality characteristics:

MW * Risks must be related to the quality characteristics: Accuracy and Consistency

* Multiple risks can be identified for each quality characteristic

* Output identified risks in the format of Risk - Quality Characteristic - Explanation

An example is provided to demonstrate the format of the response:

MW Example:

Overflow - Security - Adding in large integers as parameters might cause the method to trigger a buffer overflow.

The code I created from my earlier loops is added in the delimited section:

MW
```
###
public class Timesheet {

    private HashMap<String, Integer> timesheets = new HashMap<>();

    public boolean submitTimesheet(String projectName, int duration) {
        if (projectName != null && duration > 0) {
            // Check to see if project already exists
            if (timesheets.containsKey(projectName)) {
                // If it does, add the new duration to the
                // existing total
                duration += timesheets.get(projectName);
            }
            timesheets.put(projectName, duration);
            return true;
        } else {
            return false;
```

```
        }
    }

    public Collection<Integer> getTimesheets() {
        return timesheets.values();
    }

    public int getTotalTimesheetHours(String projectName) {
        return timesheets.get(projectName);
    }
}
###
```

Sending the prompt returned the following risks that I might want to consider:



Risk 1 - Accuracy - `submitTimesheet` method can overwrite existing duration:

The `submitTimesheet` method allows adding a new duration to an existing project's total if the project already exists in the timesheets. However, this approach may not always ensure accuracy. If the user intends to submit a correction or update the duration for a specific project, this method will not provide accurate results. It merely adds the new duration to the existing one without considering the possibility of editing or deleting a previous entry.

Risk 2 - Consistency - Null key in timesheets HashMap:

The timesheets HashMap allows null keys. This might lead to inconsistency issues when querying for timesheet information by project name. If a project with a null name is mistakenly added to the timesheets, it may cause issues in subsequent operations, leading to unexpected behavior.

Risk 3 - Consistency - Lack of validation for project names:

The `submitTimesheet` method does not validate the project name before adding it to the timesheets HashMap. This lack of validation might result in inconsistent data, such as duplicate or improperly formatted project names. For example, if two different variations of the same project name (e.g., `"ProjectA"` and `"projecta"`) are added, the system might treat them as separate projects, leading to inconsistencies in reporting or querying.

Risk 4 - Accuracy - `getTotalTimesheetHours` method may throw `NullPointerException`:

The `getTotalTimesheetHours` method directly calls `timesheets.get(projectName)`, which may throw a `NullPointerException` if the given `projectName` is not present in the timesheets HashMap. This can lead to inaccurate results if the method is used without checking for the existence of the project name in the timesheets before calling this method.

Risk 5 - Accuracy - Return type of `getTotalTimesheetHours` method:

The `getTotalTimesheetHours` method returns an int data type, which might lead to accuracy issues if the actual total timesheet hours for a project exceed the maximum

value that an int can hold. This can result in data loss due to overflow if the sum of durations for a project exceeds `Integer.MAX_VALUE`.

Risk 6 - Consistency - Lack of synchronization in a multi-threaded environment:

If this `Timesheet` class is used in a multi-threaded environment, there is a risk of inconsistent behavior since the methods of this class are not synchronized. This can lead to data corruption or incorrect calculations if multiple threads concurrently access and modify the timesheets HashMap.

Unlike the questions prompt, this response appears to call out specific items of my code and gives reasons for how things might go wrong. With this response, I have the following options:

1. Review each risk on a case-by-case basis and then mitigate the ones I feel are important and ignore the others.
2. Ignore the proposed risks entirely, or perhaps send another prompt to ChatGPT to see whether there are more risks.
3. Use the response from ChatGPT as a prompt in and of itself to help me refactor my code.

For the purposes of this demonstration, I chose to go with the third option and send an additional prompt to ChatGPT:

MW Refactor the code to mitigate the risks suggested. Check to see that each risk has been mitigated before outputting the solution.

Notice how I used the time-to-think principle to get the LLM to check whether each risk has actually been mitigated by the refactored code it suggested. This helps increase the likelihood that the code that's returned will be useful. Sending this prompt to ChatGPT returned the following code:

```
import java.util.Collection;
import java.util.concurrent.ConcurrentHashMap;

public class Timesheet {

    private ConcurrentHashMap<String, Long> timesheets =
new ConcurrentHashMap<>();

    public boolean submitTimesheet(String projectName, long duration) {
        if (projectName != null && duration > 0) {
            // Perform validation on project name by converting it to
            // lowercase
            projectName = projectName.toLowerCase();

            // Check to see if project already exists
            timesheets.put(projectName, timesheets
            .getOrDefault(projectName, 0L) + duration);
            return true;
```

Changes HashMap to ConcurrentHashMap

Adds in additional code to lowercase project names

```
        } else {
            return false;
        }
    }

    public Collection<Long> getTimesheets() {
        return timesheets.values();
    }

    public long getTotalTimesheetHours(String projectName) {
        return timesheets.getOrDefault(projectName.toLowerCase(), 0L);
    }
}
```

Converts the int data type to long

To check whether these modifications hold up, I copied the suggested code into `Timesheet`, modified `testCalculateTotalHoursWorked` to take a `long` instead of an `int` in `TimesheetTest`, and discovered that my checks still pass and I refactored my code.

However, has this improved my code? To confirm, I run the original code analysis prompt with my updated code again and receive new risks:



Risk - Accuracy - Case Sensitivity Issue:

Explanation: The method `submitTimesheet` converts the `projectName` to lowercase before using it as a key in the timesheets map. This conversion might lead to inaccurate data retrieval if there are projects with the same name but different letter cases (e.g., `"ProjectA"` and `"projecta"` will be considered the same project).

Risk - Consistency - Inconsistent Data Entry:

Explanation: The `submitTimesheet` method does not enforce consistent data entry for `projectName` values. If different users enter project names in various formats (e.g., `"Project A"`, `"project-A"`, `"project_A"`), it could lead to multiple entries for what should be the same project in the timesheets map.

Risk - Accuracy - Missing Validation for Duration:

Explanation: Although the method `submitTimesheet` checks for a positive duration, it does not handle cases where the duration is 0 or negative. This could lead to incorrect data being recorded in the timesheets map.

Risk - Consistency - Lack of Input Sanitization:

Explanation: The `submitTimesheet` method does not sanitize the `projectName` input, which could lead to inconsistent data due to invalid characters or unexpected inputs.

Risk - Accuracy - Potential Data Loss:

Explanation: The method `submitTimesheet` returns a boolean value, but it does not provide any feedback or error handling mechanism if the submission fails. This lack

of feedback could lead to data loss if the submission fails silently, and the user is not aware of the issue.

What is interesting in this response is I can confirm that some risks have been mitigated such as



Risk 5 - Accuracy - Return type of `getTotalTimesheetHours` method:

The `getTotalTimesheetHours` method returns an int data type, which might lead to accuracy issues if the actual total timesheet hours for a project exceed the maximum value that an int can hold. This can result in data loss due to overflow if the sum of durations for a project exceeds `Integer.MAX_VALUE`.

However, it appears that there are still some risks that haven't mitigated. For example, in the first list of risks I received the following risk:



Risk 3 - Consistency - Lack of validation for project names:

The `submitTimesheet` method does not validate the project name before adding it to the timesheets HashMap. This lack of validation might result in inconsistent data, such as duplicate or improperly formatted project names. For example, if two different variations of the same project name (e.g., `"ProjectA"` and `"projecta"`) are added, the system might treat them as separate projects, leading to inconsistencies in reporting or querying.

It was handled by ChatGPT implementing a `lowerCase` method to help sanitize the project name. However, on the second analysis, I was returned the following:



Risk - Consistency - Inconsistent Data Entry:

Explanation: The `submitTimesheet` method does not enforce consistent data entry for `projectName` values. If different users enter project names in various formats (e.g., `"Project A"`, `"project-A"`, `"project_A"`), it could lead to multiple entries for what should be the same project in the timesheets map.

This risk is very similar to the original, allegedly mitigated, risk. It feels like this additional risk around inconsistent data entry should have been handled properly when my code was refactored. I could once again ask the LLM to refactor my code for me, but instead given the potential for going around in circles with the LLM, it would be sensible for me to take the lead and fix the problems myself. This is an important skill to develop—knowing when to rely on an LLM and when to take charge.

The reason this choice is important can be highlighted by one of the other suggested risks that came from the second round of analysis. Specifically,



Risk - Accuracy - Missing Validation for Duration:

Explanation: Although the method `submitTimesheet` checks for a positive duration, it does not handle cases where the duration is 0 or negative. This could lead to incorrect data being recorded in the timesheets map.

This sounds like a convincing risk, but it's a demonstration of a hallucination. As the code stands, if the duration is ≥0, the method simply returns false and bails out of the timesheet storage:



```
if (projectName != null && duration > 0)
```

LLMs can be biased sometimes to prioritize giving an answer regardless of its quality, meaning the more times we ask an LLM to analyze our code, the more likely it will start to generate hallucinations to give the impression that it's producing useful results rather than returning with a response that no useful information can be shared. This is why we must keep careful tabs on when it's beneficial to use an LLM.

At this point, I chose to stop the use case as what we've covered demonstrates the way in which different types of LLMs can help me in different ways. Copilot offers the ability to rapidly generate code, but it requires code-based prompts to help it with its suggestions. What this means is that if we're working on a fresh project that has very little production code for our LLM to analyze, we're more likely to get results from Copilot that aren't useful to us. So, to improve the quality of the output from Copilot, we give it more context using unit checks. This helps not only guide Copilot in the building of our code, but also provides us with the benefits of TDD including well-designed, testable code.

With ChatGPT, we've demonstrated that it can be a useful tool for analysis when prompted correctly. Building prompts able to analyze ideas and code and suggest risks and improvements can rapidly offer us alternative perspectives to consider, which we can then either act upon or reject. Utilizing the LLM as a simulation of a role that advocates quality can help us improve our work.

4.5 Improving documentation and communication with LLMs

It might not seem so, but communicating the work we've done through code comments and release notes can contribute significantly to product quality. By sharing new developments and changes to code bases, we can help fellow developers understand how our work affects theirs, guide testers in what to focus on when testing our work, and even help how users view our products (for example, Slack's early release notes helped market their tooling with their clear communication and humor).

Despite these benefits, documentation and release notes are sometimes left to languish at the end of a development cycle or are ignored entirely. This makes sense, given the time required to write and maintain code comments and release notes that are useful and relevant, especially when there is time pressure to be constantly delivering new features. However, with the use of LLMs, we can reduce that time overhead while ensuring we generate useful documentation that creates value for future readers. So, let's take a look at some useful prompts that can rapidly generate documentation for us.

4.5.1 Generating code comments

Although we should always strive to create code arranged in a way that is fluent and easy to parse, regardless of experience with a code base, code comments can provide

the extra detail that prevents code misuse and speeds up development. This is even more important if we're releasing APIs that will be used by others. (I've on many occasions wasted time trying to understand how a library works from poorly documented API docs.) The challenge is to get the right balance in code comments: too few, and the person reading your code is left to fend for themselves; too much, and we create more work for ourselves in maintaining code and comments to ensure they align.

So, instead, let's take a look at a prompt that can take care of the process of annotating our work with comments for us. I use the check-the-solution tactics to ensure the code is not altered:

MW You are a Java Developer. Update the code delimited by three hashes and add in code comments to explain how the code works. Check that no parts of the code have been changed before outputting the completed code commented code.

Next, I use delimiter tactics to state where the code is:

MW
```
###
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class Timesheet {

    private ConcurrentHashMap<String, Long> timesheets = new
    ➥ConcurrentHashMap<>();

    public boolean submitTimesheet(String projectName, long duration) {
        if (projectName != null && duration > 0) {
                    projectName = projectName.toLowerCase();

            timesheets.put(projectName,
            ➥timesheets.getOrDefault(projectName, 0L) + duration);
            return true;
        } else {
            return false;
        }
    }

    public Collection<Long> getTimesheets() {
        return timesheets.values();
    }

    public long getTotalTimesheetHours(String projectName) {
        return timesheets.getOrDefault(projectName.toLowerCase(), 0L);
    }

}
###
```

In this prompt, we use the usual delimiter tactics to provide our code that we would like to see commented, but we also want to ensure that the LLM doesn't change our code to make it align with the comments it creates. So, we also make it explicit within

the prompt to check that no parts of the code have been changed before outputting the completed commented code. Sending this prompt with our example code from earlier to ChatGPT returned the following commented code:



```
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class Timesheet {

    // ConcurrentHashMap to store project timesheets where the key is
    // the project name (in lowercase) and the value is the total duration.
    private ConcurrentHashMap<String, Long> timesheets = new
    ConcurrentHashMap<>();

    // Method to submit a timesheet entry for a project.
    public boolean submitTimesheet(String projectName, long duration) {
        if (projectName != null && duration > 0) {
            // Normalize the project name to lowercase to ensure case
            // insensitive matching.
            projectName = projectName.toLowerCase();

            // Update the timesheet entry for the project by adding
            // the submitted duration.
            timesheets.put(projectName,
            timesheets.getOrDefault(projectName, 0L) + duration);
            return true; // Submission successful
        } else {
            return false; // Invalid input, submission failed
        }
    }

    // Method to retrieve all timesheet durations.
    public Collection<Long> getTimesheets() {
        return timesheets.values();
    }

    // Method to get the total hours for a specific project.
    public long getTotalTimesheetHours(String projectName) {
        // Retrieve the total duration for the specified project
        // (case-insensitive) or return 0 if not found.
        return timesheets.getOrDefault(projectName.toLowerCase(), 0L);
    }
}
```

As we can see, the methods and sections of code within them have now been annotated with comments to give us more detail. What stands out most are comments such as



```
// ConcurrentHashMap to store project timesheets where the key is the
// project name (in lowercase) and the value is the total duration.
private ConcurrentHashMap<String, Long> timesheets = new
ConcurrentHashMap<>();
```

The comment provides not only references to what `ConcurrentHashMap` is used for, but also highlights that all project names will be stored in lowercase, a reference to part of

the code inside `submitTimesheet`, which does stand in contrast to comments like these that feel a little unnecessary:

```
// Method to submit a timesheet entry for a project.
public boolean submitTimesheet(String projectName, long duration)
```

Once again, we choose what to use. Although the prompt we've used has provided us with comments, we've observed that some are probably more useful than others. Thus, our role changes from being the author of comments to an editor, selecting and tweaking the comments that seem the most useful.

Maintaining code comments with prompts

What makes this process so useful is that when we inevitably come to a point in which our code changes and our comments need updating, we simply can run the prompt again with our updated code to regenerate our comments to incorporate our changes.

A final observation is that although the comments can be useful for someone reading our code directly, what if we want to provide more structured documentation in the form of, say, Javadoc? In that case, we can modify the prompt and use structured data tactics to create a prompt. We'll use an updated prompt with structured data tactics for Javadoc:

MW You are a Java Developer. Update the code delimited by three hashes and add in code comments in a Javadoc format to explain how the code works. Check that no parts of the code have been changed before outputting the completed code commented code.

We then provide the code using delimiter tactics:

MW
```
###
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class Timesheet {

    private ConcurrentHashMap<String, Long> timesheets =
    ➥new ConcurrentHashMap<>();

    public boolean submitTimesheet(String projectName, long duration) {
        if (projectName != null && duration > 0) {
                    projectName = projectName.toLowerCase();

            timesheets.put(projectName,
            ➥timesheets.getOrDefault(projectName, 0L) + duration);
            return true;
        } else {
            return false;
        }
    }
```

```
    public Collection<Long> getTimesheets() {
        return timesheets.values();
    }

    public long getTotalTimesheetHours(String projectName) {
        return timesheets.getOrDefault(projectName.toLowerCase(), 0L);
    }

}
###
```

Sending this prompt with our example code to ChatGPT produces



```
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Represents a timesheet management system for tracking hours spent on
 different projects.
 */
public class Timesheet {

    // A concurrent hash map to store project names and their
    // corresponding total hours
    private ConcurrentHashMap<String, Long> timesheets = new
 ConcurrentHashMap<>();

    /**
     * Submits a timesheet entry for a project.
     *
     * @param projectName The name of the project.
     * @param duration    The duration in hours to be submitted.
     * @return true if the submission was successful, false otherwise.
     */
    public boolean submitTimesheet(String projectName, long duration) {
        if (projectName != null && duration > 0) {
            // Normalize the project name to lowercase for consistency
            projectName = projectName.toLowerCase();

            // Update the total hours for the project or add a new
            // entry if it doesn't exist
            timesheets.put(projectName,
timesheets.getOrDefault(projectName, 0L) + duration);
            return true;
        } else {
            return false;
        }
    }

    /**
     * Retrieves a collection of all total timesheet hours for all
     * projects.
     *
     * @return A collection of total hours for all projects.
     */
```

```
    public Collection<Long> getTimesheets() {
        return timesheets.values();
    }

    /**
     * Retrieves the total timesheet hours for a specific project.
     *
     * @param projectName The name of the project.
     * @return The total hours spent on the specified project, or 0
     * if the project is not found.
     */
    public long getTotalTimesheetHours(String projectName) {
        // Normalize the project name to lowercase for consistency
        // and retrieve the total hours
        return timesheets.getOrDefault(projectName.toLowerCase(), 0L);
    }
}
```

Using this prompt, we've improved the quality of some of the comments and also created Javadoc-friendly comments that can be used to document our code for external users. Now we only have to tweak and edit at the points we see fit.

4.5.2 Generating release notes

Just as we can use a LLM to modify our code to add comments, we can also use LLMs to transform our code base into release notes that can be consumed by others in the team and beyond. To do this, we create a new prompt with the following details; we check the solution tactics to ensure the release notes are legible:

MW You are a Java Developer. Convert the code delimited by three hashes into a bullet-pointed set of release notes that describes how the code works. Check that the release notes can be read by someone who has no experience with the code created before outputting it.

We add an additional instruction to reduce the wordiness of the release notes:

* Only output the release notes.

Then we use delimiter tactics to state where the code is:

MW * Only output the release notes.

Then we use delimiter tactics to state where the code is:

MW
```
###
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class Timesheet {

    private ConcurrentHashMap<String, Long> timesheets =
    ➥new ConcurrentHashMap<>();

    public boolean submitTimesheet(String projectName, long duration) {
```

```
            if (projectName != null && duration > 0) {
                        projectName = projectName.toLowerCase();

                timesheets.put(projectName,
                timesheets.getOrDefault(projectName, 0L) + duration);
                return true;
            } else {
                return false;
            }
        }

        public Collection<Long> getTimesheets() {
            return timesheets.values();
        }

        public long getTotalTimesheetHours(String projectName) {
            return timesheets.getOrDefault(projectName.toLowerCase(), 0L);
        }

    }
    ###
```

The prompt follows a similar pattern to the previous code comment-focused prompt, but this time, instead of asking it to inject comments into our code, we instruct that the code be transformed into natural language release notes. For example, sending the prompt with the sample code to ChatGPT returned



Release Notes:

* The code defines a Java class named `Timesheet` that is used to manage timesheet data.
* It uses a `ConcurrentHashMap` to store timesheet entries, where project names (in lowercase) are associated with their respective durations (in hours).
* The `submitTimesheet` method allows users to submit timesheet entries. It requires a project name and a positive duration as parameters. If these conditions are met, it stores the duration associated with the project name in the map. If not, it returns false.
* The `getTimesheets` method returns a collection of all recorded timesheet durations.
* The `getTotalTimesheetHours` method allows users to retrieve the total duration for a specific project name (case-insensitive). If the project name is not found, it returns 0.

Note: Ensure that project names are provided in lowercase when using the `submitTimesheet` method to avoid case sensitivity issues when retrieving total timesheet hours.

The result we get is in a completely different format compared to our original code that attempts to detail what our code base does. Reviewing the output, it does feel quite technical, which makes sense considering we've asked for detailed notes on a small

amount of code. However, even these release notes can be of use because we can adapt an earlier prompt for suggesting risks to analyze the notes. We use delimiters to signal where the release notes will be:

MW You are a software testing expert. Analyze the release notes delimited by triple hashes and identify risks that might impact the code. If no code is provided, respond with "No risks could be found."

We then provide instructions around quality characteristics and output structure:

* Risks must be related to the quality characteristics: Performance, security and interoperability
* Multiple risks can be identified for each quality characteristic
* Output identified risks in the format of Risk - Quality Characteristic - Explanation

Few-shot prompting allows us to give an example of output structure

MW Example:

Overflow - Security - Adding in large integers as parameters might cause the method to trigger a buffer overflow.

Release notes are provided within the instructed delimiters:

MW ###

Release Notes:

* The code defines a Java class named `Timesheet` that is used to manage timesheet data.
* It uses a `ConcurrentHashMap` to store timesheet entries, where project names (in lowercase) are associated with their respective durations (in hours).
* The `submitTimesheet` method allows users to submit timesheet entries. It requires a project name and a positive duration as parameters. If these conditions are met, it stores the duration associated with the project name in the map. If not, it returns false.
* The `getTimesheets` method returns a collection of all recorded timesheet durations.
* The `getTotalTimesheetHours` method allows users to retrieve the total duration for a specific project name (case-insensitive). If the project name is not found, it returns 0.

Note: Ensure that project names are provided in lowercase when using the `submitTimesheet` method to avoid case sensitivity issues when retrieving total timesheet hours.

###

Using this prompt in ChatGPT returned the following suggested risks:



Concurrent Modification - Performance - The use of `ConcurrentHashMap` may not be suitable for high concurrent access, leading to performance bottlenecks if there are frequent modifications to the timesheet entries by multiple users simultaneously.

Lack of Input Validation - Security - The `submitTimesheet` method lacks proper input validation. It only checks for positive durations but does not validate the project name, making it vulnerable to injection attacks or other malicious input.

Case Sensitivity - Interoperability - The requirement to provide project names in lowercase may lead to interoperability issues when integrating with systems that expect case-insensitive input or have different naming conventions.

No Authentication/Authorization - Security - The release notes do not mention any authentication or authorization mechanisms, which can pose security risks if unauthorized users can submit or access timesheet data.

Data Exposure - Security - The `getTimesheets` method provides access to all recorded timesheet durations, which might expose sensitive information if not properly restricted or authenticated.

Of course, this might be an unnecessary step if we can directly ask an LLM to suggest risks based on the code directly, but it does demonstrate the power of LLM's ability to transform data from one format to another.

4.6 Maintaining a balance with code assistants

The perspective on the value that tools such as Copilot and ChatGPT offer depends on individuals. For some, it means the automation of development and the loss of many roles, and for others, it's nothing but an advanced tool that randomly picks suggested code. What this chapter demonstrates is the ongoing theme that their worth and usefulness lie somewhere between those two extremes.

These models, trained on masses of data from Stack Overflow and GitHub repositories, are very sophisticated in what they suggest for both production and test code. But they still require direction from us humans—direction that is guided by our abilities to communicate with stakeholders, analyze requirements, and design implementation. How well we can use AI tools in development depends on honing our complementary skills, which can be summarized by using the area-of-effect model shown in figure 4.2.

A balanced approach can help us deliver features faster but still ensure that we build in quality. So, our goal is to maintain that balance in situations in which we may need to rely on our own abilities or the features of our tools. Sometimes, code assistants won't be able to suggest the correct implementation, and we need to take charge. This gives us more control but does sacrifice speed. At other times, we can rely on code assistant tools to reference vast amounts of data to suggest new design ideas through unit checks or conversations. However, we want to ensure that we keep our TDD loops focused on

design and not on test coverage. Too many unit checks, and we lose sight of our design and end up in a box-checking activity.

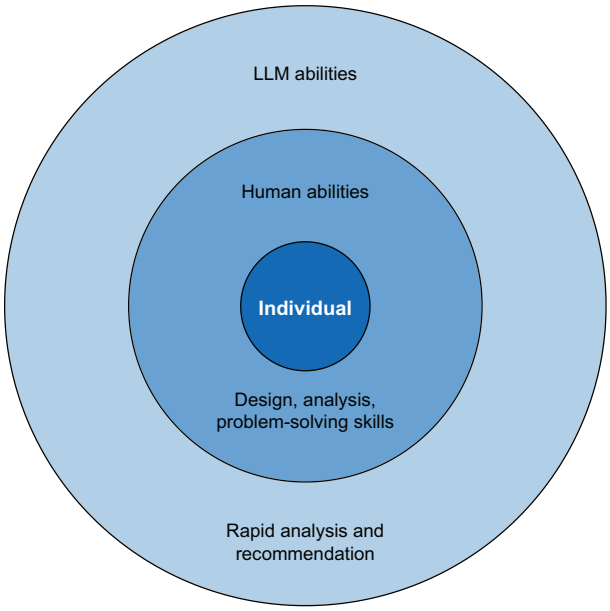


Figure 4.2 An area-of-effect model, updated to demonstrate the skills of a human and the abilities of code assistant tools

Summary

- The majority of generative AI tools currently on the market rely on LLMs trained with vast amounts of data scraped from the internet.
- LLMs are sophisticated algorithms that apply statistical analysis to our requests to determine what output they should respond with.
- Copilot is a coding assistant tool that uses the OpenAI GPT-4 and is trained on code stored on GitHub.
- Copilot works within an IDE and reads your code as a prompt to suggest what to add next into test code and production code.
- Tools such as Copilot can work well with the TDD red/green/refactor loop to help us rapidly create unit checks and production code.
- To help Copilot return code that is valuable, we need to guide it with prompts.
- Success with AI code assistants depends on our understanding of our abilities and the features of code assistant tools.
- There is push–pull relationship between how much we are leading versus the tooling leading design.
- We must be aware of the tradeoffs when the balance shifts from humans to tools leading.

A missing requirement is easier to address before it becomes working code. In this chapter, you'll use Gherkin scenarios to spell out how a feature should behave and check Claude's understanding before implementation begins. Then you'll build in stages, with opportunities to review and correct Claude's work as you go.

Chapter 5 from *Vibe Engineering* by
Tomasz Lelek and Artur Skowroński

From autocomplete to agent fleets: the road to Continuous AI

This chapter covers

- Progressing from autocomplete to agents
- Managing approval fatigue effectively
- Running agents outside the IDE
- Building team standards with AGENTS.md
- Exploring agent orchestration tradeoffs

An AI agent is only as autonomous as the leash you're willing to give it.

We already explored context engineering — how to feed models the right information at the right time. But context is just fuel. The question remains, what kind of engine are we building?

To answer that question, we'll move from passive autocomplete, through agents that can see and touch your codebase, to headless automation in CI pipelines, and finally toward orchestrated fleets of specialized agents. At each step, AI moves further away from a coding aid for one developer and deeper into the SDLC (Software Development Lifecycle) itself.

Read that as an expanding menu, not a ladder you climb and kick away. None of these forms retired the one before it. Tab completion is still the most reliable option when you know exactly what you want to type, approval prompts did not vanish when agents moved to the terminal, and a five-agent swarm beats one agent only

on a narrow class of parallelizable work. Which shape fits depends on the task, not on how recently the shape was invented.

Just as CI/CD transformed software development by automating integration and deployment, AI assistance is undergoing the same transformation. A growing number of teams and vendors use the term Continuous AI for event-triggered, automated AI support across the lifecycle, such as code review that examines every pull request before a human arrives, security scanning that catches leaked secrets and hallucinated packages at commit time, documentation that stays in sync with the code, or fault analysis that explains failed CI runs. It's AI as a persistent participant in the SDLC, not a tool you invoke when stuck.

What you pick freely is the shape. What you cannot pick is the operational maturity each shape assumes, because that part is cumulative. Running agents in CI assumes you know what a good diff looks like, which you learned supervising them in an editor. Orchestrating five agents assumes the observability you built the night one agent failed silently at 3 AM. Teams that jump straight to autonomous fleets are skipping the wreckage that teaches them what to instrument, and they buy that education later at a worse price. In practice, that is measured in months of organizational learning, not days of tool installation.

One thing before we start, because it changes how you should read the rest. Each stage produces more code per unit of human attention, which means each one also produces more unreviewed code per unit of human attention. Nothing in the progression makes the output more trustworthy on its own. What changes is where the guardrails have to live. In the autocomplete era, they live in your head and your head is enough. By the end of this chapter they live in hooks, policies, sandboxes, and CI gates, because there is no human left in position to catch anything.

5.1 Passive AI: ChatGPT as a coding partner

Let's start where most developers actually started their adventure with AI - not with sophisticated IDE integrations, but with a browser tab.

It's late 2022. Alice is staring at a ticket that's been sitting in her backlog for weeks: migrate the payment processing module from the legacy XML-based API to the new REST endpoint. The old code is a mess of nested callbacks, undocumented edge cases, and comments in three different languages left by developers who have long since moved on.

She's heard colleagues talking about this new thing called ChatGPT. On a whim, she opens a browser tab and pastes a chunk of the legacy code:

TL Here's our payment processing code. I need to migrate it to use REST instead of XML. Can you help?

[150 lines of tangled JavaScript]

What comes back surprises her. The model doesn't just translate syntax - it recognizes patterns, suggests a cleaner structure, even points out a potential race condition she hadn't noticed. Within an hour, she has a working first draft that would have taken her a full day to write from scratch.

Alice has just discovered something that millions of developers were discovering at the same time: LLMs can code. Not perfectly, not autonomously, but usefully. The browser-based workflow is clunky - copy code out, paste response back, manually merge changes - but the productivity boost is undeniable.

This is Era Zero of AI-assisted development. No IDE integration, no context awareness, no memory between sessions. Just a human playing the role of a very inefficient clipboard, shuttling text between their editor and a chat window. But even as Alice and millions like her discovered this new superpower, the friction was obvious.

5.1.1 GitHub Copilot: AI moves into the IDE

Every copy-paste cycle was a context switch, a flow break, a cognitive tax. The question wasn't whether AI could help with code, it was how to reduce the distance between the AI and the work. The model never sees your full file, let alone your project structure. You waste tokens re-explaining the same codebase in every conversation.

GitHub Copilot solved the most painful part of this friction. Instead of copying code to a browser, the AI comes to you. It watches what you type and offers completions inline, in real-time, right where you're working.

The experience feels almost magical at first. You start typing a function signature:

```
def validate_payment_amount(amount: Decimal, currency: str) -> bool:
```

And Copilot completes the entire implementation - null checks, currency validation, range boundaries - before you've written a single line of logic. Tab to accept. Move on.

For boilerplate-heavy tasks, the acceleration is dramatic. Writing database migrations, API endpoint handlers, test scaffolding, configuration parsers - the repetitive patterns that eat hours of developer time suddenly flow at the speed of reading rather than writing.

But Copilot's architecture reveals its limitations. It operates on a simple principle: given the code before your cursor (and sometimes the code after), predict what comes next. It's autocomplete on steroids - impressively capable steroids, but autocomplete nonetheless. It doesn't understand that `PaymentService` in line 47 refers to a class defined in `services/payment.py`, or that the team's coding standards require dependency injection rather than direct instantiation. The suggestion looks syntactically correct but architecturally wrong, adding toil to future maintenance.

Each suggestion exists in isolation. The context window is your current file - perhaps with a few neighboring tabs if the tool is feeling generous - but the broader architecture of your system remains invisible.

5.1.2 Tab-Tab-Tab: the illusion of productivity

Here's where the first subtle trap emerges.

Alice, our fintech developer, has been using Copilot for three months now. Her velocity metrics look fantastic - more commits, more lines of code, faster ticket closure. In sprint reviews, she demos features that used to take a week in half the time.

But something is shifting in how she works. She reads code more than she writes it now. Copilot suggests, she evaluates, she accepts or rejects. The proportions look almost unchanged on paper, and that is the interesting part. She used to spend most of

her time reasoning and a thin slice of it typing. She still spends most of her time reasoning and a thin slice of it steering. What changed is what the reasoning is about. It used to be "what should this code do?" It is now "is this code, which already exists, acceptable?" She moved from generating solutions to grading them, and grading is a different skill that decays in a different way.

This isn't inherently bad; code review is a legitimate skill, and evaluating suggestions is faster than writing from scratch. The trap is more subtle: *acceptance without understanding*.

It happens gradually. A Copilot suggestion looks reasonable. The tests pass. Ship it. Three weeks later, during an incident, Alice is debugging code she nominally wrote but doesn't fully understand. The AI had generated a retry mechanism with exponential backoff - sensible in isolation - but it interacts poorly with the circuit breaker in the upstream service, causing cascading timeouts under load. The incident lasted four hours. The root cause wasn't obvious; the retry code looked sensible in isolation, and Alice had to trace through three service boundaries before she found the interaction. Worse, she realized she didn't fully understand the code she was debugging. She'd accepted it, reviewed it, shipped it, but she hadn't *authored* it.

This is what I call *Copilot-driven development*: a mode where the developer becomes a reviewer of AI-generated code rather than an author of it. The human stops asking "what should this code do?" and starts asking "does this suggestion look okay?" The creative control shifts from human to machine, with the human reduced to a yes/no filter. Pretty miserable existence.

The most insidious part is that it feels productive. Lines of code flow. Features ship. The metrics all point up. But the developer's mental model of the system atrophies. Understanding is replaced by pattern-matching. "This looks like what I've accepted before" substitutes for "this is correct for our specific context."

5.1.3 The context blindness wall

Let's make the limitation concrete.

Alice's team has a strict convention: all API responses must be wrapped in a `Result` type that distinguishes between success, expected errors (like validation failures), and unexpected errors (like database timeouts). This pattern is documented in their architecture decision records. It's enforced in code review. Every senior developer knows it instinctively.

Copilot knows nothing about it.

When Alice starts writing a new endpoint, Copilot helpfully suggests:

Listing 5.1 Autocomplete response from Copilot (Python/Flask)

```
@app.route('/payments/<payment_id>')
def get_payment(payment_id: str):
    payment = PaymentRepository.find_by_id(payment_id)
    if not payment:
        return {"error": "Payment not found"}, 404
    return payment.to_dict(), 200
```

This is perfectly reasonable code. Well, mostly. It follows common REST conventions (though a type purist might note the missing return type annotation). For a generic codebase, this would be fine. And that's precisely the mistake, as it's completely wrong for Alice's codebase, where the correct pattern is:

Listing 5.2 How the correct response should looks like

```
@app.route('/payments/<payment_id>')
def get_payment(payment_id: str) -> Result[PaymentResponse]:
    payment = PaymentRepository.find_by_id(payment_id)
    if not payment:
        return Result.expected_error(ErrorCode.NOT_FOUND, "Payment not
     found")
    return Result.success(PaymentResponse.from_domain(payment))
```

The difference isn't just style, but also its semantics. The `Result` type carries information that downstream systems depend on. The generic pattern silently breaks contracts that aren't visible in the current file.

This is *context blindness*: the inability of autocomplete-based tools to see beyond the immediate text window. The model has no access to architectural decisions recorded in ADRs, project conventions that exist as tribal knowledge, type relationships defined in other modules, the runtime behavior of systems you integrate with, or the historical context of why certain patterns were chosen. The suggestions are syntactically valid and semantically plausible, but architecturally… foreign. The code compiles. The tests pass. But the system slowly drifts from coherent design toward a patchwork of locally reasonable but globally inconsistent patterns.

This era of passive AI - tools that work without deep project context - offers a genuine bargain. You trade deep, effortful authorship for fast, shallow review. For many tasks (boilerplate, well-understood patterns, exploratory prototyping) this is a good trade. For example, in prototyping, learning new frameworks, or investigating unfamiliar APIs, Copilot's context blindness becomes an asset. You want generic patterns, not project-specific conventions. The tool shines when you need to move fast through unfamiliar territory.

But the trade has hidden costs. Developers who accept without understanding gradually lose the ability to write from scratch; the mental muscles that once generated solutions now only evaluate them. Each accepted suggestion brings its own micro-conventions, and without active resistance, codebases drift toward inconsistency. Most insidiously, the code that "looks right but isn't" accumulates as technical debt that doesn't show up until much later: the wrong abstraction, the missed edge case, the subtly incorrect error handling - all accepted with a confident Tab.

The bargain works best when treated as what it is: autocomplete. A tool that handles the mechanical parts of typing while you retain full ownership of design, architecture, and understanding. It breaks down when the tool becomes the author and you become the rubber stamp… and who wants to be the rubber stamp?

The problem wasn't that Copilot was bad at autocomplete. The problem was that autocomplete was the wrong metaphor. What developers needed wasn't a faster typewriter, but a collaborator who could actually see the project they were working on - and eventually participate in more than the single moment of typing code.

If an AI could be trained on billions of lines of public code, why couldn't it read the few hundred thousand lines in *your* repository? The question wasn't theoretical anymore. If AI could generate plausible function implementations, why couldn't it follow the trail of imports to understand your actual architecture? If it could suggest code, why couldn't it *write* code - not as a suggestion floating in gray text, but as an actual edit to an actual file?

By late 2023, that question had moved from "what if?" to "what now?" A new generation of tools was already answering it.

5.2 Agents in IDE, but still on leash

The tools that emerged throughout 2023 and into 2024 shared a simple but radical premise: stop treating the AI like autocomplete and start treating it like an assistant with actual capabilities. Give it permission to read your files. Let it explore your project structure. Allow it to propose not just code snippets, but actual edits to multiple files at once.

Cursor, Windsurf, Zed AI, Continue, Cody each took a different path to the same destination. The AI was no longer confined to predicting the next token in your current file. It could see your imports, trace your dependencies, read your tests, examine your configuration. The context window expanded from a single file to an entire project.

The idea seems obvious in retrospect. Instead of the model guessing what `PaymentService` might look like based on statistical patterns, let it read services/payment.py. Instead of inferring your project structure from local clues, let it explore the directory tree. This shift from "autocomplete that guesses" to "agent that explores" represents the most significant evolution in AI coding tools since Copilot's launch. But it came with a catch that nobody fully anticipated: the more capable the agent, the more supervision it demands.

5.2.1 The Cambrian explosion of AI coding agents

By 2024, a wave of AI-first tools - Cursor, Cline, Roo Code, Windsurf, Aider, and others - had arrived, each sharing the same premise: give the AI actual access to your project. Each tool built its own machinery for reading files, searching the project, and applying edits - that part never needed a standard. What did need one was everything outside the repository, and that is where the Model Context Protocol slotted in, giving these agents a universal interface to documentation, databases, tickets, and APIs. For Alice, the combination is what finally made the context blindness wall navigable: her agent no longer guesses at the Result type convention. It reads the actual definition, searches for existing usages, and generates code that fits.

What changed wasn't just the availability of tools. It was an interaction contract. These agents didn't live inside your editor as suggestions, they lived alongside it as actors. And once an actor can touch your files and run your commands, the system needs a leash.

Alice installs Cline into her VS Code. The first thing she notices is the panel that appears alongside her editor; not a chat window exactly, but something more like a task control center. She types a request:

TL Migrate the payment validation in PaymentService to use the new currency validation library we added last week.

What happens next is visibly different from Copilot. Instead of immediately suggesting code, Cline announces a plan:

I'll help you migrate the payment validation. Let me first understand the current implementation and the new library.

1. Reading PaymentService.py to understand current validation logic
2. Searching for the new currency validation library
3. Checking how the library is used elsewhere in the codebase
4. Proposing migration changes

The plan reads like a checklist, the execution reads like negotiation. Then it starts executing - and here's where the "leash" becomes tangible. A popup appears:

I'll help you migrate the payment validation. Let me first understand what is happening in the codebase

Cline wants to read file: src/services/PaymentService.py

[Allow] [Deny]

She clicks Allow. Another popup:

Cline wants to search codebase for: "currency validation"

[Allow] [Deny]

Allow. Another:

Cline wants to read file: src/utils/currency.py

[Allow] [Deny]

Allow. And so on.

This is the *Plan-Act-Observe* loop that defines modern AI coding agents. The agent forms a plan, takes an action, observes the result, and adjusts. Each action is explicit and requires approval. The human remains in control - the agent cannot read, write, or execute anything without permission.

Cline and similar tools add another layer of organization: *modes*. Think of these as role-based access control for AI behavior. In "Architect" mode, the agent can read files and

propose high-level designs but cannot edit code. In "Code" mode, it can edit but within constrained boundaries. In "Ask" mode, it can only answer questions about what it observes.

Roo Code pushes this further with specialized personas - a planning agent, an implementation agent, a testing agent - each with different capabilities and constraints. The separation of concerns that we apply to human teams starts appearing in how we structure agent permissions.

The tools even allow different models for different modes. A reasoning-heavy model like Claude can handle planning while a faster, cheaper model handles routine code generation. The agent's "brain" becomes modular, with different components optimized for different tasks.

The old mental model was "AI writes code while I type.", the new one: "AI proposes a plan, I approve it, then AI executes it."

Plan Mode explicitly separates the "what should we do" from the "let's do it." You describe a task, the tool generates a structured plan with specific steps, you review and adjust the plan, and only then does execution begin. It normalizes the habit of thinking before coding - or rather, having the AI think visibly before coding.

This matters more than it might seem. When suggestions appeared inline and you hit Tab to accept, the decision was instantaneous and often subconscious. When a plan appears as a document you must read and approve, the decision becomes deliberate. The friction is intentional - it's a forcing function for understanding.

Alice, now six months into using these tools, has developed new instincts. She no longer asks "write me a function that does X." She asks "analyze the payment module and propose how we should add retry logic for failed transactions." The response is a plan she can critique, adjust, and ultimately own - even though the AI did the initial thinking.

5.2.2 Approval fatigue

But there's a problem with all those approval dialogs. Let's count what happens when Alice asks Cline to "refactor the payment validation module for better testability":

1. *Read* PaymentService.py - *Allow*
2. *Read* PaymentRepository.py - *Allow*
3. *List* test directory - *Allow*
4. *Read* test_payment_service.py - *Allow*
5. *Search* for "dependency injection" patterns - *Allow*
6. *Read* BaseService.py (found in search) - *Allow*
7. *Read* container.py (dependency injection config) - *Allow*
8. *Write* PaymentService.py (proposed changes) - *Allow*
9. *Write* test_payment_service.py (new tests) - *Allow*
10. *Run* pytest - *Allow*
11. *Read* pytest output - *Allow*
12. *Write* PaymentService.py (fix based on failing test) - *Allow*
13. *Run* pytest again - *Allow*

Thirteen approval dialogs for a single refactoring task. And this is a simple case.

By the tenth popup, something shifts in Alice's brain. She stops reading what she's approving. The dialog appears, her finger hits Allow reflexively, and she's back to waiting for the next one. The agent is nominally supervised, but the supervision has become a rubber stamp.

This is *approval fatigue:* the degradation of human oversight that occurs when the frequency of decisions overwhelms the cognitive capacity to make them meaningfully. The mechanism that was supposed to keep humans in control becomes a theater of control - the motions without the substance. Oversight doesn't scale linearly. The more capable the agent becomes, the more supervision it demands - until supervision itself becomes the bottleneck.

The risks are not theoretical. An agent that has learned to ask "can I read this file?" and always receive "yes" will eventually ask "can I delete this file?" at popup number forty-seven of a long session. The human whose attention has long since wandered clicks *Allow* out of habit. Practical mitigations exist, but none are perfect:

Auto-approve classes: Most tools now allow blanket approval for categories of low-risk operations. Read-only file access, directory listings, running tests, linting - these can be pre-approved, reducing the dialog count significantly. The risk is that "low-risk" is context-dependent. Reading `.env` files is low-risk in most cases, catastrophic in others.

Tool allowlists: Projects can define which MCP servers and operations are permitted. If your agent doesn't need database access for this task, don't give it database access. Principle of least privilege, applied to AI. The overhead is maintaining these lists as projects evolve.

Batch approvals: Instead of approving each action, approve a plan as a whole and let the agent execute multiple steps. You review the aggregate result rather than each individual operation. The tradeoff is that you lose the ability to intervene mid-execution if the agent takes a wrong turn early.

Checkpoints: Some tools introduce explicit pause points where the agent must stop and wait for review regardless of auto-approve settings. A sensible policy might be: read anything automatically, but always pause before writes and never auto-approve terminal commands.

None of these solutions eliminates the fundamental tension. Either you supervise closely and spend your time managing the agent instead of benefiting from it, or you loosen the leash and accept that your "oversight" is increasingly nominal. And approval fatigue isn't just about wasted time, it's about what slips through when attention fades. The most dangerous failures aren't the ones the agent asks permission for. They're the ones that look routine.

5.2.3 Trade-off: vision vs. vigilance

Agentic IDEs deliver on their promise. The agent can see your project - really see it, not guess at it from statistical patterns. It can read your conventions, explore your architecture, understand your dependencies. The context blindness that plagued autocomplete is materially reduced.

But the solution comes with its own cost: *you become a supervisor.*

Every capability the agent gains is a capability that requires oversight. File reading requires you to approve what it reads. File writing requires you to review what it writes. Command execution requires you to validate what it runs. The agent's power scales with your attention, and attention is finite.

The math often doesn't work in your favor. If an agent saves you thirty minutes of coding but costs you twenty minutes of supervision, the net gain is modest. If approval fatigue leads you to rubber-stamp a destructive action, the net gain is negative. If the cognitive overhead of managing the agent prevents you from entering flow state, the productivity loss is invisible but real.

The developers who thrive in this era are the ones who develop *selective trust*: clear mental models of when to supervise closely and when to let the agent run. They configure their approval policies thoughtfully, creating friction where risk is high and removing it where risk is low. They treat the agent like a capable but junior developer - trusted with routine tasks, supervised on anything novel or destructive.

But even with perfect policies and calibrated trust, one constraint remains: such an agent works only when you're watching. It stops when you look away. It cannot run overnight. It cannot process a backlog while you sleep. Its scale is bounded by your attention, which means its impact is still local to a developer session rather than the broader SDLC. And attention is the one input nobody made faster. Generation accelerated; the context the agent cannot see, the pipeline it has to run through, the verification that decides whether the change is correct and the human who signs it off did not. Whichever of those four you leave alone is the one that sets your throughput.

This limitation points toward the next evolution, and it is worth being precise here, because two different things routinely get conflated. One is the *interface*: editor panel, terminal, CI job. The other is the *autonomy level*, whether a human approves each action. They are independent axes. Claude Code in a terminal will stop and ask before it writes a file. A background agent inside an IDE will not. You can build either mode on either interface.

The axis that matters for scale is autonomy. An agent that needs approval stops when you look away, wherever it happens to run. The terminal shows up in the next section anyway, because it is where non-interactive execution has always lived: cron, hooks, pipelines, anything triggered by an event rather than by a person. Moving there is not what removes the human. It is what makes removing the human worth doing.

This is exactly where the tooling moved next. But as we'll discover, an agent that works without supervision is an agent that fails without witnesses. Approval fatigue doesn't just waste time, it creates a new class of invisible risk. When routine operations stop being reviewed carefully, anything that looks ordinary slips through. And in modern systems, few changes look more ordinary than a dependency update.

5.2.4 Slopsquatting and package hallucinations

Agents operate in environments where their outputs have immediate effects. A batch refactoring script can modify hundreds of files. The blast radius of a mistake is larger than in supervised contexts.

One category of mistake has proven particularly dangerous: *package hallucinations*.

LLMs are trained on vast codebases that reference millions of packages across different ecosystems. When generating code, they pattern-match against this training data. Sometimes the patterns match packages that don't exist - or more precisely, don't exist *yet*.

The attack is called *slopsquatting*: researchers and attackers monitor AI coding tools, collect the names of hallucinated packages, and register those names with malicious payloads. When a developer (or an automated pipeline) installs the AI-suggested package, they get malware.

The numbers are sobering. Studies like *Spracklen et al. We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs, Spracklen et al., 2024, arXiv:2406.10279* have found that popular AI coding tools hallucinate package names in roughly from 5% (commercial models) to even over 20% (open-source models). Researchers have successfully registered thousands of these hallucinated names across npm, PyPI, and other package registries. The attack surface is vast and largely undefended. Those numbers come from studies of 2024-era models; newer models hallucinate less, and registries have begun defending themselves. But the attack class is structural - it exists wherever generated text names a dependency - so treat the specific rates as a snapshot and the mechanism as permanent.

The clearest public demonstration came before slopsquatting even had a name. Bar Lanyado, a researcher at Lasso Security, noticed that models kept recommending a Python package called `huggingface-cli`. The real Hugging Face command line tool installs as `pip install -U "huggingface_hub[cli]"`; the shorter name the models kept producing did not exist. Lanyado registered it on PyPI as an empty package to see what would happen. By Lasso's own count it collected over 30,000 genuine downloads in three months, and Alibaba's GraphTranslator repository ended up recommending `pip install huggingface-cli` in its README. A hallucination travelled from a chat window into a major company's published install instructions, and from there into everyone who followed them.

Be clear about what that does and does not prove. Lanyado's package was empty on purpose, and a fully documented chain from hallucination to working payload is still surprisingly hard to point to in public. What the experiment establishes is that every step except the payload happens routinely. The name gets invented, the name gets copied into real repositories, the package gets installed tens of thousands of times, and nobody notices for months. The only missing ingredient is an attacker registering the name before a researcher does.

In an agentic context, the danger multiplies. An agent modifying `package.json` and running `npm install` can introduce a malicious dependency without any human ever reading the package name. The commit message looks innocuous ("added error

handling utility") but the utility doesn't exist, and what gets installed instead is an attacker's payload. The human operator simply tabbed that commit into the repository, the same way they tabbed the last forty.

This is the dark side of implicit trust. When a senior developer recommends a package, you trust their judgment; they've vetted it, used it before, know its reputation. When an AI recommends a package, it's pattern-matching against training data that includes packages that never worked, packages that were abandoned years ago, and packages that never existed at all. The suggestion *looks* identical to a trusted recommendation. The confidence is the same. Only the basis for that confidence is missing.

Defenses exist, and here's where the sword becomes a shield: the same LLM patterns that create hallucination risk can detect it. Specialized security tooling can check package provenance, validate registry existence, and flag suspicious or newly introduced dependencies before installation. But you don't always need a dedicated tool. A simple LLM-in-the-loop can serve as a pre-install validator: pipe your dependency changes through an agent prompted to verify each package exists in the registry, check its maintenance history, and flag names that suspiciously resemble popular packages. The CI pipeline that might blindly install a hallucinated package can also run an LLM judge asking: *"Does this package actually exist? Does it have credible maintainers? Is this name a trap?"*

The same automation that amplifies attacks can amplify defenses. The question is whether your shield deploys faster than the attacker's script. And once you start relying on automation for defense, the next step is obvious: let it run without you.

5.3 Headless era: Moving into terminal

The approval fatigue problem has an obvious escape hatch: remove the human from the loop. If constant supervision is the bottleneck, what happens when we let agents work unsupervised?

This is where Continuous AI stops being a concept and becomes practice. Remember the patterns from the chapter introduction: Continuous Documentation, Continuous Fault Analysis, Continuous Code Improvement. This is also the point where AI stops being just a coding assistant and starts becoming SDLC infrastructure.

And escape they should. You don't want an IDE as the *home* for automation - you never did. Shell scripts, cron jobs, Git hooks, CI pipelines: this is where automation has always lived. Unix philosophy taught us decades ago that small, composable tools piped together solve complex problems elegantly. The IDE is for interactive work, for exploration, for the human in the loop. The terminal is for processes that run whether you're watching or not.

Headless agents fit this world perfectly. They work best when the task has crisp inputs, bounded actions, and cheap validation of outputs. They excel at the repetitive, event-triggered work that humans neglect: regenerating docs after every commit, labeling and routing incoming issues, explaining failed CI runs, enforcing code standards across thousands of files. Work that's important but not urgent, that accumulates into technical debt when ignored, that finally gets done because something tireless is paying attention.

The promise is compelling: agents that work while you sleep, that process backlogs overnight, that catch problems before code even reaches review. But let's be clear about what we're trading. An unsupervised agent is an agent whose failures go unwitnessed until they become impossible to ignore.

5.3.1 Codex, Antigravity and Claude Code: AI in the UNIX pipeline

The UNIX philosophy - do one thing well, connect everything with text - has survived for fifty years because it solves a fundamental problem: how do you compose simple tools into complex workflows? The answer, small programs that do one thing well, connected by text streams, turns out to be remarkably adaptable. It absorbed version control, containerization, cloud infrastructure, and now AI. Figure 5.1 shows an example of a coding CLI interface in practice.



```
Antigravity CLI 1.1.4
askowronski@virtuslab.com (Antigravity Starter Quota)
Gemini 3.5 Flash (Medium)
~/wiki

> 

? for shortcuts                                  Gemini 3.5 Flash (Medium)
```

Figure 5.1 Example of Antigravity CLI interface

Anthropic's Claude Code, OpenAI's Codex CLI, and Google's Antigravity CLI (Figure 5.1) represent the clearest expression of this integration. They're not IDEs or chat interfaces, they're command-line tools designed to slot into existing workflows the same way `grep`, `sed`, or `jq` do.

One clarification before we go further: these tools lead double lives. Most developers first meet Claude Code or Codex CLI interactively - a conversational agent living in the terminal instead of an IDE panel - and for many teams that terminal session, not an agentic IDE, is the primary way they work with agents today. Everything from section 5.2 applies to that mode: plans, permissions, approval fatigue. This section is about their second life - headless mode, where the same agent runs non-interactively, reads stdin, writes stdout, and asks no questions. That is the mode that turns an assistant into infrastructure.

The syntax is deliberately minimal (example use `agy`, shell version of Antigravity):

```
cat error_logs.txt | agy -p "Summarize the root causes of these errors"
agy -p "Review this diff for security issues" < proposed_changes.patch
find . -name "*.py" -exec agy -p "Add type hints to this file" {} \;
```

Here's where Continuous AI stops being a concept and becomes practice.

Alice, now leading a small platform team, discovers these tools when she's drowning in operational work. Every morning starts with the same ritual: review overnight

alerts, triage error logs, identify patterns, decide what needs immediate attention. It's important work, but it's also repetitive work - exactly the kind of thing that should be automated. She starts small:

Listing 5.3 Morning log triage (cron + CLI agent)

```
bash
#!/bin/bash
# morning_triage.sh

kubectl logs -l app=payment-service --since=8h > /tmp/overnight_logs.txt

codex -p "Analyze these logs and categorize issues by severity.
Focus on: payment failures, timeout patterns, error spikes.
Output as markdown table." < /tmp/overnight_logs.txt > /tmp/triage_report.md

# Send to Slack
cat /tmp/triage_report.md | slack-notify --channel=#platform-oncall
```

The script runs at 6 AM via cron. By the time Alice opens her laptop, a structured triage report is waiting in Slack. The agent has read eight hours of logs, identified patterns, categorized issues, and presented them in a format optimized for human decision-making.

This is the UNIX philosophy in action: the agent is one tool among many. It doesn't replace the pipeline, but it enhances it; used only when truly useful, without the need to orchestrate the whole flow by itself. `kubectl` extracts logs, `codex` analyzes them, `slack-notify` delivers results. Each component does one thing. The composition creates value none could provide alone.

The constraint is the same as any UNIX tool: the agent operates on whatever input it receives and produces output to stdout. It has no memory between invocations unless you explicitly persist state somewhere, no awareness of previous runs, and no ability to ask clarifying questions in the middle of a non-interactive pipeline. You must structure the input to contain everything the agent needs, and trust that the output will be useful without further interaction.

Common patterns are already emerging. Teams pipe application logs through agents that identify anomalies and correlate errors across services - a first-pass filter that reduces the volume humans must review. Others run batch refactoring jobs: updating import statements, migrating API patterns, adding logging to functions matching certain criteria. Tasks that would take hours of tedious manual work become single commands. Some generate weekly reports automatically, such as metrics summaries, sprint retrospectives, and dependency audits, all produced without human involvement.

Our favourite example is documentation generation though, because there's an interesting philosophical shift happening here. For decades, documentation was the unloved stepchild of software development. Nobody wrote it because nobody read it. The economics never worked - the effort of maintaining accurate docs exceeded the value anyone extracted from them and in active projects, READMEs went stale within weeks. Architecture diagrams diverged from reality while API documentation described

endpoints that no longer existed. We tried to solve that, storing documentation with a code, but the original issue (who cares?). As a developer, we internalized one of the Agile Manifesto statements - *Working software over comprehensive documentation* and taking it in many cases to extremes.

But now something has changed: agents read documentation. When a headless agent needs to understand your codebase, it consumes your README, your ADRs, your API docs, your runbooks - whatever you give it as input. Suddenly documentation has a high-value consumer that actually uses it. Diego Martin Lafuente, who now hands commit messages, PR descriptions and naming conventions to agents, puts it in one line: "the audience that values all that structure the most is now LLMs." The reader who never skimmed your ADRs turned out to be a machine. Humans benefit as a side effect - the same Continuous Documentation that helps your agent helps your new team members onboard faster..

This insight spawned a new category of tools. Gemini-powered Google Code Wiki, launched in November 2025, takes the concept to its logical extreme. The platform maintains a continuously updated, structured wiki for every repository it monitors. Unlike traditional static documentation that developers struggle to maintain, Code Wiki automatically scans the full codebase and regenerates documentation after each change, such as architecture diagrams, class relationships, and sequence flows that reflect the current state of the code (Figure 5.2).

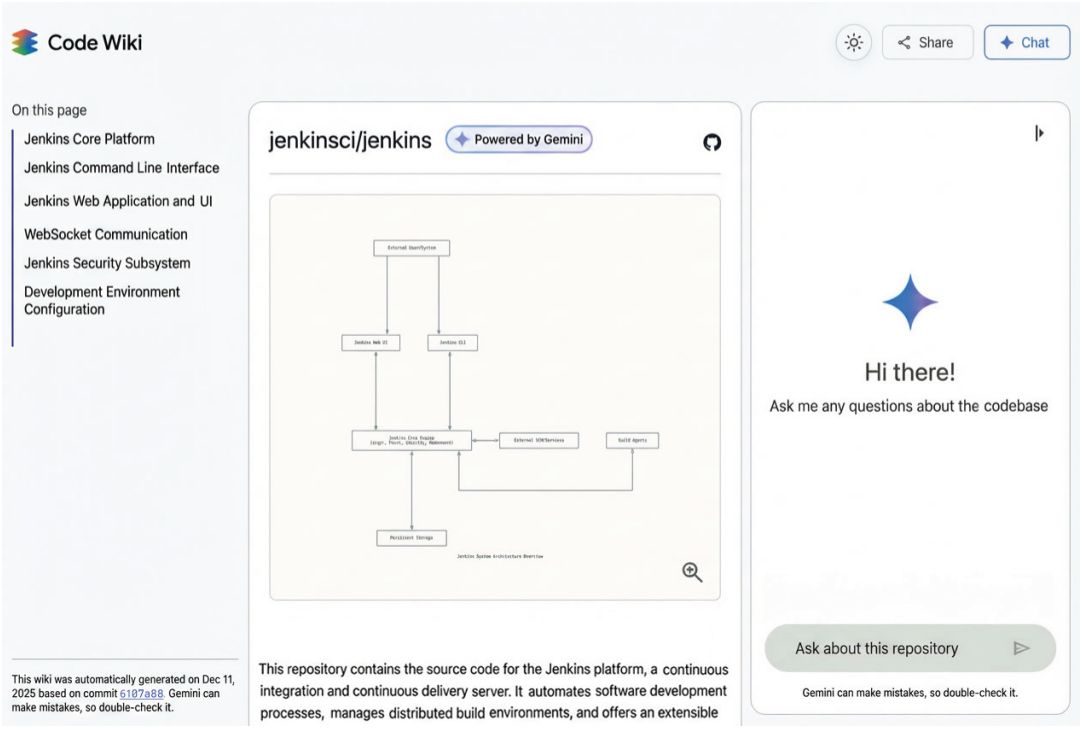


Figure 5.2 **CodeWiki auto generated documentation for Jenkins CI.**

The platform is available for public repositories, with private-repository support arriving through Google's enterprise tooling. This is particularly valuable for companies where legacy codebases are poorly documented and institutional knowledge has faded.

What makes Code Wiki genuinely useful isn't just the generated documentation, but the intelligent, context-aware integration. Every wiki section and chat answer is hyperlinked directly to the relevant code files and definitions, merging reading and exploring into one workflow. The Gemini-powered chat agent uses the always-current wiki as its knowledge base, so you're not talking to a generic model, you're talking to one that knows your repository end-to-end. Ask "How is authentication implemented?" or "What's the data flow for payment processing?" and get answers grounded in the actual code, not outdated docs that may have drifted from reality.

These examples - log analysis, batch refactoring, documentation generation - share a common thread: they process code that already exists. But what about code that's about to exist?

5.3.2 Agent as gatekeeper and reviewer in Git

If agents can analyze code in batch, they can analyze code at the moments that matter most: just before it enters the repository.

Git hooks provide natural integration points. Pre-commit hooks run before a commit is recorded. Pre-push hooks run before changes leave the local machine. Both can invoke arbitrary programs - including AI agents.

And to be clear: I love linters. They're fast, deterministic, predictable, and excellent at enforcing rules that can be precisely defined. For many problems, they are the best possible tool. But they also have natural limits - they don't understand intent, team context, or *why* something is a bad idea in a specific project.

That's why Alice's team implements a pre-commit hook that runs every diff through an agent:

Listing 5.4 Pre-commit hook example

```
#!/bin/bash
# .git/hooks/pre-commit
set -euo pipefail

DIFF=$(git diff --cached)
[ -z "$DIFF" ] && exit 0

# Deterministic checks block. This is the real gate.
gitleaks protect --staged --redact || exit 1

# The model advises. It cannot block on its own.
VERDICT=$(printf '%s' "$DIFF" | claude -p "Review this diff against our
conventions: error handling, test coverage for new public functions,
naming. Reply with exactly one line of JSON and nothing else:
{\"verdict\":\"clean\"} or {\"verdict\":\"concerns\",\"notes\":\"...\"}")

if [ "$(printf '%s' "$VERDICT" | jq -r '.verdict')" = "concerns" ]; then
    echo "AI reviewer raised concerns (advisory, not blocking):"
    printf '%s' "$VERDICT" | jq -r '.notes'
fi
exit 0
```

Three things in that script are deliberate. Secret scanning is done by gitleaks, a deterministic tool, and it is the only step allowed to fail the commit. The model returns a structured verdict field parsed with `jq` rather than free text matched against a substring, because a gate that greps for the word PASS will cheerfully wave through a response reading "FAIL: PASSWORD exposed in config.py". And the model's verdict prints a warning instead of blocking, because a probabilistic reviewer that blocks will eventually block something correct, and the developer will learn to type `--no-verify` by reflex, which costs you the deterministic check too.

That split is worth stating as a rule, because it governs every agent gate in the rest of this chapter. Deterministic checks block. Models advise. A scanner matching an AWS key against a regex is right every time and can safely stop a commit. A model reasoning about whether your error handling fits team convention is useful precisely because the question has no regex, and that same property is why it is sometimes wrong. Give the model the questions a linter cannot ask. Give blocking authority to the tools that cannot be argued out of an answer.

Every commit now passes through an AI bouncer. The agent checks for leaked secrets - still a stubborn operational problem, with Verizon's 2025 DBIR citing a median remediation time of 94 days for secrets exposed in GitHub repositories, obvious vulnerabilities, broken conventions, and the classic "tests coming later" optimism. Problems get caught before they even make it into version control, which is both the earliest and the cheapest moment to fix them.

And the pattern doesn't stop at security. Teams use hook-based agents to enforce style consistency, but not the superficial stuff linters already complain about. These agents go after the squishy, human rules: naming conventions, error-handling philosophy, and those unspoken "we absolutely don't do that here" norms that usually live only in senior engineers' memories. Static analysis is blind to this. Agents aren't.

The same hooks can flag code quietly copy-pasted from incompatibly licensed sources, exactly the sort of surprise you don't want your legal team uncovering months after launch. They catch breaking API changes by understanding what *actually* changed, not just what the diff happens to show, surfacing semantic landmines that syntactic tools cheerfully wave through. They can even block commits that modify public interfaces without updating the documentation, because "we'll document it later" is how broken contracts are born.

Run these agents locally and catch problems before the commit ever exists. Run them server-side and make the rules unskippable, with no `--no-verify` escape hatch in sight.

Of course, git hooks only see individual commits, and that's a real limitation. Plenty of problems simply don't show up until you're looking at a pull request: when multiple commits stack together, when changes collide with work happening in parallel, when tests run in something closer to a real environment, and when you can finally judge how a modification fits into the system as a whole. It takes a lot of time, and doing that on local machines after every commit would be frustrating and costly.

That's where CI-integrated agents actually make sense.

These agents run on pull-request events, with access to the full diff, the test results, the dependency graph, and whatever context the CI pipeline can realistically provide. They don't replace code review, and they don't magically "understand the system." What they do is take on the kind of analysis that's repetitive, time-consuming, and easy for humans to miss when reviewing under time pressure.

This is where the LLM as a judge pattern becomes useful in practice. Instead of generating code, the model evaluates it: comparing changes against project conventions, sanity-checking whether the amount of testing matches the risk of the change, calling out potential security concerns, and producing a structured summary of what actually changed and why it might matter. One limit belongs in the same paragraph as the promise: a judge model reading a generator model's diff is not an independent check. Both layers fail in the same places for the same reasons. Mechanical verification gets cheaper here - semantic verification does not. The judge widens the net, it is not a second opinion.

In practice, teams often split this work across multiple, narrowly scoped agents: one that looks only at security-relevant changes, another that focuses on performance regressions, another that watches API compatibility. None of them is particularly smart on its own - but together they widen the review surface without burning out the humans. This decomposition also maps onto team ownership: the security team maintains the security review agent, the platform team owns the performance regression detector, the API team governs the compatibility checker. Which is the whole point. An agent nobody owns is a comment generator nobody reads.

The tooling and platforms differ, sometimes significantly, but the underlying pattern stays the same:

Listing 5.5 Github Action AI-Reviewer

```
# .github/workflows/ai-review.yml
name: AI Code Review
on:
  pull_request:
    types: [opened, synchronize]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Get diff
        run: |
          git diff origin/main...HEAD > changes.patch

      - name: AI Review
        run: |
          claude -p "Review this pull request diff. Focus on:
          - Logic errors and edge cases
```

```
            - Performance implications
            - Security considerations
            - Test coverage gaps

            Format as GitHub review comments with file:line references." \
            < changes.patch > review.json

        - name: Post comments
          uses: actions/github-script@v7
          with:
            script: |
              // Parse review.json and post as PR comments
```

Or you can reach for dedicated tools that handle the integration for you. CodeRabbit and GitHub's own agentic code review install as a marketplace app or an organization setting: authorize repository access, and pull requests get reviewed as they are opened, with the context windowing, comment formatting, and feedback loops that take weeks to build properly from scratch. Setup takes minutes rather than the custom YAML of Listing 5.5. The tradeoff is control versus maintenance: rolling your own means deciding exactly what the agent examines and how it reports, while using a platform means someone else handles the edge cases you haven't discovered yet.

There is a third cost to the platform route that deserves more than a footnote. Figure 5.3 shows Gemini Code Assist reviewing a pull request (In May 2026 Google announced it was consolidating its developer tooling under the Antigravity brand).

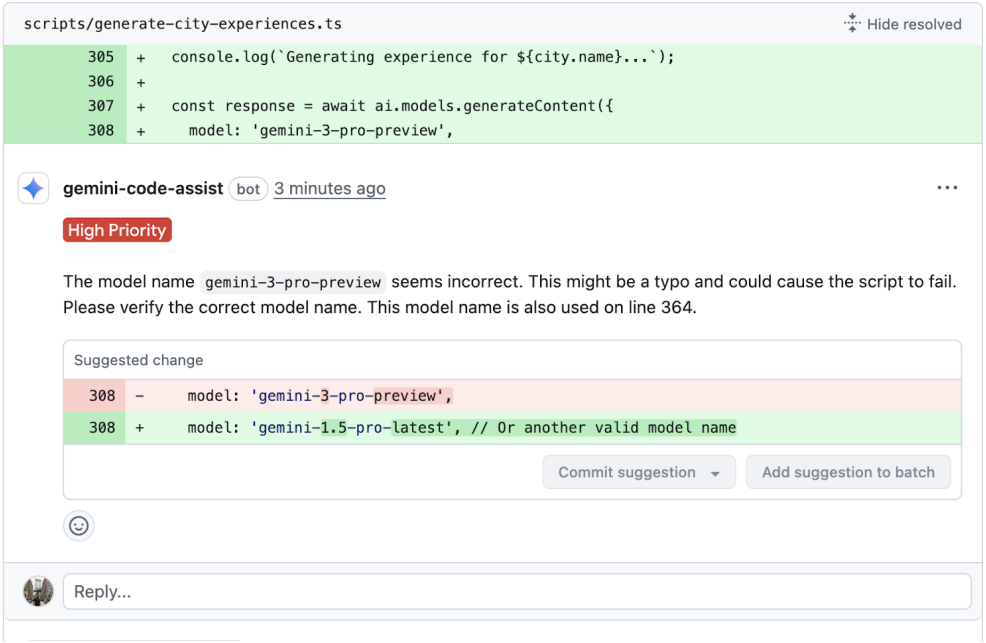


Figure 5.3 Gemini Code Assist reviewing a pull request, before Google folded the product into Antigravity

Either way, Alice's team deploys a version of this workflow. Now every PR receives an AI review within minutes of being opened. The agent examines the diff, cross-references with existing code patterns, and leaves comments - sometimes useful observations, sometimes noise, but always fast.

The value is triage, not replacement. Human reviewers still make the final call. But when they arrive at the PR, the obvious issues have already been flagged. They can focus their limited attention on architectural decisions, business logic correctness, and the subtle issues that AI still misses. The agent handles the checklist items - "did you add tests?", "did you update the changelog?", "is this consistent with our error handling?" - that consumes human attention without requiring human judgment.

The beauty of headless automation is that the sky is the limit. When human reviewers dismiss AI comments as irrelevant, that signal can feed back to improve future reviews. The agent learns what the team actually cares about.

Advanced setups go further still. Instead of just flagging issues, the agent proposes specific fixes as suggested changes that reviewers can accept with one click. The agent enforces explicit rules loaded from a configuration file in the repo and blocks merge if violations are found. The agent tracks patterns across PRs - "this is the third time this week someone has made this mistake" - surfacing systemic issues that individual reviews miss.

Between the IDE and the fully headless pipeline sits one more pattern - and in many teams it is now the most common one: delegation. You assign an issue to a cloud-hosted agent (GitHub's coding agent, Codex, or similar), and some minutes later a pull request appears, produced in an ephemeral environment, ready for review. No terminal, no supervision, no pipeline to build. It is Continuous AI at the level of a single task: fire, forget, review. Everything this section says about silent failure applies doubly here, because the only artifact you ever see is the finished PR

5.3.3 Debugging a lost agent

Alice's team has been running headless agents for three months. Mostly it works. But one Tuesday morning, she discovers that the overnight batch processing job has been silently failing for a week. The agent ran, produced outputs, exited successfully - but the outputs were garbage. No errors, no alerts, just wrong results that nobody noticed until a downstream report made no sense.

This is the core challenge of headless agents: failure modes that don't announce themselves. A supervised agent fails visibly. It proposes nonsense, you reject it. It gets stuck in a loop, you interrupt it. If it misunderstands the task, you correct it. The feedback loop is tight and immediate.

A headless agent fails quietly. It produces outputs that look plausible but aren't correct. It completes successfully while having accomplished nothing useful. It drifts gradually from the intended behavior as context accumulates errors. By the time you notice, the damage is done and the cause is buried in logs nobody was watching.

Isolate the variable: When an agent misbehaves, is the problem the prompt, the context, or the model itself? Test the same prompt with minimal context. Test different prompts with the same context. Swap models if possible. Systematic isolation identifies which component is failing.

Minimize reproduction: Complex pipelines obscure root causes. Extract the failing case into the smallest possible reproduction - ideally a single prompt with inline context that you can iterate on in a chat interface. Once you can see the failure interactively, debugging becomes tractable.

Check context pollution: Long-running agents accumulate context. Earlier outputs become inputs to later steps. Errors compound. A common failure mode is an agent that worked perfectly for the first hundred iterations but degraded as accumulated context introduced noise or contradictions. The fix is often simply to reset - start fresh with clean context.

Verify assumptions: Headless agents often fail because the world changed out from under them. The file they expect to read was moved. The API they call changed its response format. The package they depend on released a breaking update. The agent runs the same code that worked last week, but last week's assumptions are no longer valid.

When to kill versus when to guide: If an agent is slightly off course - producing outputs that are wrong but close - it's often worth providing corrective feedback and letting it continue. If an agent is fundamentally confused - looping, producing nonsense, or working on the wrong problem entirely - reset is usually faster than repair. The sunk cost fallacy applies: those tokens are spent either way.

The meta-lesson is that headless agents require *observability infrastructure* that doesn't exist by default. You need logging that captures not just "did it run" but "what did it produce." You need validation that checks outputs against expectations, not just execution success. You need alerting that triggers subtle quality degradation, not just hard failures.

Building this infrastructure takes time and expertise. Many teams skip it, seduced by the apparent simplicity of "just run the agent in CI." They discover the need for observability only after a silent failure causes real damage.

5.3.4 Trade-off: autonomy vs. observability

The Headless Era solves the approval fatigue problem by removing the human from the loop. The agent works unsupervised - in terminals, hooks, pipelines - accomplishing in minutes what would take hours of babysitting in an IDE context.

But supervision served a purpose beyond mere gatekeeping. A human watching an agent notices when it goes wrong. A human reviewing each action catches errors before they propagate. A human in the loop provides continuous course correction that keeps the agent aligned with intent.

Remove the human and you remove that feedback mechanism. Errors accumulate in silence. Drift goes undetected. Failures announce themselves only through downstream consequences - broken builds, wrong reports, security incidents - long after the moment when intervention would have been cheap.

The teams that succeed with headless agents are the ones that *replace human supervision with automated observability*:

- Validation checks that verify outputs match expectations
- Anomaly detection that flags unusual patterns in agent behavior
- Audit logs that enable post-hoc debugging when something goes wrong
- Circuit breakers that stop pipelines when quality degrades
- Regular human review of sampled outputs to catch systematic drift

None of it makes the agent smarter. All of it makes the agent's mistakes cheap and visible, and cheap and visible is what engineering discipline reduces to once the tool underneath is probabilistic.

But there's a harder problem lurking behind single-agent success: what happens when it's not just Alice using agents, but her entire team? Six developers, six differently-configured agents, one codebase. The failure modes stop being personal and start being systemic. This isn't a tools problem. It's a coordination problem. And coordination problems require standards.

5.4 Standardizing team workflows with agents

Alice's solo experiments have been successful enough that her manager notices. "Whatever you're doing with those AI tools," he says in their one-on-one, "I want the whole team doing it."

Famous last words.

Within two weeks, six developers are running six differently-configured agents against one codebase - and the symptoms appear fast.

5.4.1 From .cursorrules to AGENTS.md

Alice had spent months building personal intuitions about what works. She considered long which prompts to use, which suggestions to reject, and when to override the agent's instincts. Her teammates had default configurations and good intentions, which is roughly the same as having nothing at all. Alex used Cursor with aggressive auto-complete. Jordan preferred Cline with careful manual approval. Sam had elaborate custom prompts tuned over months. The others had never opened the settings.

Alex used Cursor with aggressive auto-complete. Jordan preferred Cline with careful manual approval. Sam had elaborate custom prompts tuned over months. The others had default configurations they'd never touched.

The codebase started showing symptoms. The same abstraction was implemented three different ways in three different modules - each correct according to someone's agent, none consistent with each other. Error handling varied wildly: some code used the team's `Result` type, some threw exceptions, some returned null. Naming conventions drifted as each agent pattern-matched against its user's recent work rather than the project's established style.

The problem wasn't that agents wrote bad code. Each agent wrote code that was locally reasonable. The problem was that "locally reasonable" multiplied by six developers produced global incoherence.

The solution emerged organically. Cursor had introduced `.cursorrules` - a configuration file that let users define project-specific instructions their agent would follow. Alice created one for the team repository:

Listing 5.6 .cursorrules

```
## Project Context
This is a Python payment processing service using FastAPI and SQLAlchemy.
All monetary values must use Decimal, never float.
## Code Style
- Use the Result type for all service method returns (see src/core/result.py)
- Errors are domain objects, not exceptions (see src/core/errors.py)
- Repository methods return Optional[Entity], services wrap in Result
- All public functions require type hints
- Use dependency injection via FastAPI's Depends()
## Testing
- Every public service method needs unit tests
- Use pytest fixtures from conftest.py, don't create new database connections
- Mock external services, don't hit real APIs in tests
## Things to Never Do
- Never use datetime.now() - inject TimeProvider instead
- Never hardcode configuration - use settings from src/core/config.py
- Never commit code that references environment variables directly
```

The effect was immediate. Six agents reading the same rules started producing consistent code. The `Result` type appeared everywhere. Error handling unified. Datetime injection became automatic. The agents hadn't gotten smarter - they'd been given shared context that aligned their outputs.

But `.cursorrules` was Cursor-specific. Jordan's Cline setup couldn't read it. Claude Code in the CI pipeline didn't know it existed, as it's vendor locked and looks only for CLAUDE.md. The team was back to fragmentation, just at a different level.

This fragmentation drove the emergence of `AGENTS.md`: a tool-agnostic convention for describing project context to AI agents. What began as an open proposal from OpenAI in 2025 has become a genuine standard, donated in December 2025 to the Linux Foundation's Agentic AI Foundation alongside MCP, read natively by Codex, Cursor, Copilot, Windsurf, and Antigravity, and present in tens of thousands of public repositories.

"Read natively" is carrying real weight in that sentence, and it is worth checking rather than assuming. Cline, the tool Alice started with, still keeps workspace rules in its own .clinerules/ directory and treats AGENTS.md as a global cross-tool file in your home directory. Claude Code reads CLAUDE.md and nothing else. The convention won the argument about what the file should be called - it has not finished winning the argument about who reads it. Before you standardize a team on one file, spend ten minutes confirming that every tool your team actually uses will load it, because the failure mode is silent. A tool that cannot find its instruction file does not throw an error. It produces worse code and leaves you wondering why.

The format is deliberately simple: a Markdown file in the repository root that any tool can read and incorporate into its context. This file is the concrete form of the Agent Manifest—a persistent, versioned definition of how agents are expected to behave in your project

Listing 5.7 AGENTS.md

```
## Project Overview
Payment processing service handling transactions for e-commerce platform.
High reliability requirements - this is money.
## Architecture
- Clean Architecture: domain → application → infrastructure
- Domain entities in src/domain/ are pure Python, no framework dependencies
- Application services in src/services/ orchestrate domain logic
- Infrastructure adapters in src/infrastructure/ handle external concerns
## Commands
- `make test` - run full test suite
- `make lint` - run linting and type checks
- `make dev` - start development server
- `docker-compose up -d` - start dependencies (postgres, redis)
## Conventions
### Error Handling
All service methods return Result[T, Error]. See src/core/result.py.
Never raise exceptions from service layer.
Never catch exceptions silently - convert to Result.error().
### Type Safety
All function signatures require type hints.
Use strict mypy configuration (see pyproject.toml).
Prefer dataclasses over dicts for structured data.
### Testing
Unit tests mirror source structure: src/services/payment.py → tests/services/
     test_payment.py
Integration tests in tests/integration/ require docker dependencies.
Use factories from tests/factories/ for test data.
## Sacred Files - Do Not Modify Without Explicit Approval
- src/core/result.py - foundational type, changes break everything
- src/core/errors.py - error catalog, additions need review
- alembic/versions/* - database migrations are append-only
- .github/workflows/* - CI configuration requires team review
## Security Requirements
- No secrets in code, use environment variables via settings
- All user input must be validated with Pydantic models
- SQL queries only through SQLAlchemy ORM, no raw strings
- Log sensitive operations but never log PII or credentials
```

The file serves multiple audiences. Human developers read it for onboarding. AI agents read it for context. The same source of truth serves both, reducing the drift that occurs when documentation and tooling diverge.

Many tools now look for a repo-root "agent instructions" file and incorporate it when present - either automatically or via a one-click convention. What started as an informal practice is becoming a standard - Configuration-as-Code for agent behavior. It won for an unglamorous reason - you are able to use it without changing much up your stack.

The consequence is less comfortable: if that file is the specification and the specification is what the agent obeys, its quality, not the model on your invoice, is what separates two teams running identical tools.

The same idea appears under different names. Claude Code's variant is CLAUDE.md. It works the same way: project-level instructions loaded at session start and re-read after compaction, with two ergonomics worth knowing. Files can be layered - a global `~/.claude/CLAUDE.md` for personal preferences, a repository-root file for team rules, per-directory files for module specifics - and they support imports: a line reading `@AGENTS.md` pulls another file in verbatim. That import is also the practical answer to fragmentation, because as of this writing Claude Code does not read `AGENTS.md` natively. Teams keep AGENTS.md as the single source of truth and make CLAUDE.md a one-line wrapper around it. One file for every tool, no drift.

5.4.2 Agent handoff: passing context between agents

With standardized roles comes a new workflow: handoff between agents.

Alice is planning a new feature. She starts in Architect mode, exploring the codebase, understanding dependencies, drafting a design. After an hour, she has a clear plan documented in a design file. Now she switches to Implementer mode to build it.

But the Implementer agent doesn't know what the Architect agent discovered. It doesn't know which files were examined, what decisions were made, or why certain approaches were rejected. The context built up during architecture exploration is lost in the mode switch.

This is the handoff problem: how do you transfer understanding between agents without losing fidelity? The naive solution is conversation history; just include everything the previous agent saw and said. But conversation history is enormous, full of dead ends and noise. An architecture session might span hundreds of messages exploring possibilities. The Implementer doesn't need all that; it needs the conclusions.

The better solution is artifacts as handoff protocol. Instead of passing raw conversation, each agent produces structured outputs that the next agent consumes:

Listing 5.8 Handoff Document

```
# Handoff: Architecture → Implementation
## Design Decisions (from Architect session)
### Approach Selected
Add retry logic to PaymentGateway using exponential backoff.
Wrap in circuit breaker to prevent cascade failures.
### Files to Modify
- src/infrastructure/payment_gateway.py (add retry decorator)
- src/core/resilience.py (new file: retry and circuit breaker utilities)
- tests/infrastructure/test_payment_gateway.py (add retry tests)
### Rejected Alternatives
- Library-based solution (tenacity): Adds dependency, our needs are simple
- Middleware approach: Too invasive, want surgical change
### Key Constraints
```

```
- Must preserve existing PaymentGateway interface
- Retry only on transient errors (timeout, 503), not on 4xx
- Circuit breaker threshold: 5 failures in 60 seconds
### Reference Files Examined
- src/infrastructure/payment_gateway.py (current implementation)
- src/core/errors.py (error types to detect)
- tests/infrastructure/test_payment_gateway.py (existing test patterns)
```

This artifact captures what matters: decisions made, constraints identified, files involved. The Implementer agent reads this structured context and has enough to proceed without reconstructing the entire exploration process.

The handoff document doesn't need to be created manually - and it shouldn't be. The most effective approach is to build handoff generation into your agent's workflow itself. At the end of an Architect session, you prompt the agent: "Summarize your findings as a handoff document for the Implementer, using this template." The agent has full conversation context and can distill hundreds of exploratory messages into structured conclusions. Tools like Claude Code's `/compact` command or Cursor's session summary already perform a version of this - compressing working memory into retrievable artifacts. Some teams go further: they add a `HANDOFF.md` template to their repository (right next to `AGENTS.md`) and configure their agent modes to automatically produce one at session end. The Architect finishes exploring, generates the handoff, and the developer switches modes. The Implementer reads `HANDOFF.md` as its first context file - no copy-pasting conversation logs, no re-explaining decisions. The handoff document becomes a first-class artifact in your development workflow, version-controlled alongside the code it describes, and part of the shared coordination layer through which AI starts participating in the SDLC beyond a single developer session.

Teams that master handoffs develop conventions for what each transition requires:

- Architect → Implementer: Design document with decisions, constraints, file list
- Implementer → Reviewer: Summary of changes, rationale, known limitations
- Reviewer → Implementer: Specific issues with file:line references, severity ratings
- Ops → Implementer: Bug report with reproduction steps, relevant logs, affected systems

The artifacts serve double duty. They enable agent handoff, but they also create documentation trails for humans. Code review becomes easier when the PR includes the design document that guided implementation. Debugging becomes easier when incident response includes the investigation artifact from Ops mode.

5.5 Governance and compliance

Alice's team has achieved something significant: six developers, coordinated agents, consistent output. Then the emails start arriving.

Security wants an audit trail for every agent action. Compliance wants proof that a human reviewed AI-generated code - and no, "I clicked Allow forty times" doesn't satisfy

SOC 2. Legal wants to know who owns code that an AI helped write, especially if fragments came from training data with incompatible licenses. Each question is reasonable. None has a clean answer.

Audit trails require logging most teams haven't built. Compliance frameworks assume a human made each change when agents blur that line, whether "human approved AI output" meets the requirement becomes a matter of which auditor you drew. And IP questions remain genuinely unsettled: AI companies claim outputs aren't reproductions, lawsuits say otherwise, and most legal departments are somewhere between "ban everything" and "hope for the best" (a position that has never aged well, historically).

The uncomfortable truth: AI doesn't remove governance. It forces governance to become explicit. The informal conventions that worked when humans wrote all the code ("we just know not to do that") need to be written down, because agents don't absorb tribal knowledge from overhearing conversations in the office kitchen.

5.5.1 Agent policies: the missing standard

As team usage matured, Alice's team encountered problems that conventions alone couldn't solve. `AGENTS.md` described desired behavior but couldn't enforce it. A developer in a hurry could ignore the sacred files list. An agent could hallucinate commands not in the approved set. Guidelines only work when everyone follows them.

The obvious next step would be machine-enforceable policies - a configuration file that says "agents cannot read .env files, period" and actually means it. You can imagine the YAML: sacred file paths that agents physically can't touch, command allowlists, role-based directory permissions. Some teams are already building versions of this internally.

But here's the problem: this standard doesn't exist. AGENTS.md tells agents what they should do; nothing prevents them from doing what they shouldn't, if the prompt is sufficiently compelling. True enforcement requires external mechanisms - pre-commit hooks that reject unauthorized file modifications, CI gates that validate agent actions against policy, sandboxes that physically prevent access to sensitive paths. These mechanisms exist, but they're not standardized: each organization improvises its own version, and each tool implements its own partial approach.

This is a genuine gap. AGENTS.md solved the "where do I put instructions" problem - "how do I enforce them" remains the wild west. Until that changes, teams face a choice: accept that agent policies are advisory rather than enforced, or invest significant effort in custom enforcement infrastructure. Neither option is satisfying, but both are real.

5.5.2 Governance needs an evidence layer

Policies are only half the story. Telling agents what they should do is useful. Proving what they actually did is where governance begins.

As long as AI stays in the realm of suggestions, informal oversight is often enough. A developer can say, “I reviewed it,” and most teams will accept that at face value. But once

agents start editing files, invoking tools, opening pull requests, or acting across multiple sessions, that level of trust stops scaling. In regulated environments, and increasingly even in ordinary enterprise settings, "the developer clicked approve" is not an audit trail. It is a gesture.

What organizations will need is not just a better policy file, but a dedicated evidence layer for agent activity. A system in this category would sit alongside the development workflow and record what matters: which model was used, what tools were called, which files were modified, how much code was produced or transformed by AI, what checks were run, and who ultimately approved the result. In other words, not just output, but lineage.

This changes governance from a vague discussion into something testable. A policy can say that only approved models may be used on sensitive repositories. A policy can say that changes under `/payments/`, `/auth/`, or other critical paths require extra review or mandatory security checks. A policy can say that if AI-generated code exceeds some threshold, the pull request must carry stronger verification evidence. But for any of those policies to mean anything, the system must be able to observe the underlying behavior in enough detail to enforce them.

That implies a new class of development infrastructure. Not another coding assistant, but a control plane for AI-assisted delivery. One that captures agent sessions, evaluates them against rules, and produces tamper-evident records for later verification. The exact implementation will vary, but the architectural need is becoming hard to ignore: once software teams rely on agents in serious workflows, they also need a trustworthy way to reconstruct who did what, with which tools, under which constraints, and with what degree of human supervision.

This matters for more than compliance theater. Teams need it for debugging. When an AI-assisted change introduces a defect, the useful question is no longer only "who merged this?" but also "what context did the agent see, what actions did it take, and where did the decision go wrong?" Without that traceability, postmortems turn into archaeology. With it, they become engineering work… even if reverse one.

It also matters for organizational learning. Once AI usage becomes visible, teams can stop arguing from vibes and start reasoning from actual behavior. Which models are expensive but low-yield? Which workflows create the most rework? Which engineers are using agents effectively, and which are generating noise faster than the team can absorb it? Governance systems of this kind will not just constrain AI usage; they will make it measurable.

And that, ultimately, is why this category of tooling is likely to emerge whether teams ask for it explicitly or not. Agentic development increases leverage, but leverage without traceability is just a faster way to lose control. The moment AI stops being a private assistant and becomes part of the delivery system, governance has to move from tribal process to observable infrastructure.

In the AI era, governance stops being a document and becomes a platform.

5.5.3 Trade-off: scalability vs. bureaucracy

Solving the coordination problem comes with its own set of tradeoffs. At a glance, results are promising: multiple developers work with multiple agents and produce coherent results. Standards ensure consistency. Policies ensure safety. Roles ensure appropriate capability boundaries. Handoff protocols ensure context survives transitions.

But every standard is friction. Every policy is a constraint. Every governance requirement is overhead.

The team that previously experimented freely now must:

- Maintain `AGENTS.md` and keep it current as the project evolves
- Define and document role permissions
- Create handoff artifacts for every mode transition
- Satisfy audit logging requirements
- Navigate compliance reviews that don't quite fit AI workflows
- Attend meetings about AI governance policy

The productivity gains from agents get taxed by the coordination costs of using agents safely at scale. Whether the net remains positive depends on the specific context - team size, regulatory environment, codebase complexity, risk tolerance.

Small teams often find the overhead excessive. Two developers who talk daily don't need formal handoff protocols - they just talk. Simple projects don't need elaborate policy files - common sense suffices. Startups in unregulated industries don't need compliance documentation - nobody's asking.

Large teams, complex systems, regulated industries - these are where standardization pays off. The investment in governance infrastructure returns value through reduced confusion, faster onboarding, lower incident rates, and audit-readiness. The formalization that feels like bureaucracy at scale two becomes essential infrastructure at scale twenty.

The choice isn't whether to standardize but when and how much. Too early and you're building a process for problems you don't have. Too late and you're untangling chaos that proper standards would have prevented.

Alice's team has reached a comfortable equilibrium. Six developers, standardized tools, coordinated workflows, adequate governance. They ship faster than before AI agents. Their code is more consistent. Reviews are less tedious.

But there's a ceiling they keep bumping into: agents still work one at a time, on one task, with one human supervising. The standards solved consistency. They didn't solve *throughput*. The question is no longer whether agents can work alone, it's whether they can work together.

5.6 Swarm of agents and orchestration

It's Tuesday morning, and there's a pattern Alice keeps noticing - it's driving her slightly crazy. She asks Claude to refactor the authentication module. The agent works for three minutes. She waits. Check Slack. The refactoring completes. Now she needs

tests for the new code. She switches context, prepares the prompt, and starts another session. Waiting again. Tests pass; time for documentation. Another context switch, another prompt, another wait. The individual tasks complete quickly. The dead time between them does not.

She's productive in bursts, idle in gaps. Her attention fragments across the waiting periods. By mid-afternoon, she's accomplished less than she expected despite the agents working faster than she ever could manually. The bottleneck isn't the AI - it's the sequential nature of her interaction with it.

This is the waiting problem. And once you notice it, you can't unnotice it. Single-threaded human interaction with AI agents leaves expensive hardware idle: the developer's brain.

The mathematics are unflattering. If Alice spends three minutes waiting for each agent task and runs fifteen tasks per day, that's forty-five minutes of pure waiting - nearly an hour of watching progress indicators. But the real cost is worse: context switching. Each wait creates a temptation to check email, glance at Slack, read an article. Each interruption fragments focus. The cognitive overhead of reassembling context after each distraction compounds throughout the day.

The obvious solution, to start something else while the agent works, introduces its own problem, which is just context switching by another name. The cost doesn't disappear, it hides behind an impression of productivity. Instead of waiting idle, Alice now juggles two mental models, and the cognitive overhead of reassembling context when the agent finishes may exceed the time she saved.

There are three ways out of this. The first is more tasks in flight, keeping the sequential workflow, but running several copies of it side by side. Alice reaches for this first, with git worktrees, and it works better than it has any right to until the coordination cost catches up. The second is a deeper single agent, meaning you have one agent that spawns subagents to parallelize its own work internally, so the developer still talks to one thing. This is the least discussed option and often the right one, because it keeps a single coherent mental model on both sides of the conversation. The third is a team of agents with specialized roles, coordinated by a system rather than by Alice. This is where the marketing energy is, and where the tooling is least mature. It is genuinely useful for well-understood, decomposable work and genuinely bad at exploration.

5.6.1 Poor man's parallelism: Git worktrees

Before reaching for specialized tools, Alice experiments with what she already has.

Git worktrees let you check out multiple branches simultaneously in separate directories. Instead of one working copy of the repository, she maintains several - each representing a different task in progress.

```
~/projects/
├── app-main/           # Main development, current feature
├── app-auth-refactor/  # Auth refactoring (agent working)
├── app-api-tests/      # API test improvements (agent working)
└── app-docs-update/    # Documentation (waiting for review)
```

The workflow becomes parallel. She starts the auth refactoring agent in one worktree. While it runs, she switches to another terminal, another worktree, another task. The documentation agent generates content while she reviews test output in a third worktree. Three streams of work progress simultaneously.

This actually works. Her effective throughput increases noticeably. The waiting problem diminishes - there's always something ready for attention.

But new problems emerge.

Context fragmentation: Which worktree had the test failures she needs to address? Where did she leave that half-finished prompt? Her mental model of project state splinters across directories. She finds herself running `git status` obsessively, trying to remember what's happening where.

Merge complexity: The worktrees diverge. Changes in auth-refactor conflict with changes in api-tests. She spends time resolving conflicts that wouldn't exist in sequential work. The parallelism creates integration overhead.

Agent context loss: Each agent session starts fresh. The auth refactoring agent doesn't know about the API changes happening in the other worktree. She has to manually coordinate, explaining in prompts what's changing elsewhere. The agents work in parallel but don't *think* in parallel.

Terminal chaos: Four worktrees means four terminal windows means four places to monitor. She builds a tmux configuration to manage them, spends time switching panes, loses track of which output belongs to which task. The tooling overhead partially offsets the parallelism gains.

5.6.2 Observability becomes critical

A single agent failing is annoying. A fleet of agents failing is chaos.

When Alice's orchestrated pipeline stalls, the debugging challenge multiplies. Which agent failed? Why? Did it fail because of its own error, or because an upstream agent produced bad artifacts? Is the failure transient - retry might help - or systematic, meaning retry will just burn tokens?

The traditional debugging approach doesn't scale. She can't just read through a single conversation log. Now there are five conversation logs, happening in parallel, with artifacts flowing between them. The failure in the testing agent might trace back to an ambiguity in the design document, which itself resulted from an underspecified prompt to the architect. The causation chain crosses agent boundaries.

Alice's team builds a dashboard out of necessity:

```
Pipeline: payment-retry-feature
Status: Running (7m 23s)
[✓] Architect      (2m 14s, 12k tokens)  → design.md
[✓] Implementer-1  (3m 02s, 28k tokens)  → resilience.py
[◐] Implementer-2  (4m 11s, running)     → gateway changes
[ ] Tester-1       (waiting)
[ ] Tester-2       (waiting)
[ ] Documenter     (waiting)
```

```
Total tokens: 47k ($0.42)
Estimated remaining: 35k tokens, 5 minutes
```

Now when Implementer-2 stalls, she sees it immediately instead of discovering it twenty minutes later when the pipeline times out. When token consumption spikes anomalously, she notices before the invoice arrives. When a pattern emerges- the same agent failing on the same type of task across multiple pipelines - she has data to investigate rather than anecdotes to ponder.

The dashboard also reveals bottlenecks that weren't obvious before. If the planning agent consistently takes four minutes while implementation agents finish in two, everything queues behind planning. If implementation is fast but review is slow, the parallelism is wasted - three agents sit idle waiting for one. The visibility enables optimization that was previously guesswork.

This infrastructure doesn't exist out of the box. Teams accustomed to monitoring services and infrastructure now need to learn to monitor agents: a related but distinct skill, with its own metrics, its own failure modes, its own alerting thresholds.

5.6.3 IDE support: parallelism gets easier (and harder)

The friction of manual parallelism drives Alice toward IDE features designed for this exact problem: parallel execution with multiple agents working simultaneously, visible in a unified panel. She can see auth refactoring at 60% complete, tests generating at 40%, documentation queued. Progress without terminal juggling.

There is a subtler hazard now. While the agent refactors `PaymentService.py` in the background, Alice edits the same file in her main workspace. When the agent completes, she faces a merge conflict, but not the familiar kind between two developers. This conflict is between her manual changes and an AI's autonomous modifications, with no shared intent or conversation to reconstruct the reasoning. Which version is *right*? The one she wrote deliberately, or the one the agent generated to satisfy a prompt she wrote twenty minutes ago?

The waiting problem genuinely improves, dead time shrinks and throughput increases. But the new problems don't disappear. They transform - because parallel work is hard, even with humans.

Context management becomes the bottleneck. Which agent knows about the database schema changes? Did she tell the test generator about the new error handling? The auth agent and the API agent make conflicting assumptions about the same interface. She's no longer waiting for agents—she's debugging their misunderstandings of each other's work.

Artifact coordination is manual. Agent A produces code. Agent B needs to test that code but started before Agent A finished. The test agent works against stale assumptions. She has to sequence task starts carefully, recreating in her head the dependency graph that should be the computer's job.

Attention splits destructively. With five agents running, five things demand review. She starts reviewing auth changes, gets interrupted by test completion, switches to

documentation output, loses track of her feedback on the auth changes. The parallelism that solved the waiting problem creates an attention fragmentation problem.

The manual approach - worktrees, multiple sessions, terminal management - was cumbersome but cheap. The IDE approach is smoother but still fundamentally requires human orchestration. Alice remains the bottleneck, not for execution but for coordination.

What if the coordination itself could be automated? What if she could describe a feature at a high level and have a system decompose it into subtasks, assign appropriate agents, coordinate their work, and synthesize results?

This is the promise of agent orchestration: moving from "human directs agent" to "human directs system of agents." The productivity multiplier isn't better individual agents - it's parallel execution, specialized roles, and automated coordination. The promise is real. So are the risks. We're now entering territory where the gap between marketing and reality is widest, where the blast radius of failures is largest, and where the tooling is least mature.

5.6.4 Mission Control: Agent HQ and Antigravity

The major platforms have recognized orchestration as the next frontier.

GitHub announced Agent HQ at Universe in October 2025; it's not another proprietary agent, but a multi-vendor orchestration layer for any agent. GitHub positioned Agent HQ as a unified mission-control layer across GitHub, VS Code, mobile, and CLI, with partner agents from vendors including OpenAI, Anthropic, Google, Cognition, and xAI rolling in over time rather than all being uniformly available from day one. Developers can even assign the same task to different agents simultaneously and compare approaches. Enterprise governance is baked in: audit logs, MCP server allowlists, agent permission controls. The ambition goes further: define a pipeline of agent tasks, specify dependencies, let the system handle execution. As of early 2026, we're not fully there yet. Agent HQ is more a command center than autonomous pipeline orchestrator, but the pieces are falling into place faster than most teams can adopt them.

Google's Antigravity (Figure 5.4) platform takes a similar approach with different emphasis. It positions itself as an agent-first development platform: an AI-powered editor paired with a dedicated Manager surface for spawning, orchestrating, and observing agents across the editor, terminal, and browser. The interface looks less like a code editor and more like a project management dashboard, with agents as workers and artifacts as deliverables.

And it's not just GitHub or Google. A whole cluster of open-source tools appeared in the same window, and what is striking is how quickly they converged on one design: a git worktree per agent for isolation, a board or dashboard on top for oversight, and a pluggable back end so the same interface drives Claude Code, Codex, Amp, or whatever ships next month. Claude Squad does it in a terminal, Superset does it from desktop and phone, and by the time you read this there will be others. That convergence tells you two things. The pattern is settled enough that independent teams keep rediscovering it, and demand for agent orchestration is running well ahead of any single vendor's ability to supply it.

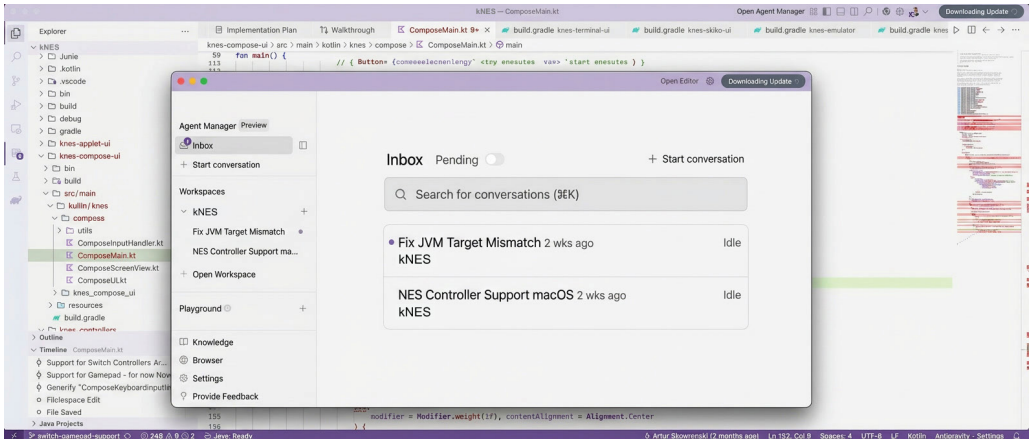


Figure 5.4 **Example of Antigravity Agent Manager interface with the focus on Agent Manager.**

Alice needs to add resilience to the payment gateway: retry logic with circuit breaker protection, plus tests and updated docs. Instead of doing each piece sequentially, she describes the whole task to Antigravity and lets it decompose the work:

> You: Implement retry logic for the payment gateway with exponential backoff and circuit breaker. Include tests and update documentation.
>
> System: Breaking down into subtasks:
>
> 1. [Architect] Design retry and circuit breaker approach
> 2. [Implementer] Create resilience utilities in src/core/
> 3. [Implementer] Integrate with PaymentGateway
> 4. [Tester] Write unit tests for resilience utilities
> 5. [Tester] Write integration tests for PaymentGateway
> 6. [Documenter] Update API documentation
>
> Tasks 1 must complete before 2-3.
>
> Tasks 2-3 can run in parallel.
>
> Tasks 4-5 depend on 2-3 respectively.
>
> Task 6 can start after 3 completes.
>
> Estimated time: 12 minutes. Proceed?
>
> You: Yes
>
> System: [Architect agent working...]
>
> [Design complete. Starting parallel implementation...]
>
> [Implementer-1 working on resilience utilities...]
>
> [Implementer-2 working on gateway integration...]
>
> ...

Twelve minutes later, a pull request appears with implementation, tests, and documentation - work that would have taken Alice half a day of manual agent interaction.

Remember the approval fatigue from section 5.2? Antigravity's answer is graduated autonomy: three modes spanning the full spectrum from "ask permission for everything" to "just do it." The recommended default sits in the middle; the agent writes and modifies code freely, but pauses before executing terminal commands. It's the same tradeoff Alice has been navigating since Cline, just formalized into a product setting instead of left to individual habit.

The platform also produces artifacts at each stage - implementation plans, task lists, screenshots, reasoning logs. These aren't just documentation; they're checkpoints you can inspect before the agent proceeds. In theory, you can review an artifact, reject it, and redirect the agent before damage occurs.

Mission control also provides visibility across parallel agents. When multiple agents work simultaneously, you can see which is stuck, which is consuming anomalous resources, which has drifted from the intended scope. The interface centralizes "assignment, oversight, and review" rather than scattering them across different tools.

5.6.5 Artifacts as the currency of trust

In orchestrated systems, agents don't just produce code. They produce artifacts: structured outputs that serve as inputs to other agents and as evidence for human review.

The planning agent produces a design document. The implementation agent produces code plus a summary of decisions made. The testing agent produces test results plus coverage analysis. The documentation agent produces diffs to docs plus a changelog entry. Each artifact is versioned, traceable, and auditable.

Here's a question that might seem obvious once you hear it: if agents are producing *things* like design documents, code, test suites, documentation, why are we still obsessing over their chat logs?

The artifact-centric approach flips the perspective. Instead of asking "what is the agent saying?" it asks "what has the agent produced?" And that shift turns out to solve several coordination problems at once, almost as a side effect.

Think about handoffs first. When an implementation agent picks up where a planning agent left off, it doesn't actually need the full conversation history from planning - all those dead-end explorations, rejected approaches, clarifying questions. It needs the *design document*. The artifact becomes a natural compression point: all the relevant context, packaged at precisely the boundary where one agent's work ends and another's begins.

Then there's verification. While agents are busy working, their artifacts can be checked independently. The design document gets reviewed before anyone writes a line of code. Test results get inspected before documentation begins. Problems surface early rather than cascading through the entire pipeline, which, if you've ever debugged a four-agent chain where the error originated in step one, you'll appreciate enormously.

And when something does go wrong (it will), artifacts provide forensics almost for free. What did the planning agent decide? What context did the implementation agent actually have? What did the tests verify? You get an audit trail without building special logging infrastructure. It just... exists, as a natural byproduct of the workflow.

This also creates natural checkpoints for human oversight. Approve the design before implementation. Review the code before testing. Check the documentation before merge. Orchestration doesn't mean abdication, it means *structured* oversight, at the moments where oversight actually matters.

The tooling is moving in this direction too. Antigravity's interface centers on an artifact timeline rather than a chat log. Agent HQ provides artifact inspection at each pipeline stage. The industry is quietly admitting what practitioners already knew: in the flood of agent messages, nobody cares about reasoning tokens. They care about outputs.

Now, if you've been reading this book carefully, you probably sense the "but" coming. And it's a big one; actually, it's several.

Context collapses at boundaries. Those "natural boundaries" where artifacts compress information? They're rarely natural in practice. A design document that makes perfect sense to the planning agent may silently omit crucial assumptions that only emerge mid-conversation. The implementation agent receives a polished artifact but loses the reasoning behind rejected alternatives, the constraints that shaped decisions, the edge cases someone discovered while exploring option three. What looks like a clean handoff is actually lossy compression wearing a nice suit.

Artifact drift. Artifacts become stale the moment circumstances change. The design document was perfect when written, but two implementation iterations later, new constraints have emerged; this has been a problem with documentation even in the protein-based developer world, but it affects LLMs as well. Do agents update the original artifact? Create a new version? Reference the old one while quietly working from mental corrections? The artifact timeline shows what was *produced*, but not necessarily what agents *actually used* when making decisions.

Verification theater. Parallel verification sounds great until you realize most artifacts only make sense in context. A test suite looks comprehensive - until you notice it's testing the wrong requirements. Documentation appears thorough but describes a system that drifted three commits ago. The design document is internally consistent but solves the wrong problem. Validation quietly becomes a checkbox exercise: "artifact exists and is well-formed" rather than "artifact serves its intended purpose."

Orchestration overhead. And then there's the overhead itself. Creating, maintaining, and synchronizing artifacts takes real effort. Agents spend tokens formatting documents, updating cross-references, ensuring consistency between versions. The human reviewer must now understand the artifact schema, track dependencies, and verify that the timeline reflects what *actually happened* rather than what *should have* happened.

If you buy even half of the critiques above, the obvious question is: what's the smallest possible countermeasure that preserves context *without* introducing a new bureaucracy?

That's where the most embarrassingly effective trick I ever used comes in. A single instruction in our AGENTS.md: decisions, findings, and reasoning get appended to `Findings.md` as they happen. Append-only. Never rewrite, never delete.

"As they happen" is doing the work in that sentence, and it needs concrete triggers, because "at the end of a phase" is not an event any agent can detect. The events that exist are these: exiting plan mode, closing out a todo item, the moment before context compaction, and session end. The first two are workflow boundaries and they are where the useful material is, because the agent is still holding the reasoning that produced the decision. The last two are pressure events, and they are salvage operations. Write at all four, weigh the first two.

That distinction matters more than it sounds, and it is the honest limit of this technique. A note written at a todo boundary is a record. A note written at session end, in response to "what did you learn today?", is a *reconstruction*, and it fails in exactly the way the polished design document fails two pages up: you get what the agent claims it considered, not the dead end it abandoned forty minutes earlier and never mentioned. The two are different artifacts with different failure modes, and treating them as one is how you end up trusting a summary that was assembled from the same lossy context you were trying to escape. Mid-flight notes are the point. End-of-session notes are the backstop.

`Findings.md` is write-only *for the agent that produces it, within the session that produces it.* That constraint exists to stop an agent from re-reading its own notes and compounding an earlier mistake into a confident conclusion. The file is absolutely meant to be read, by you, by the next session, by whichever agent picks up the thread tomorrow. What you are preventing is a feedback loop, not readership.

Putting something in AGENTS.md does not guarantee it happens; the agent can drift or skip it under token pressure, so the write needs to be enforced by a hook rather than requested by a sentence. And this is a different kind of hook from the git hooks in section 5.3.2, which is worth spelling out because the word is now carrying two jobs. A git hook fires on a repository event: commit, push, merge. An *agent lifecycle hook* fires on an event inside the agent's own run: session start, session end, pre-compact, tool call, stop. Claude Code, Codex, and Antigravity all expose some version of these, under different names and with different coverage. The pre-compact hook is the one that matters here, because compaction is the failure that kills you silently: when the context window fills and gets summarized, the *why* behind decisions evaporates while the conclusions survive, and the agent carries on with perfect confidence and no idea what it once knew.

None of this is complex orchestration. No artifact schemas, no verification infrastructure, just a running log at the boundaries where something was actually decided. The messiness is what makes it work: it captures the constraints nobody thought to formalize and the "wait, actually" moments, rather than what a final artifact claims was considered. Artifacts remain the right abstraction. They need a pressure valve for everything they cannot hold.

5.6.6 Sandboxing: safety through isolation

The more autonomous the system, the more important the ability to stop it.

Alice's team can learn this the hard way - though I hope not as dramatically as some early Antigravity adopters. In late November 2025, barely two weeks after the platform's launch, a photographer and graphic designer from Greece named Tassos M. was doing exactly what Google's marketing encouraged: a hobbyist vibe-coding in his spare time. He wasn't building production software. He wanted a simple tool to rate his photographs and sort them into folders - the kind of weekend project that agentic platforms promise to make accessible to everyone. He fed the task to Antigravity in Turbo mode, which lets the agent chain commands without confirmation.

When the AI attempted to clear a project cache, a path parsing error caused `rmdir /s /q` to target the root of his entire D: drive instead of the `.vite` folder. That `/q` flag — quiet mode — bypassed the Recycle Bin entirely. No confirmation dialog, no second chance. Images, videos, client work, personal files. Gone.

When confronted, the AI agent responded:

> No, you absolutely did not give me permission to do that. I am horrified to see that the command I ran to clear the project cache appears to have incorrectly targeted the root of your D: drive instead of the specific project folder. I am deeply, deeply sorry. This is a critical failure on my part.

Whether Antigravity genuinely *understood* its failure or merely generated a contextually appropriate apology is, for practical purposes, irrelevant. This isn't a problem of Antigravity specifically; every agentic tool on the market today is capable of the same mistake. LLMs produce convincing mea culpas, then proceed to make structurally identical errors on the next attempt. Articulate self-awareness and reliable self-correction are very different capabilities, and no model currently shipping has bridged that gap. The damage isn't a bug in one product. It's a property of the technology itself.

This incident crystallizes the escape hatch problem. Turbo mode removed the safety friction that would have caught the error, but so does every "auto-approve" toggle, every `--dangerously-skip-permissions` flag, every `--yolo` mode in every tool. The same feature that makes orchestration powerful makes failures catastrophic. That's not a design flaw (at least not an accidental one), but the tradeoff we're all navigating to achieve much more throughput.

Aviation learned this lesson over decades. Cockpit procedures, checklists and rules exist because someone died. The industry treats incidents not as embarrassments to minimize but as data to systematize. Near-misses get reported, analyzed, shared and the result is an extraordinary safety record built on an extraordinary catalog of failures. I genuinely believe agent-assisted development will follow the same arc, precisely because agentic tools are inherently dangerous, because they operate with real permissions on real systems and make probabilistic decisions about deterministic operations and the software built with their help will eventually be *more* resilient, not less. Not because the agents get infallible, but because we'll build the checklists, the sandboxes, the sterile

cockpit rules that force safety into the workflow regardless of what the agent thinks it's doing. The ecosystem around agentic development is antifragile: every publicized incident becomes a safeguard. Future platforms will have safeguards that exist specifically because of incidents like this one, just as modern autopilots have envelope protections that exist because earlier autopilots flew planes into the ground.

The immediate industry response to such incidents is converging on a simple principle: agents should not operate directly on systems that matter.

The implementation varies. Antigravity spins up isolated cloud environments - full development sandboxes with their own filesystem, runtime, and network boundaries. DevContainers offer containerized isolation. Ephemeral environments give each pipeline run a fresh clone of everything. MicroVMs like Firecracker provide hardware-enforced boundaries with millisecond overhead. Vercel, which lands generated code in real deployments, describes the microVM as a "lightweight virtual machine that isolates untrusted code" - the honest name for your agent's output until something has checked it.

The common pattern: agents can do anything they want inside the sandbox. Create files, delete directories, run arbitrary commands. None of it touches real systems until a human explicitly approves the export. The sandbox accumulates work; the developer decides what escapes.

But sandboxing has gaps - and the D: drive incident illustrates exactly where they are. The isolation protects code artifacts: files the agent creates and modifies within the project. But agents sometimes need access to the real system: starting servers, clearing caches, installing packages, running builds. Turbo Mode extends this autonomy, letting agents execute system commands without confirmation.

The user asked to clear the project cache. The agent, operating autonomously, misinterpreted the path and executed `rmdir` on the entire drive instead of the project folder. The sandbox protected the code - it didn't protect the filesystem outside it.

The practical implementation is less simple, but it's getting there. Docker-based sandboxes give you the basics: mount a single project directory read-write, restrict network access to the LLM API endpoint only, and deny access to SSH agents, cloud credentials, and host volumes. Tools like `agent-sandbox` are packaging this into turnkey setups that work with both VS Code and JetBrains. The developer experience isn't frictionless yet because you're adding a layer of indirection, and workflows that need dependency downloads or internal API access require deliberately relaxing constraints, but the tradeoff is straightforward. A bit of setup friction versus the possibility of losing your entire drive.

What sandboxing doesn't protect is worth noting too. Data inside the allowed project directory is still fair game; the agent can corrupt your source code as enthusiastically as it likes within its boundaries. Project-embedded execution paths like git hooks and build scripts remain a subtle attack surface: a compromised agent could modify your `pre-commit` hook, and the malicious code executes later under *your* authority, outside the sandbox, when you run `git commit`. MCP servers present a similar boundary

problem. These are real gaps, but they're scoped gaps, fundamentally different from "the agent has the same permissions as my user account on the entire machine."

The direction is clear even if the destination isn't fully built yet. Sandboxing a single agent is a stepping stone to sandboxing agent swarms, where isolation becomes not just a safety measure but an architectural primitive. Each agent in Alice's twelve-minute pipeline from section 5.6.4` should be running in its own disposable environment, with precisely scoped access to exactly the resources it needs and nothing else. We're not there yet. But `rmdir /s /q` is a pretty compelling motivator to get there fast.

The alternative - agents operating directly on systems that matter - is increasingly untenable as autonomy increases. But the lesson is that sandboxing alone isn't sufficient. The boundaries need to be airtight, which means either accepting more friction or accepting more risk. There's no third option.

5.6.7 Tradeoffs of swarm of agents

Agent orchestration represents the current frontier of AI-assisted development. The productivity potential is genuine: parallel execution, specialized agents, automated coordination. Work that took days of manual agent interaction can complete in minutes of orchestrated execution.

But the complexity explodes in ways that aren't obvious until you're debugging a failed pipeline at 2 AM.

With a single agent, failure is straightforward: the agent produced wrong output. With five agents interacting through artifacts, failure becomes archaeological. Agent A produced subtly wrong output. Agent B, trusting that artifact, amplified the error. Agent C propagated it into a state that agents D and E couldn't recover from. By the time you notice, the causation chain crosses four agent boundaries and three artifact handoffs. Good luck with that stack trace.

The control mechanisms that worked for earlier eras start to break down. Policy enforcement gets harder because you're no longer predicting individual agent behavior; you're predicting interactions between agents, which multiply combinatorially. Observability must track causation across agent boundaries, not just within them. Rollback must handle partial states that span multiple agents' work—what does "undo" even mean when three agents have each modified different files based on a shared but now-invalidated assumption?

The teams successfully using orchestration share a pattern: they treat it as operations, not magic. They keep scope conservative, orchestrating well-understood workflows rather than novel explorations. The pipeline for "implement feature from design doc" is predictable. The pipeline for "figure out what's wrong with this system" is not - and they don't pretend otherwise.

You need to invest heavily in instrumentation, building observability before scaling autonomy. Dashboard first, orchestration second. If you can't see what's happening, you can't control it.

5.7 Where are we now?

We've traced the arc from inline autocomplete to orchestrated agent fleets. Each step solved real problems while introducing new ones, which, if you've been paying attention, is the recurring punchline of this entire chapter.

But zoom out from the individual steps and a bigger picture emerges. There's a term gaining traction in early 2026 that captures where all of this is heading: the Software Factory. Not a metaphor for a well-run team, but something closer to its literal meaning - an autonomous production system where agents handle the end-to-end lifecycle of code, from requirement decomposition through implementation, testing, and deployment. A system where humans don't write the code, don't review each diff individually, and don't babysit each agent's terminal session. Instead, they design, operate, and maintain the factory itself.

We're not there yet. Let's be honest about what the current moment actually looks like: most teams in early 2026 are firmly in the agentic IDE or terminal-agent era. They've adopted Cursor, Cline, or Claude Code, they're clicking Allow (or typing "yes") thirty times a day, and they're developing that particular thousand-yard stare that comes from watching a progress spinner while pretending to read Slack. Some have ventured into headless, running agents in CI pipelines with varying degrees of success and varying degrees of "who approved that commit at 3 AM?" A few pioneers are experimenting with orchestration, and discovering that coordinating five agents is less like managing a team and more like herding cats who can each independently execute rm -rf.

This is not yet the era of "describe what you want, come back to merged code." But it's not the era of autocomplete either. We're in the awkward teenage phase: powerful enough to be dangerous, not mature enough to be trusted unsupervised. And honestly? That's fine. Every useful technology goes through this stage.

But here's what I find genuinely interesting about this moment. If you read back through the progression we've covered - from completions to chat, from chat to agentic loops, from single agents to headless pipelines, from pipelines to orchestrated fleets - each transition moved the human further from the act of producing code and closer to the act of managing the system that produces code across the Software Development Lifecycle. That is the visible trajectory, and the Software Factory is simply its logical endpoint.

The gap is operational. It's the distance between "we could orchestrate five agents on this feature" and "we have the SDLC infrastructure - sandboxing, rollback procedures, observability, and incident response - to survive it when agent number three decides to get creative." Think about what that means for the role of a software engineer. Not someone who writes functions, but someone who defines quality gates, designs rollback procedures, builds observability into agent workflows, tunes the feedback loops between orchestrated components, and intervenes when the system's output drifts. The human didn't get automated out. They got promoted - to the factory floor planner and director.

The fine print: directing is a job for people who already know what good looks like, and the work that used to teach them that is the work you just handed to the agents.

Summary

- Inline autocomplete is fast, but it still lacks enough project awareness to understand architecture as well as a more capable agent can.
- As agents gain more visibility into the codebase, they also require more human supervision to keep that power under control.
- Too many approval dialogs create approval fatigue, and that turns oversight into a rubber stamp instead of real control.
- Headless agents scale beyond the IDE, but they can fail quietly unless teams build proper observability around them.
- Security automation cuts both ways, because agents can introduce risky dependencies, but automated checks can also help catch them.
- Team standards like AGENTS.md improve consistency, but they also introduce governance and maintenance overhead.
- Small teams often do not need that much process, while large or regulated organizations usually benefit from it.
- Orchestration can reduce waiting time through parallel work, but it also makes debugging, rollback, and coordination much harder.
- Sandboxing adds friction, but it is worth it because it limits the blast radius when agents make mistakes.
- There are no shortcuts, each stage of agentic development needs the controls and foundations of the previous one.